

RETENIA · NOMBRE DE TRABAJO

Investigación completa y plan maestro

Todo lo que hay que saber para construir un software de escritorio de aprendizaje y memoria continua: rutas estilo Duolingo generadas con IA desde tus propias fuentes, un sistema de memoria con repetición espaciada de última generación y ejercicios interactivos, con múltiples IAs por API.

Preparado para

Thomi (Pilar, Buenos Aires)

Stack decidido

Electron + React + TypeScript · Windows 11 primero · local-first

Fecha de la investigación

1-2 de septiembre de 2026

Documento compañero

prompts.md — fase 0 de setup + 14 fases de desarrollo (76 sub-fases) con modelo y esfuerzo para Claude Code

Índice

1. Resumen ejecutivo	5
Qué vas a construir	5
Las decisiones que ya están tomadas	5
Por qué nadie lo tiene todavía	6
Cómo usar este documento y su compañero	6
Advertencias honestas	6
2. Visión del producto y decisiones de alcance	7
Los cinco focos, como requisitos	7
Decisiones tomadas en las rondas de preguntas	7
Alcance de la v1 y no-objetivos	9
Principios de diseño que gobiernan todo	9
Propuestas de nombre	9
3. Panorama competitivo: qué existe y qué falta	11
RemNote 1.28 — el competidor más cercano	11
SuperMemo 20 — las ideas avanzadas que nadie copia	11
Anki — el estándar de portabilidad	12
Duolingo — la referencia de motivación	12
Gemini Notebook (ex NotebookLM) y la ola de "sube y genera"	12
Idiomas, PKM y lectura: qué tomar de cada uno	12
Matriz comparativa (resumen)	13
Quince diferenciadores que nadie ha combinado	13
4. Ciencia del aprendizaje aplicada al diseño	15
Qué dice la evidencia (y qué implica)	15
Las 20 reglas de Wozniak como reglas para el generador de tarjetas	15
Cuándo NO usar flashcards	16
Marcos de diseño instruccional codificados en los prompts	16
Tipo de contenido → actividades → ítems de memoria	17
5. El sistema de memoria	18
Decisiones	18
Fundamentos: curva de olvido y modelo de dos componentes	19
FSRS-6: fórmulas completas para implementar	19
Importancia por ítem: diseño	21
Exámenes con fecha	22
Ejercicios interactivos en el mismo scheduler	23
De la ruta de aprendizaje al sistema de memoria	24
Composición de la sesión diaria	24
Métricas que la app debe mostrar	24
Modelo de datos del sistema de memoria	25
Notificaciones y ciclo de vida del optimizador	26
6. Catálogo de actividades interactivas	27
Lo que se aprendió de cada catálogo	27
Primitivas, envoltorios y calificadores	27

Tabla maestra: 98 tipos	28
Reglas para el repaso espaciado	31
Priorización	31
Esquema JSON: envoltorio común + 22 familias de payload	31
Arquitectura del "activity engine" (React)	33
7. Generar rutas de aprendizaje con IA	35
Lo que enseñan los productos de referencia	35
El pipeline en 10 etapas	35
Anatomía de una lección (Gagné + Merrill + CLT)	36
Costo de generar una ruta desde un libro de 300 páginas	37
Determinismo, versionado y multi-fuente	37
Esquemas (extracto)	38
Los 11 prompts del pipeline	38
Diagnóstico de conocimientos previos	39
Política de remediación	41
Calificación de respuestas libres	41
UX del flujo "Generar con IA"	42
8. Ingesta de fuentes y RAG local	43
Estrategia por tipo de fuente	43
OCR y parsing cloud: precio por 1.000 páginas	44
Embeddings, rerankers y vector store	44
Diseño del RAG local	45
Costo mensual de ingesta (usuario intensivo)	45
9. Inteligencia artificial: proveedores, precios y costos	46
Modelos de texto: la foto de septiembre de 2026	46
Matriz de proveedores por tarea	47
Costo mensual de texto (usuario intensivo)	47
Structured outputs, AI SDK 7 y abstracción de proveedores	48
Modelos locales en la RTX 4070 Super (12 GB)	48
Privacidad, límites y Argentina	49
Voz: evaluación de pronunciación	49
Voz: STT, TTS y tiempo real	50
Imagen, diagramas y video	51
Presupuesto mensual consolidado	52
10. Arquitectura técnica	53
Stack recomendado	53
Monorepo	55
Fundación Electron	55
Capa de datos	56
Procesamiento en segundo plano y sidecars	57
Sandboxes de código	57
Importación y exportación	57
Calidad, CI y release	58
Riesgos técnicos y mitigaciones	58

Esfuerzo estimado	58
11. Experiencia de usuario y gamificación	60
Principios de UX	60
Mapa de pantallas	60
El camino, en detalle	61
Gamificación con evidencia	61
Detalles de plataforma esperados en 2026	62
12. Catálogo maestro de funciones	63
1 · Fuentes e ingesta	63
2 · Generación de rutas	63
3 · Lecciones y teoría	63
4 · Práctica y actividades	64
5 · Memoria y programación	64
6 · Exámenes	65
7 · Idiomas y habla	65
8 · Notas y base de conocimiento	65
9 · Tutor IA / chat	65
10 · Generación multimedia	66
11 · Gamificación	66
12 · Estadísticas	66
13 · Importación / exportación	67
14 · Ajustes y plataforma	67
15 · Comercial (después)	67
13. Plan de desarrollo y uso de prompts.md	68
Las fases de un vistazo	68
Las 76 sub-fases, con modelo y esfuerzo	68
Mapeo de modelos y esfuerzo a Claude Code	70
Cómo usar prompts.md	71
Configuración de Claude Code que la fase 0 deja lista	72
14. Costos, riesgos y decisiones abiertas	73
Costos fijos de desarrollo y distribución	73
Costo operativo mensual de IA (resumen)	73
Riesgos principales	73
Decisiones abiertas (para tomar durante el desarrollo)	74
Qué re-verificar antes de fijar precios o depender de ello	74
15. Glosario y fuentes	76
Glosario	76
Fuentes principales	77

1 Resumen ejecutivo

Qué es Retenia, qué se decidió, cuánto cuesta y por qué nadie lo tiene todavía. Cinco minutos de lectura antes de entrar en el detalle.

Qué vas a construir

Retenia (nombre de trabajo) es un programa de escritorio para *aprender y no olvidar*. Combina tres cosas que hoy viven en productos separados: (1) **rutas de aprendizaje estilo Duolingo**, lineales y gamificadas, donde cada lección tiene una parte de teoría y otra de práctica; (2) esas rutas se **generan con IA a partir de tus propias fuentes** (libros PDF, cursos en video, links, DOCX, imágenes, audio), al estilo NotebookLM, y quedan fijas para que las completes; y (3) un **sistema de memoria** de última generación (FSRS-6, el mismo algoritmo que adoptaron Anki y RemNote) con niveles de importancia por ítem, exámenes con fecha, simulacros y, sobre todo, **ejercicios interactivos programados por el mismo scheduler que las flashcards**. Todo local-first, con tus propias claves de API y varias IAs especializadas (texto, voz, pronunciación, imagen).

USD 9–14

costo mensual estimado de IA para vos como usuario intensivo (texto), dentro del presupuesto de 20–50

≈ USD 3–6

generar una ruta completa desde un libro de 300 páginas (batch + cache)

98

tipos de actividad interactiva catalogados; 21 en el MVP

76

sub-fases en 15 fases (F0–F14), cada una con prompt, modelo y esfuerzo para Claude Code

Las decisiones que ya están tomadas

PLATAFORMA

Electron 44 + React 19 + TypeScript

Windows 11 primero, macOS después (Intel y Apple Silicon). Un solo motor Chromium para video, PDF, micrófono y WASM. Tauri descartado por WebKit en Mac y por el ecosistema Node de ingesta.

MODELO DE PRODUCTO V1

Uso propio, 100 % local, BYOK

Sin backend ni cuentas. Vos cargás tus claves (guardadas con `safeStorage`). El esquema de datos nace "sync-ready" (UUIDv7, soft deletes, outbox) para agregar cuentas, licencias y sync sin reescribir.

SCHEDULER DE MEMORIA

FSRS-6 vía `ts-fsrs`

21 parámetros optimizables por usuario, "desired retention" como palanca de importancia. SM-2 solo para importar. Modo examen construido como capa encima.

IA DE TEXTO

Claude Sonnet 5 + Gemini 3.7 Flash + local

Sonnet 5 (USD 2/10 por millón) genera rutas, lecciones y exámenes con batch y cache; Gemini 3.7 Flash corrige, chatea y procesa volumen; Ollama/LM Studio en tu RTX 4070 Super para embeddings y tareas livianas.

VOZ

Azure AI Speech (pronunciación) + Whisper local + Azure TTS / ElevenLabs

Único proveedor con evaluación de pronunciación en los 6 idiomas de la v1. USD 1,00/h (+0,30 con prosodia); 5 h/mes gratis. Acento irlandés vía voces en-IE y shadowing con curva de entonación.

MEDIA

Diagramas como código + Nano Banana 2 Lite + video "ligero"

Mermaid/Excalidraw/KaTeX para diagramas (centavos); imágenes a USD 0,034; video generativo solo opt-in (USD 8–60 por minuto real). Default: slides + TTS + Remotion.

RUTA FIJA CON REMEDIACIÓN**PathSpec congelada + mini-lecciones opcionales**

La ruta no cambia; la IA inserta desvíos de refuerzo (L07.r1) sin reenumerar y sube la prioridad en memoria. Diagnóstico adaptativo de 25–30 ítems para marcar lo que ya sabés.

GAMIFICACIÓN**Completa estilo Duolingo + "modo sobrio"**

XP, rachas con freezes, metas separadas, misiones, logros, mascota; nunca se castiga el error (sin corazones). Toggle sobrio para adultos y B2B.

Por qué nadie lo tiene todavía

Tras revisar más de 40 productos, la conclusión es clara: **nadie combina rutas lineales generadas desde fuentes propias con un scheduler FSRS transparente, importancia por ítem, exámenes con fecha y ejercicios interactivos dentro del sistema de memoria**. RemNote 1.28 (agosto 2026) es el competidor más cercano —tiene FSRS, Exam Scheduler y un "Guided Learn Mode"— pero sin camino gamificado ni ejercicios más allá de MCQ/cloze. Kinnu tiene rutas pero sin fuentes propias ni FSRS. Anki no genera nada. Duolingo no acepta contenido propio. SuperMemo tiene la prioridad más sofisticada del mercado con una interfaz de 1995. Google Learn Your Way demostró en un ensayo controlado que transformar una fuente en múltiples formatos con chequeos de comprensión mejora la retención (+11 puntos a 3–5 días), pero es un experimento de laboratorio.

Cómo usar este documento y su compañero

ESTE PDF

Es el "qué" y el "por qué": la investigación verificada (precios y funciones chequeados el 1 de septiembre de 2026), las decisiones de diseño, los algoritmos con fórmulas, el catálogo de funciones y la arquitectura. Leelo entero una vez; después consultalo por sección cuando Claude Code te pregunte algo o cuando quieras cambiar una decisión.

PROMPTS.MD

Es el "cómo": una fase 0 de configuración y 14 fases de desarrollo divididas en 76 sub-fases, cada una con un prompt autocontenido en inglés, criterios de aceptación, comandos de verificación, el **modelo** (Haiku 4.5 / Sonnet 5 / Opus 5 / Fable 5.1) y el **esfuerzo** (Bajo / Medio / Alto / Extra / Max / UltraCode) recomendados. La sección 13 explica cómo se mapean a los controles reales de Claude Code.

Advertencias honestas

- **Los precios de IA cambian semana a semana.** Todo se verificó en páginas oficiales el 1-sep-2026, pero Gemini duplica precios el 1-ene-2027, DeepSeek subió 10× en agosto y OpenAI recortó en julio. La sección 14 lista qué re-verificar antes de fijar precios comerciales.
- **El alcance total es grande:** unas 40–55 semanas-persona de desarrollo para la v1 completa en Windows. La buena noticia es que un MVP usable por vos (repaso + editor + PDF + rutas generadas) es alcanzable en 12–16 y el plan está ordenado para que cada fase deje algo que puedas usar.
- **Los modelos de audio no sirven para puntuar pronunciación** (un estudio de 2026 muestra que diagnostican por estereotipo); por eso la evaluación la hace Azure y el LLM solo explica.
- **Lo que no pudo verificarse** está marcado en el texto con la palabra "no verificado" y consolidado en la sección 14.

2 Visión del producto y decisiones de alcance

Los cinco focos que definiste, traducidos a requisitos verificables; lo que entra y lo que no entra en la v1; y las decisiones que tomamos juntos en las rondas de preguntas.

Los cinco focos, como requisitos

Foco	Lo que pediste	Cómo se traduce en producto (y dónde se detalla)
1 · Rutas estilo Duolingo	Camino lineal de lecciones; cada lección con teoría y práctica; módulos de refuerzo tipo autotest con flashcards.	Estructura Secciones → Módulos → Lecciones → nodos de Refuerzo cada 3-5 lecciones, checkpoints acumulativos y examen final. Cada lección: hook, activación, explicación con citas, worked example, diagrama, misconceptions, resumen + 4-8 actividades (≥3 tipos, máx. 40 % MCQ, ≥1 de nivel "aplicar"). Ver §7 y §11.
2 · Generadas con IA desde tus fuentes	Cargar libros PDF, videos, cursos por módulos, links, DOCX, imágenes, audio; botón "Generar con IA"; ruta fija; test de conocimiento previo o "desde cero".	Pipeline "outline first, then expand" en 10 etapas con QA automática; PathSpec congelada y versionada; diagnóstico adaptativo (Elo-lite + prerrequisitos, 25-30 ítems) que marca lecciones como completadas y siembra su memoria con prioridad baja. Ver §7 y §8.
3 · Sistema de memoria (lo más importante)	Importancia por concepto (urgente vs mantenimiento), simulacros, exámenes agendables; no solo flashcards sino ejercicios interactivos tipo Storyline/Captivate/Moodle.	FSRS-6 con "desired retention" por nivel de importancia (Urgente 0,95-0,97 · Alta 0,92 · Normal 0,90 · Mantenimiento 0,80-0,85 · Pausado), auto-postpone que protege lo importante, exámenes con fecha como capa sobre el scheduler, simulacros por blueprint, y 89 de los 98 tipos de actividad mapeados a la escala 1-4 del scheduler. Ver §5 y §6.
4 · Varias IAs por API con créditos	Imagen, video, audio, texto inteligente y texto rápido/barato, reconocimiento de voz que corrija pronunciación/entonación/acento; calidad sin disparar el costo; posiblemente Claude.	Matriz de 6 proveedores primarios (Anthropic, Google, Azure Speech, ElevenLabs, Google/BFL para imagen, local) con fallbacks. Presupuesto mensual y contador de costo por llamada dentro de la app. Ver §9.
5 · La mayor cantidad de funciones	Todo lo bueno de RemNote, SuperMemo y compañía; un sistema "recontra potente".	Catálogo maestro de 125 funciones etiquetadas MVP / V1 / V2 / Later, con atribución de dónde viene cada idea. Ver §12.

Decisiones tomadas en las rondas de preguntas

Pregunta	Tu decisión	Implicación en el plan
Stack de escritorio	Electron + React + TypeScript	Un solo Chromium (152) y Node 24 en el proceso principal; misma base que tu programa DM Gestión; ecosistema de ingesta en Node. Instalador ≈ 90-120 MB.
Modelo de producto v1	Uso propio, 100 % local, con tu API key	Sin backend, cuentas ni pagos en v1. Se diseña "sync-ready" (UUIDv7, soft deletes, outbox, repos por puerto) para no reescribir al comercializar.
Presupuesto de IA	USD 20-50 / mes	Matriz híbrida ≈ USD 9-14 (texto) + USD 6-14 (voz) + USD 5-6 (media) en uso intensivo: entra con margen si el video generativo queda opt-in.
Idiomas con pronunciación	Inglés + portugués, francés, alemán, italiano, español	Azure AI Speech es el único que cubre los 6 (en-US/GB/AU/CA/IN, pt-BR/PT, fr-FR/CA, de-DE, it-IT, es-ES/MX). Prosodia solo en en-US; en-IE no existe como locale de evaluación (se resuelve con "Accent Lab").

Pregunta	Tu decisión	Implicación en el plan
Videos	Transcribir audio + capturar frames clave	Whisper local gratis (Parakeet/whisper-turbo vía sherpa-onnx o whisper.cpp) + 20-60 keyframes por hora → OCR/visión barata (Gemini Flash-Lite o Qwen3.7-flash). Nunca enviar el video entero (20 h ≈ 19 M tokens).
IA local	Sí, como proveedor opcional para tareas livianas	Ollama/LM Studio como "un proveedor más" con fallback a nube: embeddings, tagging, pistas, chat sobre notas. Nunca para generar rutas completas.
Import/export v1	Anki (.apkg) y CSV; RemNote; Markdown/Obsidian	Parser propio de .apkg (legacy 2 y anki21b con zstd), RemNote vía su export Anki/OPML/Markdown, Obsidian con sintaxis del plugin Spaced Repetition; export Markdown y .apkg propio.
Ruta fija vs. refuerzo	Insertar mini-lecciones de refuerzo sin alterar la ruta base	Remediaciones con id derivado (L07.r1), opcionales, máximo 1 activa por módulo y 3 por semana, disparadas por el sistema de memoria, el refuerzo o el usuario.
Nombre	Retenia (de trabajo)	retenia.app / retenia.io sin DNS (posiblemente libres); .com ocupado. Verificar WHOIS e INPI antes de fijar marca. Las 12 propuestas están al final de esta sección.
Prompts	En inglés; con setup de Claude Code; muy detallado	prompts.md: fase 0 de configuración + 14 fases de desarrollo, 76 sub-fases con modelo y esfuerzo.
Plataformas	Windows 11 primero, Mac después	electron-builder con NSIS; macOS (arm64 + x64 por separado, notarización con Apple Developer USD 99/año) en la fase final.
Gamificación	Completa estilo Duolingo + modo sobrio	XP, rachas con freezes, metas diaria y de racha separadas, misiones, logros, mascota (Rive), ligas opcionales; nunca se castiga el error.
Proveedores de texto	Claude Sonnet 5 + Gemini 3.7 Flash + local	Roles "smart" y "cheap" configurables; Opus 5 como modo premium opcional; Haiku 4.5 y Flash-Lite para extracción masiva.

Alcance de la v1 y no-objetivos

ENTRA EN LA V1 (WINDOWS)

- Biblioteca de fuentes con ingesta local (PDF, DOCX, EPUB, MD/TXT, imágenes, audio, video, YouTube, web) y RAG local.
- "Generar con IA": ruta fija con diagnóstico, lecciones teoría+práctica, refuerzos, examen final; regeneración versionada.
- Sistema de memoria FSRS-6 con importancia, exámenes con fecha, simulacros, final drill, postpone/mercy, estadísticas.
- Activity engine con 21 tipos MVP y ampliación a ~50 en fases 2-3 (audio, imagen, código, juegos, voz).
- Módulo de idiomas: pronunciación (Azure), dictado, listening, shadowing de acento, TTS multi-voz.
- Editor de notas con cloze/LaTeX/oclusión, lector PDF con highlights → cards, tutor con citas.
- Gamificación completa con modo sobrio; import/export Anki/CSV/RemNote/Markdown; auto-update firmado.

NO ENTRA EN LA V1 (EXPLÍCITAMENTE)

- Cuentas de usuario, sync en la nube, pagos, licencias, B2B (queda preparado en el esquema).
- Móvil (el core en `packages/core` se comparte con un futuro cliente Expo).
- Video generativo como default (solo opt-in con costo visible).
- Conversación por voz en tiempo real (fase 2; Gemini Live / gpt-realttime-mini).
- Evaluación de pronunciación offline (GOP local es un proyecto de I+D para v2).
- Marketplace de rutas, plugins de terceros, colaboración en tiempo real.
- Descarga de YouTube/Udemy dentro del producto base (ToS); el usuario aporta archivos propios.

Principios de diseño que gobiernan todo

1. **La evidencia manda.** Práctica de recuperación y espaciado son las únicas técnicas de "alta utilidad" (Dunlosky 2013); todo termina en recuperación activa con feedback, nunca en relectura.
2. **El scheduler es transparente.** El usuario ve estabilidad, retrievability y por qué algo aparece hoy; "cuánto me va a costar en repasos" acompaña cada decisión de importancia.
3. **Contenido ≠ presentación.** Un mismo skill (concepto) se repasa en formatos variados (principio "switch template" de Wordwall); el scheduler programa skills, no pantallas.
4. **Fidelidad a las fuentes.** Cada afirmación sustantiva cita bloque y página/timestamp; los claims se verifican; lo que es conocimiento general se marca.
5. **Nunca castigar el error.** Los errores son datos del scheduler; sin corazones ni vidas (la lección que Duolingo aprendió con "Energy").
6. **Local-first y BYOK.** Tus datos en tu PC, claves cifradas con DPAPI, costo visible por llamada y presupuesto mensual con alertas.
7. **Determinismo donde importa.** La secuenciación de la ruta y los graders son código puro; la IA propone, el código valida.
8. **Cada fase deja algo usable.** El plan está ordenado para que, desde la fase 4, puedas repasar de verdad en la app.

Propuestas de nombre

Verificación realizada: solo resolución DNS de `.app/.com/.io` el 1-sep-2026. "Sin DNS" sugiere que el dominio podría estar libre (no lo garantiza). Falta WHOIS y búsqueda de marca (INPI en Argentina, EUIPO/USPTO si se expande).

#	Nombre	Racional	Pistas de dominio
1	Retenia ✓ elegido	De "retener": memoria que se queda; suena a marca; fácil en ES/EN.	retenia.app, retenia.io sin DNS; .com ocupado.
2	Mentia	"Mente" + terminación de marca; corto y pronunciable.	mentia.app/.io sin DNS; .com ocupado. Riesgo: cercanía a "dementia" en inglés.
3	Memtica	"Memoria" + "-tica" (técnica/práctica); suena a método.	memtica.app sin DNS; .com ocupado.
4	Brotea	El conocimiento que brota y crece; mascota natural (brote).	brotea.app sin DNS; .com ocupado.
5	Aprendix	"Aprender" + sufijo tech; claro para B2B en LatAm.	aprendix.app sin DNS.
6	Anclaje	Anclar conocimiento; concepto de psicología; muy diferenciado.	anclaje.app sin DNS; .com ocupado; difícil en inglés.
7	Retenta	Variante latina de "retener"; sonora y marcable.	retenta.app sin DNS; .com ocupado.
8	Fíjalo	Imperativo coloquial "fíjalo en la memoria"; voz propia LatAm.	fijalo.app sin DNS; el acento complica el dominio.
9	Sendalab	"Senda" (camino → rutas) + lab.	sendalab.app sin DNS; senda. * ocupados.
10	Retiene	Verbo directo: "retiene lo que aprendés".	retiene.app sin DNS; .com ocupado.
11	Evoca	Evocar = recuperar de la memoria (active recall); bilingüe.	evoca.app y .com ocupados; viable solo como "EvocaLab".
12	Lumbre	La brasa que se mantiene encendida = la racha viva.	lumbre.app ocupado; lumbrelab.app sin DNS.

3 Panorama competitivo: qué existe y qué falta

Inventario verificado (help centers, changelogs y páginas oficiales, 1-sep-2026) de RemNote, SuperMemo, Anki, Duolingo, Gemini Notebook (ex NotebookLM) y unas 40 apps modernas. Lo que no pudo verificarse está marcado.

RemNote 1.28 — el competidor más cercano

RemNote (v1.28.9 del 31-ago-2026; planes Free / Pro USD 8 / Pro+AI USD 18 por mes) es hoy "el 80 % de la visión" en el eje notas + cards + IA + exámenes. Lo que tiene y conviene igualar o superar:

Área	Funciones verificadas
Editor y KB	Outliner de "Rems" con referencias y backlinks, portales (embeber un bloque en otro lugar), search portals con query builder, tags y framework concepto/descriptor, templates, alias, tablas V2 con AI Autofill y cards por celda, LaTeX con cloze, code blocks, Daily Documents, pestañas y multi-ventana (1.28), canvas infinito y documentos manuscritos, web clipper.
Fuentes	Reader PDF con highlights vinculados, anotación web, YouTube Annotator, Lecture Recorder con transcripción on-device (iOS), OCR de imágenes, importación desde Anki/Quizlet/Markdown/CSV, exportación Markdown/PDF/Anki/CSV.
Flashcards	Basic (>>), reverse, bidireccional (<>), concepto (: :), descriptor (; ;), cloze, multi-line list/set, multiple choice (+ "Multiple-Choice Learn Mode"), type-in con fuzzy y AI Grading, image occlusion, card clusters, hints, extra detail, Edit Later, Flashcard Insights ("Why True", "Near Misses"), cola "Cards to Learn", TTS en cola, card table.
Scheduler	SM-2 (Anki-like) y FSRS v6 (beta) con learning steps y auto-train; 4 botones + Skip; daily goal y streak goal separados ("Forgot" no cuenta); streak freezes (1 cada 6 días, banco de 2); prioridades por documento (Exam / Currently Studying / Maintaining / Paused).
Exam Scheduler V2	Asistente de 10 pasos; periodos Learning / Catch-up / Final review; "Ensure Mastery" (2 aciertos seguidos); meta diaria calculada; avisos de retraso; prioridad de las cards del examen en la cola; presets (p. ej. MCAT).
IA	AI Tutor con citas clicables y lectura de pantalla; Guided Learn Mode (PDF/video/audio → baseline → secciones de 5-10 min → resumen + flashcards + quizzes → Mastery Tracker → Study Guide); AI grading; selección de modelo por costo; créditos por plan; servidor MCP local (beta).
Plataforma	Sync ilimitado + offline; plugins con SDK React y marketplace; compartir/publicar; badge del icono con cards de hoy; update reanudable.

LO QUE REMNOTE NO TIENE (NUESTRO ESPACIO)

Camino gamificado con lecciones teoría+práctica; ejercicios interactivos más allá de MCQ/cloze/type-in; importancia por ítem que module la retención objetivo; herramientas de sobrecarga (postpone, mercy, final drill); multi-IA por modalidad con BYOK y modelos locales; voz/pronunciación; plantillas de examen LatAm.

SuperMemo 20 — las ideas avanzadas que nadie copia

SuperMemo 20 (20.00.31, junio 2026; algoritmo SM-20 publicado en febrero 2026, ajustado por machine learning con ~40 parámetros y un "Algorithm Arena" que compara SM-2...SM-20 y FSRS sobre tus datos) sigue siendo la referencia de: **priority queue** 0-100 % con auto-sort y **auto-postpone** que sacrifica primero lo menos importante; **Mercy** (redistribuir el atraso), Postpone, Advance, Add to outstanding; **final drill** (repetir lo calificado < Good hasta aprobar en la misma sesión); subset review, search & review y **neural review** (activación que se propaga por enlaces conceptuales); incremental reading/video con extractos y clozes; PDF incremental nativo; Plan (agenda diaria), Tasklist, Sleep Chart (modelo de dos componentes de alertness); análisis con 400 curvas de olvido, simulación de carga a años. supermemo.com añade cursos, Live (diálogo hablado con IA, 15 min/día), MemoChat y generación de cards con ChatGPT. Debilidad: UX de otra época.

Anki — el estándar de portabilidad

Anki (25.09.x; 26.05 en beta con instalador sin terminal) define el formato de intercambio (.apkg) y la implementación de referencia de FSRS: desired retention 0,70–0,97 por preset y por mazo, optimize, health check, simulador de carga, easy days, load balancer, bury de hermanas, leeches (8 lapsos), custom study y filtered decks (repasar olvidadas, review ahead, por tag/estado), image occlusion nativa, cloze anidado, estadísticas completas (Future Due, Calendar, Card Stability/Difficulty/Retrievability, True Retention). La brecha con sus add-ons es una lista de requisitos para nosotros: heatmap, postpone/advance/flatten/vacaciones (FSRS Helper), TTS multi-servicio (HyperTTS), Speed Focus, AnkiConnect (API local), leaderboard.

Duolingo — la referencia de motivación

Un único camino visible (Sections → Units → Levels → Lessons con Guidebook), micro-lecciones con feedback inmediato, Stories, DuoRadio, Adventures, Practice Hub (Mistakes, Words, Listening, Speaking), placement test, Legendary, Duolingo Score 0–160 alineado a CEFR; Max: Explain My Answer, Roleplay, Video Call con Lily; cursos de Math, Music, Chess (con Game Review, ago-2026). Gamificación: streak + freezes + friend streaks + Streak Society, XP y boosts, 10 ligas, gems, quests diarias/amigos/mensuales, badges, avatares, timed challenges (Match Madness), notificaciones optimizadas con bandits. Evidencia pública: las ligas subieron 17 % el tiempo de aprendizaje; la notificación "streak saver" fue el mayor salto de retención; los usuarios con racha ≥ 7 días pasaron de ~17 % a >50 % del DAU; un contador de "movimientos" estilo Gardenscapes no movió nada; los corazones fueron reemplazados por "Energy" en 2025 tras años de críticas. Lo que Duolingo no hace: contenido propio, scheduler transparente, desktop serio.

Gemini Notebook (ex NotebookLM) y la ola de "sube y genera"

Renombrado el 16-jul-2026, genera artefactos sueltos desde fuentes (chat con citas, Audio/Video Overviews, mind maps, reports, flashcards y quizzes con dificultad y "Explain", infografías, slide decks, data tables, Deep Research) y un modo "Learning Guide" socrático; límites por plan (50/100/300/500/600 fuentes por notebook; 500.000 palabras o 200 MB por fuente; YouTube solo transcript). No produce un currículo secuenciado ni tiene scheduler. Lo mismo vale para Turbo, Knowt, Mindgrasp, StudyFetch, Gizmo, Quizlet Magic Notes: notas + cards + quiz + podcast + tutor, con quejas repetidas de contenido superficial, quizzes repetitivos y errores factuales. Oboe (curso completo desde un prompt) repite las preguntas del quiz en el examen final. La evidencia más sólida a favor de nuestro enfoque es **Learn Your Way** (Google Research, 2025): fuente → texto inmersivo con preguntas embebidas, quizzes por sección, slides + audio, mind map; RCT con 60 alumnos: +9 % inmediato y 78 % vs 67 % de retención a 3–5 días.

Idiomas, PKM y lectura: qué tomar de cada uno

- **ELSA / Speak / Praktika / BoldVoice:** feedback por fonema con palabras coloreadas, roleplay con avatares, tono de feedback configurable (soft/balanced/strict), transcript post-conversación, elección de acento, "accent strength" con modelos propios (BoldVoice). Definen el estándar de la voz.
- **Migaku / LingQ / Readlang / Clozemaster / Lingvist:** click-to-define en contenido real, cards en un clic con frase + audio + captura, "known words" y comprehension score, cloze por frecuencia, custom decks desde cualquier texto.
- **Obsidian / Logseq / Heptabase / Tana:** Markdown local, backlinks, canvas, command palette, plugins, historial de versiones, CLI/MCP para agentes; el plugin Spaced Repetition de Obsidian fija la sintaxis de import (`: : , ==cloze== , {{1:...}}`).
- **Readwise / Reader / Matter / Zotero:** highlights → repaso diario con mastery, Ghostreader, TTS, MCP server; Zotero para bibliografía y anotación académica.
- **Brilliant / Khan Academy:** manipulativos interactivos en la explicación; mastery por skill (Familiar → Proficient → Mastered) con unit tests y course challenges; Khanmigo socrático y sus herramientas docentes.

Matriz comparativa (resumen)

✓ nativo · ◀ parcial o vía add-on · ✗ no · ? no verificado.

Función	Retenia (plan)	RemNote	SuperMemo 20	Anki	Duolingo	Gemini Notebook	Kinnu	Quizlet
Rutas lineales tipo Duolingo	✓ generadas desde fuentes	◀ Guided Learn (sin camino)	✗	✗	✓ (contenido propio de Duolingo)	✗	✓ (curado)	✗
Fuentes propias → contenido	✓ PDF/video/web/audio/img	✓ PDF/video/audio	◀ incremental reading	✗	✗	✓ (artefactos sueltos)	✗	◀ Magic Notes
Algoritmo de memoria	FSRS-6 + importancia	FSRS-6 (beta) / SM-2	SM-20 (+Arena)	FSRS-6 / SM-2	HLR (interno)	✗ (Got it / Missed it)	propio	regresión logística (Learn)
Prioridad / importancia por ítem	✓ 5 niveles → DR, orden, postpone	◀ por documento	✓ 0-100 %	✗	✗	✗	✗	✗
Examen con fecha	✓ capa sobre FSRS + simulacros	✓ Exam Scheduler V2	◀ subset + final drill	◀ filtered decks	✗	✗	✗	◀ Learn con fecha
Ejercicios interactivos programados	✓ 89 tipos con rating 1-4	◀ MCQ / type-in	◀ spelling/MCQ	◀ type-in	✓ (no programados por SRS)	◀ quizzes	◀ MCQ	◀ Learn/Test/Match
Postpone / Mercy / final drill	✓ de serie	✗	✓	◀ add-on	✗	✗	✗	✗
Pronunciación por fonema	✓ Azure + Accent Lab	✗	◀ (supermemo.com STT)	✗	◀ ASR pasa/no pasa	✗	✗	✗
Gamificación completa	✓ + modo sobrio	◀ rachas/metast	✗	◀ add-ons	✓	✗	◀	◀
Local-first + BYOK + IA local	✓	◀ offline; IA propia	✓ local (sin IA)	✓ local (sin IA)	✗	✗	✗	✗
Import/export Anki	✓	✓	◀	✓ (es el formato)	✗	◀ CSV	✗	◀

Quince diferenciadores que nadie ha combinado

1. Ruta Duolingo generada desde *tus* fuentes + scheduler FSRS transparente.
2. Ejercicios interactivos (ordenar, clasificar, etiquetar, code-fill, dictado...) con estado D/S/R propio, reprogramados como cards.
3. Importancia como ciudadano de primera clase (herencia SuperMemo) con UI de 4 niveles y explicación del costo en repasos.
4. Exámenes programados multi-fuente con simulacros en formato real y plan Learning / Catch-up / Final review, en español y con plantillas LatAm (ICFES, ENEM, EXANI, PAES, oposiciones) en V2.
5. Desktop local-first serio + multi-IA con BYOK y modelos locales.
6. Multi-IA por modalidad (texto, imagen, TTS, STT/pronunciación) con caché local y costo visible.
7. Postpone / Advance / Flatten / vacaciones / Easy Days / Final Drill de serie, con un asistente de sobrecarga.
8. Gamificación por perfiles y con evidencia: modo arcade y modo sobrio; nunca castigar errores; métricas de sustancia junto a las de vanidad.
9. Tutor con memoria longitudinal del alumno (qué olvida, cuándo, a qué hora rinde) que crea práctica remedial.
10. Neural/semantic review al fallar: repasar vecinos conceptuales del grafo.
11. Modo socrático + knowledge checks dentro de la teoría, conectados al scheduler.

12. Rendimiento por hora → recomendación de horario (y, opcional, registro de sueño).
13. MCP server/CLI local para usar tu asistente favorito sobre tu base de estudio.
14. B2B (futuro) con informes de retención real, no solo "completó el curso".
15. Voz completa en español LatAm: TTS neuronal con acentos regionales + pronunciación en 6 idiomas integrada en el mismo producto de estudio general.

4 Ciencia del aprendizaje aplicada al diseño

Las reglas que la app y la IA deben respetar, con su evidencia. Cada principio aparece después como restricción concreta en prompts, validadores o UX.

Qué dice la evidencia (y qué implica)

Principio	Evidencia	Implicación de diseño en Retenia
Práctica de recuperación (testing effect)	Dunlosky et al. (2013): practice testing y distributed practice son las únicas técnicas de "alta utilidad"; relectura, subrayado y resúmenes son de baja utilidad. Roediger & Karpicke (2006): ~60 % de retención a una semana con test vs ~40 % con reestudio. Feedback + test supera al test solo.	Toda lección termina en recuperación activa; siempre se muestra la respuesta correcta tras el intento; los refuerzos son autotests, no relecturas.
Espaciado	Cepeda et al. (2006): espaciado > masivo en 259 de 271 comparaciones; el gap óptimo ≈ 10–20 % del intervalo de retención (2008).	El scheduler lo hace solo. En exámenes, la ventana final de repaso en los últimos días replica el gap óptimo.
Dificultades deseables (Bjork)	Spacing, interleaving, testing, generation y variabilidad "parecen entorpecer el entrenamiento pero producen beneficios a largo plazo"; storage strength vs retrieval strength.	Repasar cuando $R \approx 0,85-0,90$, no antes; sin pistas antes del intento; variar el formato de la pregunta ("switch template").
Interleaving	Rohrer & Taylor: mezclar tipos de problema triplicó el rendimiento en examen vs práctica por bloques; ayuda a discriminar categorías parecidas.	La sesión diaria mezcla temas y formatos; los refuerzos intercalan lecciones previas; los simulacros mezclan módulos.
Efecto de generación	Slamecka & Graf (1978): lo generado se recuerda mejor que lo leído; se debilita con material no familiar.	Cloze y "escribí la respuesta" > reconocimiento; MCQ solo para material nuevo (como el Learn Mode de RemNote), luego producción.
Elaboración / self-explanation	Utilidad moderada (Dunlosky); mejora la transferencia.	Botón "¿por qué?" (Flashcard Insights), actividad "explicalo a un novato" con rúbrica.
Dual coding	Paivio: canal verbal + no verbal; picture superiority effect. Wozniak, regla 6: "use imagery".	Image occlusion, diagramas como código en la lección, figura de la fuente adjunta a la card cuando exista.
Metacognición	Brainscape (confidence-based repetition), Moodle CBM: calibrar confianza vs desempeño.	Confianza en el diagnóstico y en los simulacros; mostrar "readiness predicho vs puntaje real".
Cramming	Wozniak: útil para aprobar, dañino a largo plazo ("toxic memory"); Anki: retención > 0,97 = práctica masiva.	Modo urgente explícitamente temporal (DR 0,97 solo la última semana); nunca como default.
Carga cognitiva (Sweller)	Worked-example effect, split-attention, redundancia, expertise reversal.	Texto y diagrama juntos; no narrar lo que está escrito; worked example → completion problem → problema; menos ejemplos en módulos "partial".
Mastery learning (Bloom 2-sigma)	El alumno tutorizado promedio superó al 98 % del grupo control; mastery exige ≥ 80–90 % antes de avanzar.	Gate suave: el refuerzo del módulo debe alcanzar ≥ 80 %; si no, remediación opcional; nunca bloquear por frustración.
Microlearning y preguntas embebidas	Kinnu (10.000 personas): pathways por concepto, preguntas introductorias y "collapsing questions" +59–70 %; el chatbot suelto no tuvo efecto. Learn Your Way: +11 pp de retención con preguntas por sección.	Lecciones de 5–15 min con 1–3 objetivos; knowledge checks dentro de la teoría; el tutor es complemento, no centro.

Las 20 reglas de Wozniak como reglas para el generador de tarjetas

Reformuladas como instrucciones verificables para el prompt `P5_make_flashcards` y su validador:

- 1. No memorizar sin entender:** cada tarjeta se enlaza a la de la teoría. sección que la explica; la ruta la introduce solo después

2. **Aprender antes de memorizar:** las tarjetas nacen en "Need to Learn" y se programan al completar la lección.
3. **Construir sobre lo básico:** orden por prerrequisitos.
4. **Mínima información:** una respuesta atómica ($\leq \sim 7$ palabras o un dato); si tiene varias partes, dividir.
5. **Cloze por defecto** para hechos y definiciones: una deletación por tarjeta, contexto suficiente, pista cuando haya ambigüedad, no ocultar palabras triviales.
6. **Imágenes:** figura de la fuente; image occlusion para anatomía, mapas y diagramas.
7. **Mnemónicos** solo en el 1–5 % de los ítems, como insight bajo demanda.
8. **Cloze visual** (graphic deletion).
9. **Evitar sets:** "¿qué países fundaron la CEE?" → tarjetas individuales con contexto o clozes solapadas.
10. **Evitar enumeraciones:** clozes solapadas (A...E, B...F); si el orden importa, ejercicio de ordenar pasos, no flashcard.
11. **Combatir interferencia:** detectar pares con respuestas similares (embeddings) y generar tarjeta de contraste.
12. **Redacción óptima:** pregunta $\leq \sim 25$ palabras.
13. **Referir a otras memorias:** usar como contexto ítems que el usuario ya domina (S alta).
14. **Personalizar:** campo "mi ejemplo".
15. **Estados emocionales:** ejemplos vívidos cuando la fuente los tenga.
16. **Context cues:** prefijos "[CSS Grid]" en lugar de perífrasis.
17. **Redundancia aceptable:** pares activo/pasivo, pasos de derivación como ítems separados.
18. **Fuentes:** cada tarjeta guarda `source_id` + localizador (página, timestamp, URL).
19. **Date stamping** para conocimiento volátil (campo `as_of`).
20. **Priorizar:** la IA propone importancia inicial (núcleo = Alta; anecdótico = Mantenimiento) y el usuario confirma.

Ratio orientativo: 3–8 tarjetas por lección, máximo 12 (RemNote advierte que "las listas están entre las cosas más difíciles de recordar... la memoria funciona por conexiones, no por secuencias lineales"). Dedupe por embeddings (coseno $> 0,92 \rightarrow$ fusionar).

Cuándo NO usar flashcards

- **Habilidades procedimentales y motoras:** "¿cuáles son los pasos para X?" viola las reglas 9–10. Usar ordenar pasos, completar un procedimiento con huecos, resolver un problema con verificación automática, simulación, "worked example → problema análogo".
- **Comprensión y transferencia:** preguntas de aplicación ("¿qué pasaría si...?"), comparación, clasificación con interleaving, respuesta corta con rúbrica.
- **Conocimiento que se consulta, no se memoriza:** regla de los 5 minutos de gwern: si buscarlo costará menos de ~ 5 minutos en toda la vida, no merece tarjeta; la app ofrece "guardar como referencia".
- **Material no entendido:** regla 1: volver a la teoría, no a la cola.

Marcos de diseño instruccional codificados en los prompts

Marco	Cómo se usa
Bloom revisada	Cada objetivo lleva un verbo (definir, explicar, aplicar, analizar, evaluar, crear); cada actividad declara su nivel; la QA rechaza lecciones sin ≥ 1 actividad de nivel "aplicar" o superior.
Gagné (9 eventos)	Es la plantilla literal de la lección: atención → objetivos → activación → contenido → guía (ejemplos y no-ejemplos) → práctica → feedback → evaluación → retención/transferencia.
Merrill (First Principles)	Cada módulo abre con un problema real de la fuente; ciclo activar → demostrar → aplicar → integrar ("explícalo a un novato", "aplicalo a tu caso").
Objetivos ABCD	Audiencia, comportamiento, condición, grado, guardados estructurados para generar ítems alineados.
CEFR can-do	Objetivos de módulo en rutas de idiomas; diagnóstico por destreza (reading, listening, writing, speaking) con Elo independiente.
PRIMM / Parsons	Programación: Predict → Run → Investigate → Modify → Make; Parsons problems se resuelven en mucho menos tiempo que escribir código con aprendizaje similar (Ericson 2017); patrón "código con huecos" del Learning mode de Claude Code.

Marco	Cómo se usa
Medicina / derecho / historia	Viñetas clínicas NBME (one-best-answer, opciones homogéneas, sin "todas las anteriores"); case briefs e IRAC con rúbrica; timelines y mapas como preguntas gráficas.

Tipo de contenido → actividades → ítems de memoria

Tipo de contenido	Bloom	Mejores actividades	Ítems de memoria
Hechos, fechas, cifras, terminología	remember	cloze, matching, MCQ rápido, concept lookup	cloze (1 dato), basic Q→A, reverse solo para vocabulario
Definiciones y conceptos	understand	explicar con tus palabras (rúbrica), clasificar ejemplos/no-ejemplos, comparar	basic (concepto → definición corta), contraste, ejemplo → concepto
Procedimientos (algoritmos, protocolos, gramática)	apply	worked example → completion → problema completo; Parsons; predict output; transformación de frases	cloze de pasos (uno por tarjeta), "¿cuál es el siguiente paso?"
Principios y modelos causales	analyze	case analysis, error-spotting, "¿qué pasaría si...?", diagramas a completar	Q→A causa-efecto; contraste
Criterios, trade-offs, normas	evaluate	juzgar soluciones con rúbrica, ranking justificado, debate de dos posturas	"¿cuándo conviene X sobre Y?"
Producción (código, ensayo, diseño, L2)	create	proyecto guiado, code_exercise con tests, essay con rúbrica, speaking prompt	pocas tarjetas: solo patrones reutilizables
Contenido visual (anatomía, mapas, diagramas)	remember / understand	image occlusion, etiquetar diagrama, timelines	image_occlusion
Vocabulario L2	remember / apply	cloze en oración, matching, producción guiada, dictado	cloze en contexto; reverse limitado

5 El sistema de memoria

El corazón del producto: FSRS-6 con fórmulas listas para implementar, niveles de importancia que modulan la retención objetivo, exámenes con fecha como capa sobre el scheduler, simulacros, métricas y el mapeo de cualquier ejercicio interactivo a la escala 1–4.

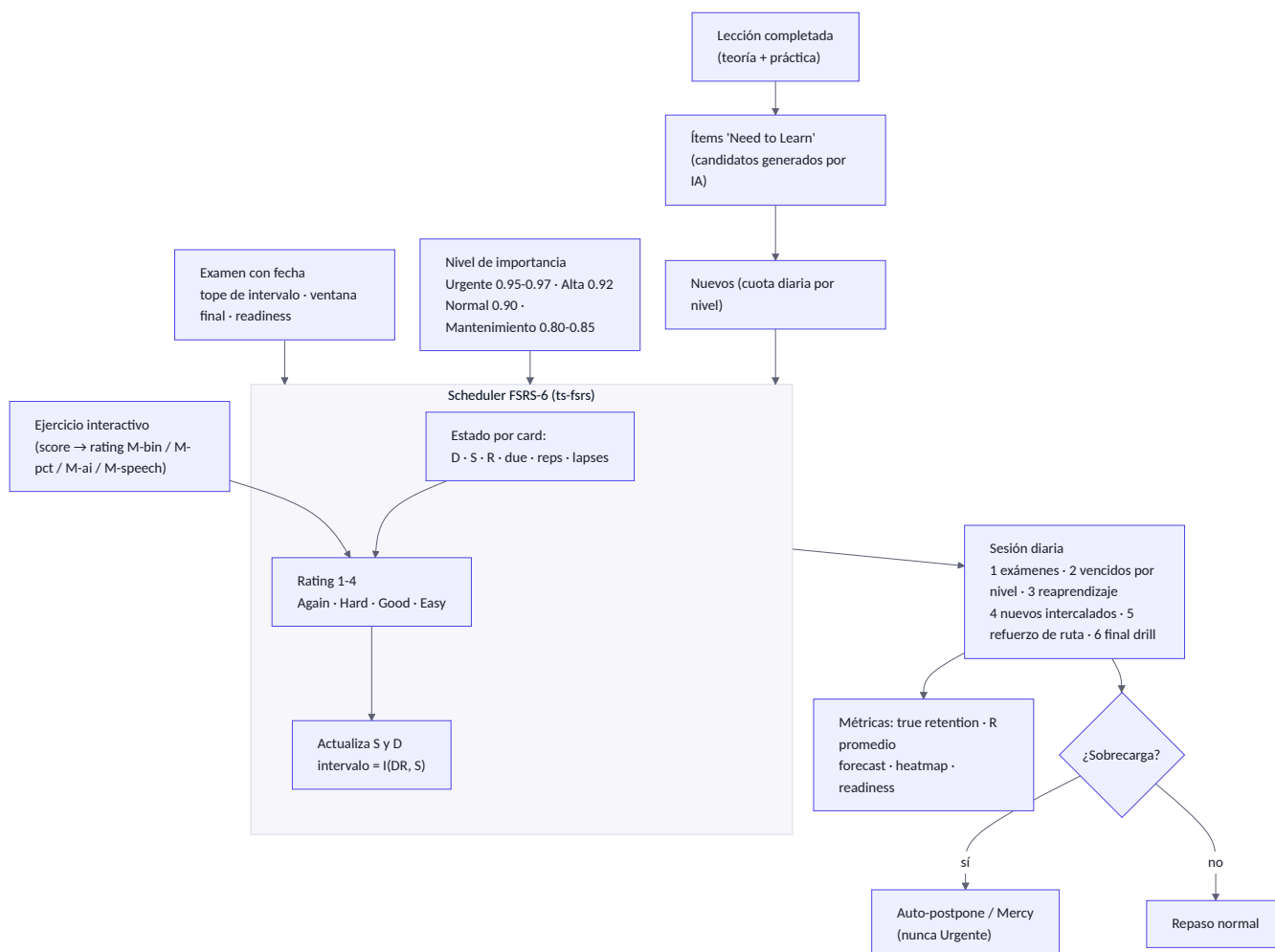


Figura 5.1 · Flujo del sistema de memoria: de la lección a la sesión diaria, con importancia y exámenes como entradas del scheduler.

Decisiones

ALGORITMO

FSRS-6 vía ts-fsrs 5.4.x (MIT)

Mejor calibración pública: log loss 0,346 vs 0,469 de HLR (Duolingo) y 99,6 % de "superioridad" sobre SM-2 en ~350 M de reviews. Lo usan Anki, RemNote y Mochi. SM-20 de SuperMemo convergió al mismo enfoque paramétrico.

MODELO MENTAL

DSR: Difficulty · Stability · Retrievability

Todo lo que la app muestra (fuerza, decaimiento, pronóstico, readiness) se deriva de S y R. Cualquier actividad que produzca evidencia de recuerdo se convierte en un rating 1–4.

IMPORTANCIA

Desired retention por nivel + tope de intervalo + orden + postpone

Combina el priority queue de SuperMemo con las prioridades de RemNote. La importancia no cambia el modelo de memoria: cambia lo que se le pide al scheduler y qué se sacrifica ante la sobrecarga.

EXÁMENES

Capa sobre FSRS (no un algoritmo aparte)

Tope de intervalo $\min(I, \text{fecha} - \text{margen})$, DR creciente, ventana final ordenada por R ascendente, "ensure mastery", readiness = $\sum R$ el día del examen. FSRS no tiene modo examen nativo (issue abierto); RemNote es la referencia.

Fundamentos: curva de olvido y modelo de dos componentes

Tras aprender algo, la probabilidad de recordarlo decae con el tiempo; cada repaso exitoso reduce la tasa de decaimiento futuro. El modelo de dos componentes (Wozniak & Gorzelańczyk, 1990/1995; equivalente a storage/retrieval strength de Bjork) describe una memoria con **Stability (S)** —cuánto dura si no se perturba— y **Retrievability (R)** —la probabilidad de recuperarla ahora—. Operativamente, S es el intervalo tras el cual R cae a 0,9. La **dificultad (D)** (1-10 en FSRS) determina cuánto puede crecer S en cada repaso. El efecto de espaciado aparece explícito: el incremento de S es mayor cuando R es baja ("el mejor momento para repasar es cuando casi lo olvidaste").

FSRS-6: fórmulas completas para implementar

Versión vigente: FSRS-6, 21 parámetros ($w_0 \dots w_{20}$), implementada en `ts-fsrs` (TypeScript), `py-fsrs`, `fsrs-rs` (Rust, usado por Anki). Parámetros por defecto (`ts-fsrs 5.4.2` = `py-fsrs`):

```
w = [0.212, 1.2931, 2.3065, 8.2956, // w0..w3 S0 inicial para Again/Hard/Good/Easy
     6.4133, 0.8334, // w4,w5 dificultad inicial
     3.0194, 0.001, // w6,w7 Δ dificultad y mean reversion
     1.8722, 0.1666, 0.796, // w8..w10 crecimiento de S tras acierto
     1.4835, 0.0614, 0.2629, 1.6483, // w11..w14 S tras olvido
     0.6014, 1.8729, // w15 hard penalty, w16 easy bonus
     0.5425, 0.0912, 0.0658, // w17..w19 memoria a corto plazo (mismo día)
     0.1542] // w20 decay de la curva de olvido (FSRS-6)
```

Componente	Fórmula	Intuición
(a) Curva de olvido	$\text{decay} = -w_{20}$; $\text{factor} = 0.9^{(1/\text{decay})} - 1$; $R(t, S) = (1 + \text{factor} \cdot t/S)^{\text{decay}}$	Ley de potencia (material heterogéneo); garantiza $R(S, S) = 0.9$. FSRS-4.5/5 usaban decay fijo -0.5 ; FSRS-6 lo optimiza por usuario.
(b) Intervalo para retención r	$I(r, S) = S \cdot (r^{(1/\text{decay})} - 1) / \text{factor} \rightarrow I(0.9, S) = S$; $\text{interval} = \text{clamp}(\text{round}(I), 1, \text{max_interval}) + \text{fuzz}$	Fórmula cerrada: permite saber en $O(1)$ si un ítem "llega" a la fecha de examen con $R \geq$ objetivo.
(c) Estado inicial (grado $G \in 1..4$)	$S_0(G) = w[G-1]$; $D_0(G) = w_4 - e^{(w_5 \cdot (G-1))} + 1$, $\text{clamp}[1, 10]$	Cuatro estabildades iniciales según el primer botón.
(d) Dificultad	$\Delta D = -w_6 \cdot (G-3)$; $D' = D + \Delta D \cdot (10-D)/9$; $D'' = w_7 \cdot D_0(4) + (1-w_7) \cdot D'$; $\text{clamp}[1, 10]$	Linear damping (D nunca llega a 10 "de golpe") + mean reversion hacia $D_0(\text{Easy})$.
(e) S tras acierto ($t \geq 1$ día)	$S' = S \cdot (1 + e^{w_8 \cdot (11-D)} \cdot S^{(-w_9)} \cdot (e^{(w_{10} \cdot (1-R))} - 1) \cdot w_{15}^{[G=\text{Hard}]} \cdot w_{16}^{[G=\text{Easy}]})$	$(11-D)$: lo difícil estabiliza menos; $S^{(-w_9)}$: rendimientos decrecientes; $e^{(w_{10}(1-R))}-1$: efecto de espaciado; $w_{15} < 1$ penaliza Hard, $w_{16} > 1$ bonifica Easy.
(f) S tras olvido (Again)	$S'f = w_{11} \cdot D^{(-w_{12})} \cdot ((S+1)^{w_{13}} - 1) \cdot e^{(w_{14} \cdot (1-R))}$; $S' = \min(S'f, S/e^{(w_{17} \cdot w_{18})})$	Un olvido nunca aumenta la estabilidad.
(g) Mismo día ($t = 0$)	$S_{\text{Inc}} = S^{(-w_{19})} \cdot e^{(w_{17} \cdot (G-3+w_{18}))}$; $S_{\text{Inc}} = \max(S_{\text{Inc}}, 1)$ si $G \geq \text{Hard}$; $S' = S \cdot S_{\text{Inc}}$	Memoria a corto plazo: crece rápido cuando S es pequeña, lento cuando es grande.
(i) Fuzz	Solo si $\text{interval} \geq 2.5$ días: $\pm 15\%$ en $[2.5, 7]$, $\pm 10\%$ en $[7, 20]$, $\pm 5\%$ en $[20, \infty)$; PRNG sembrado por tarjeta; $\text{min_ivl} \geq \text{elapsed_days} + 1$	Evita que las tarjetas creadas juntas venzan juntas. Anki 24.11 añadió un load balancer: dentro del rango de fuzz elige el día con menos repasos.

Rangos de clamping de los parámetros (ts-fsrs): w_0 – $w_3 \in [0.001, 100]$; $w_4 \in [1, 10]$; $w_5, w_6 \in [0.001, 4]$; $w_7 \in [0.001, 0.75]$; $w_8 \in [0, 4.5]$; $w_9 \in [0, 0.8]$; $w_{10} \in [0.001, 3.5]$; $w_{11} \in [0.001, 5]$; $w_{12} \in [0.001, 0.25]$; $w_{13} \in [0.001, 0.9]$; $w_{14} \in [0, 4]$; $w_{15} \in [0, 1]$; $w_{16} \in [1, 6]$; $w_{17}, w_{18} \in [0, 2]$; $w_{19} \in [0.01, 0.8]$; $w_{20} \in [0.1, 0.8]$.

Estados, pasos de aprendizaje, leeches y hermanas

- **Estados:** New = 0, Learning = 1, Review = 2, Relearning = 3. **Ratings:** Manual = 0, Again = 1, Hard = 2, Good = 3, Easy = 4.
- **Learning steps** por defecto ['1m', '10m'], relearning ['10m']. Again → paso 0; Hard → 1,5× el paso (o promedio de los dos primeros); Good → siguiente paso o gradúa a Review con $I(r, S)$; Easy → gradúa directo. Recomendación de Anki y Expertium: pasos cortos (10–30 min), nunca ≥ 1 día con FSRS.
- **Intervalo mínimo** 1 día tras el reaprendizaje; **máximo** 36.500 días por defecto (Expertium usa 5 años en general y 365 días en material difícil).
- **Leeches:** Anki etiqueta y suspende a los 8 lapsos (avisos cada mitad del umbral); RemNote interrumpe la cola a los 4 con Disable / Ignore / Edit Later. Nosotros: umbral por nivel de importancia + sugerencia de reescritura por IA.
- **Bury de hermanas:** las tarjetas de la misma nota (reversas, otras clozes) se posponen al día siguiente para que una no revele otra; "disperse siblings" separa sus fechas.
- **Easy days:** por día de la semana (Normal / Reducido / Mínimo) y fechas específicas; ajusta la fecha de vencimiento "en un pequeño margen".

Tipos de ts-fsrs que se copian al modelo de datos

```
interface Card { due: Date; stability: number; difficulty: number; scheduled_days: number;
  learning_steps: number; reps: number; lapses: number; state: State; last_review?: Date; }
interface ReviewLog { rating: Rating; state: State; due: Date; stability: number; difficulty:
  number;
  elapsed_days: number; scheduled_days: number; learning_steps: number; review: Date; }
interface FSRSParameters { request_retention: number /*0.9*/; maximum_interval: number /*36500*/;
  w: number[] /*21*/; enable_fuzz: boolean; enable_short_term: boolean; learning_steps: Steps;
  relearning_steps: Steps; }
// API: fsrs(params) · createEmptyCard(now) · f.repeat(card, now) (preview de los 4 botones)
//      f.next(card, now, Rating.Good) · f.get_retrievability(card, now) · f.rollback · f.forget ·
  f.reschedule(card, history)
```

Nota: ts-fsrs@6.0.0-beta elimina elapsed_days de Card; no depender de ese campo. Fijar ts-fsrs como referencia y cubrir con tests de regresión contra py-fsrs (hay discrepancias menores: clamp de 50 0,1 vs 0,001; máscara ≥ 1 en Hard del mismo día).

Optimizador, desired retention y benchmarks

- **Entrenamiento:** pérdida log-loss (cada review es binaria); los 4 primeros parámetros se estiman de las curvas de retención tras el 1.º y 2.º repaso; luego descenso de gradiente. Anki 24.06+ no exige mínimo de reviews; RemNote recomienda ≥ 1.000 . Re-optimizar cada 2ⁿ reviews (512, 1.024, 2.048...) o mensualmente. En Node: @open-spaced-repetition/binding (napi sobre fsrs-rs) en un worker, o fsrs-browser (WASM) para evitar binarios nativos.
- **Desired retention:** "el ajuste más importante"; permitido 0,70–0,99; recomendado 0,80–0,95; 0,90 por defecto; por encima de 0,97 "transforma la repetición espaciada en repetición masiva". Cambiarla no requiere re-optimizar.
- **Simulador** (fsrs-rs SimulatorConfig): proyecta reviews/día, minutos/día, memorizado y costo a un año para un DR dado; Anki lo expone en las opciones de mazo. Lo usamos para explicar el costo de "Urgente" antes de aplicarlo.
- **DR vs retención media:** con DR 0,90 se recuerda $\approx 94,7\%$ de todas las tarjetas hoy (la mayoría no está vencida). La app muestra ambas.

Benchmark público (srs-benchmark, 9.999 colecciones de Anki, ~350 M de reviews)

Algoritmo	Log loss	RMSE(bins)	AUC	Parámetros
RWKV-Instant (red neuronal)	0,2773	0,0250	0,833	2.762.884

Algoritmo	Log loss	RMSE(bins)	AUC	Parámetros
GRU / LSTM	0,333	0,054	0,73	503 / 8.869
FSRS-6 (fsrs-rs)	0,3443	0,0635	0,707	21
FSRS-5 / FSRS-4.5	0,3561 / 0,3625	0,074 / 0,076	0,70 / 0,69	19 / 17
DASH / ACT-R	0,3682 / 0,4033	0,084 / 0,107	0,63 / 0,52	9 / 5
HLR (Duolingo)	0,4694	0,1275	0,637	3
Ebisu v2	0,4989	0,1627	0,605	0

Las redes neuronales ganan con cinco órdenes de magnitud más parámetros y sin interpretabilidad; para una app local-first con ~10⁴ tarjetas por usuario, FSRS-6 es el óptimo. Comparado con SM-17 (fsrs-vs-sm17, 19 usuarios, 687.662 repeticiones): log loss 0,367 vs 0,432, "83,3 % de superioridad".

Otros algoritmos y qué se reutiliza de cada uno

Algoritmo	Modelo	Qué aporta a Retenia
SM-2 / Anki SM-2	EF × intervalo; ease piso 130 % ("ease hell")	Solo fallback determinista y formato de intercambio al importar (.apkg trae ease/intervalo; memory_state_from_sm2 de fsrs-rs los convierte).
SM-17/18/20 (SuperMemo)	S, R, D con matrices SInc/Recall; SM-20 pasa a ~40 parámetros con ML	El diseño de prioridades, auto-postpone, final drill y "memory lability" (S puede caer en ítems muy difíciles).
Ebisu (bayesiano)	Beta(α,β) sobre p a tiempo t; sin fechas de vencimiento	La idea de cola "sin due date" ordenada por R (modo "repasá lo que quieras hoy" y ventana final de examen), que FSRS permite con get_retrievability .
HLR (Duolingo)	$p = 2^{-(\Delta/h)}$, $h = 2^{(\Theta \cdot x)}$	La UI del "strength meter" por skill, alimentada con R de FSRS.
Brainscape CBR	Confianza 1–5 → intervalo	El valor metacognitivo: confianza declarada en diagnóstico y simulacros; mapeo 1–2 ≈ Again, 3 ≈ Hard, 4 ≈ Good, 5 ≈ Easy.
Mochi (2 botones)	Remembered / Forgot → Good / Again	Demuestra que una UI de 2 botones es compatible con FSRS (modo simple opcional).

Importancia por ítem: diseño

Principio: la importancia no cambia S, D ni R (son propiedades del par ítem-usuario); cambia *qué se le pide al scheduler* (desired retention, tope de intervalo), *qué se sacrifica ante la sobrecarga* (orden y postpone) y *cuánto presupuesto diario consume*.

Costo de la retención objetivo (derivado de l(r,s) con w20 = 0,1542; frecuencia relativa a 0,90)

DR	0,70	0,80	0,85	0,90	0,93	0,95	0,97	0,98	0,99
Intervalo (× S)	9,29	3,32	1,91	1,00	0,61	0,40	0,22	0,14	0,07
Frecuencia de repasos	0,11×	0,30×	0,52×	1,00×	1,63×	2,48×	4,49×	7,0×	14,6×

Dos correcciones de segundo orden empeoran los extremos: con DR alta se repasa con R alta y S crece más lento (workload real mayor); con DR baja hay más lapsos y reaprendizajes. De ahí la curva en U que la función "optimal retention" minimizaba (workload/knowledge). Conclusión: **0,97 es el techo del modo urgente y 0,80–0,85 el suelo del mantenimiento**; por debajo de 0,80 el usuario percibe "me hace olvidar".

Nivel (UI)	Semántica	DR	Max interval	Orden	Ante sobrecarga	Nuevas/día	Leech
Urgente / Examen	"tenelo siempre presente" hasta una fecha	0,95 (0,97 última semana)	min(180 d, examen – hoy – margen)	1.º, por R ascendente	nunca se pospone; puede superar el límite diario (catch-up)	sin tope (por fecha)	avisar, nunca suspender
Alta	material central del camino actual	0,92	1 año	2.º	solo si backlog > 2 días	prioridad de introducción	avisar + sugerir reescritura
Normal	por defecto	0,90	5 años	3.º	factor 1,1 sobre los más estables primero	cuota estándar (10–20)	umbral 8 → editar; opción suspender
Mantenimiento	"que no se pierda, sin cargar el día"	0,80–0,85	10 años	4.º	se pospone primero (Mercy)	0 (solo repaso)	suspender tras 8 lapsos

Nivel (UI)	Semántica	DR	Max interval	Orden	Ante sobrecarga	Nuevas/día	Leech
Pausado	fuera de la cola; el reloj sigue corriendo	—	—	no aparece	—	—	—

Reglas de implementación: (1) DR por ítem, no por mazo: `effective_retention(item) = level.dr` con override por ítem y por examen (el examen manda); instanciar un `fsrs()` por nivel es barato. (2) Cambiar de nivel no reprograma en masa (como "Reschedule cards on change = off" de Anki): el nuevo DR aplica desde el próximo repaso; "reprogramar ahora" es una acción explícita que muestra el impacto simulado. (3) Protección ante sobrecarga: si `due_today × tiempo_mediano > capacidad`, posponer con factor 1,1 empezando por Mantenimiento y por los ítems de mayor S (menor daño); nunca Urgente; registrar cada postpone con `rating = Manual`. (4) Sesgo de prioridad: mostrar el porcentaje por nivel y limitar Urgente + Alta a ~30 % ("si todo es urgente, nada lo es"). (5) Modo urgente de 48-72 h: DR 0,97 + final drill + pasos del mismo día 10m 1h, explícitamente temporal. (6) Decaimiento visible: "esto hoy lo recordás con ~82 %".

Exámenes con fecha

Cómo lo resuelven los demás

Anki no tiene modo examen: se compone con filtered decks, custom study (review ahead, review forgotten), "Set due date" y el helper con Advance ("adelanta repasos minimizando la desviación del schedule original"); hay un feature request abierto para "deadline option in FSRS". **SuperMemo**: subset review (Learn / Review all sobre una rama), Add to outstanding y Final drill; SM 20 añade "Active Final Drill" para exámenes. **RemNote Exam Scheduler V2** es la referencia más completa: learning period (cada tarjeta nueva sale tras 2 aciertos), catch-up (sube el daily goal en vez de reprogramar en silencio), final review period ("cada flashcard se muestra una última vez"), ensure mastery (tras un fallo, 2 aciertos seguidos), daily goal calculado, notificaciones si te atrasás.

Algoritmo propuesto: "estudiar hacia la fecha X"

Objetivo: para el conjunto C de ítems del examen, maximizar $\sum R_i(E)$ sujeto a un presupuesto diario y llegar a $R_i(E) \geq r_{\text{target}}$ (0,95) sin destruir la memoria de largo plazo. Se implementa como capa sobre FSRS usando $R(t,S)$ e $I(r,S)$:

```

Entrada: E (fecha), C (ítems), r_target = 0.95, buffer_final = 1-3 días, study_days (máscara
semanal), daily_capacity_minutes
Fase 0 – Diagnóstico: R_E = R(E - last_review, S) si Review; 0 si New. readiness =  $\sum R_E / |C|$ 
(se muestra como "preparación estimada")
Fase 1 – Learning period (hasta E - buffer_final - k):
  cuota_nuevas/día = ceil(nuevos_pendientes / días_de_estudio_restantes); cada nueva requiere 2
  aciertos para salir de la fase; DR del ámbito = 0.92
Fase 2 – Consolidación (todo el periodo):
  interval = min( I(DR_exam, S'), (E - buffer_final) - hoy ) → ningún repaso queda después del
  examen
  DR_exam sube linealmente 0.92 → 0.95 en las últimas 2 semanas y a 0.97 en los últimos 3 días
  si  $I(r_{\text{target}}, S') \geq (E - hoy)$ : el ítem "llega" con  $R \geq r_{\text{target}}$  → no forzar más repasos (evita
  massed practice)
Fase 3 – Final review window (E - buffer_final ... E - 1):
  cola = ítems por R_E ascendente, luego por peso del tema en el blueprint; "ensure mastery": un
  Again exige 2 aciertos seguidos
Catch-up: si backlog(C) > capacidad → subir el daily goal para C y posponer ítems de otros niveles
Post-examen (E + 1): quitar el override; ofrecer "pasar a Mantenimiento" (DR 0.85) o "archivar"; S
y D quedan intactas

```

El tope `min(I, E - buffer)` es la única intervención sobre el scheduler: las actualizaciones de S y D siguen siendo las de FSRS, y el `review_log` guarda el `scheduled_days` real para no contaminar el optimizador. El "readiness" ($\sum R_E$ ponderado por blueprint) es la métrica de producto. Si el usuario llega 2 días antes con 500 tarjetas nuevas, la app

lo dice (estimado total de tiempo, como RemNote) y ofrece el modo urgente. Conviene validar la política con el simulador de fsrs-rs antes de lanzar.

Simulacros de examen

- **Blueprint:** tabla tema → peso % (importable del temario o generada desde las fuentes) × nivel Bloom × dificultad (30/50/20); el simulacro muestrea proporcionalmente, priorizando ítems con R baja pero incluyendo ≥ 20 % de "seguros" para medir sobreconfianza.
- **Formatos:** MCQ con distractores plausibles (los "near misses" son la mejor fuente), V/F, cloze, respuesta corta con rúbrica IA, ordenar pasos, emparejar, oclusión de imágenes, casos.
- **Tiempo y scoring:** límite total y por ítem (MCQ 60–90 s, corta 3–4 min); `time_ms` por pregunta; puntaje bruto, por tema y ponderado; umbral de aprobación.
- **Item analysis:** dificultad p, discriminación (punto-biserial), análisis de distractores, tiempo medio; ítems con $p > 0,95$ o discriminación $< 0,1$ se marcan para revisión de la IA.
- **Realimentación al scheduler:** cada respuesta es un review con `context = 'exam_sim'`: correcto → Good (Hard si tardó $> 2 \times$ su mediana); incorrecto → Again; opción "no afectar mi programación" (modo preview).
- **Parallel forms:** dos formas A/B por celda del blueprint, deduplicadas contra los quizzes de lección (el fallo de Oboe); el simulacro usa A, el examen final B.
- **Post-examen:** puntaje vs readiness predicho (calibración), temas débiles, ítems fallados con sus fuentes, curva de fatiga por posición; acierto < 60 % en un módulo → prioridad $\times 1,5$ y remediación; error confiado → tarjeta de contraste; acierto rápido y seguro → prioridad $\times 0,8$.

Ejercicios interactivos en el mismo scheduler

Toda actividad produce `{ correct, partial ∈ [0,1], time_ms, hints_used, attempts }` y un mapeo determinista a rating:

Tipo de ejercicio	Again (1)	Hard (2)	Good (3)	Easy (4)
Flashcard / cloze (autoevaluada)	el usuario elige (4 botones con preview del próximo intervalo)			
Escribir respuesta (fuzzy)	similitud $< 0,6$	0,6–0,85 o con pista	$\geq 0,85$	$\geq 0,85$ y tiempo $<$ mediana personal $\times 0,6$
Opción múltiple	incorrecta	correcta al 2.º intento o tiempo $> 2 \times$ mediana	correcta al 1.º	correcta, rápida y con confianza alta declarada
Ordenar pasos	> 1 par fuera de orden	1 par fuera de orden	correcto	correcto y rápido
Emparejar (n pares)	< 70 %	70–99 %	100 %	100 % sin errores previos y rápido
Problema numérico / código con tests	falla	pasa con pista o > 2 intentos	pasa	pasa a la primera y rápido
Respuesta corta con rúbrica IA	$< 0,5$	0,5–0,79	0,8–0,94	$\geq 0,95$
Pronunciación (score API)	$< 0,5$	0,5–0,75	0,75–0,9	$\geq 0,9$
Simulacro de examen	incorrecta	correcta pero lenta	correcta	— (no usar Easy en examen)

Reglas: Hard nunca se asigna a una respuesta incorrecta ("si apretás Hard cuando fallaste, los intervalos serán irrazonablemente altos"); Easy solo con señales fuertes (w16 acelera mucho); el `exercise_score` continuo se guarda en el log para análisis futuro; un ejercicio compuesto genera reviews para los skills que usa; los ejercicios tienen D/S propios (un procedimiento puede ser difícil aunque sus hechos sean fáciles). Los juegos con azar (memory, word search, arcade con distractores en movimiento) *no* alimentan el scheduler.

De la ruta de aprendizaje al sistema de memoria

- 1. **Generación:** por lección, la IA produce teoría, 4–8 actividades y 3–8 ítems candidatos que quedan en "Need to Learn" (no programados), como en RemNote.
- 2. **Completar la lección** = práctica inmediata → los ítems pasan a New con `due = ahora`, respetando la cuota de nuevas del nivel; si la práctica fue un ejercicio válido, se registra como primer review (contexto `lesson`).
- 3. **Importancia inicial:** propuesta por la IA + nivel de la ruta (una ruta "para examen" hereda Urgente); el usuario ajusta con un gesto.
- 4. **Refuerzos en la ruta:** cada 3–5 lecciones, un nodo que compone (a) vencidos de la ruta, (b) ítems de lecciones anteriores con $R < 0,8$ aunque no estén vencidos (Advance), (c) 2–3 ejercicios de aplicación intercalados, (d) mini-quiz con feedback.
- 5. **Desbloqueo:** no bloquear por olvidos; sí mostrar "esta lección se apoya en 4 ítems que hoy recordás al 60 %; repasalos (3 min)".
- 6. **Fin de ruta:** examen final por blueprint y transición automática a Normal o Mantenimiento según el resultado, con explicación del costo.

Composición de la sesión diaria

```
presupuesto = minutos_objetivo (usuario); streak_goal = 10 tarjetas (mínimo para no romper la racha)
1. Exámenes activos: cola por R_E ascendente, luego peso de tema ← siempre primero
2. Vencidos por nivel: Urgente → Alta → Normal → Mantenimiento; dentro del nivel, "relative overdueness"
   o R ascendente (Anki 24.11: "descending retrievability... mejor cuando hay backlog"), hermanas dispersadas
3. Reaprendizaje intercalado según sus pasos (10 min)
4. Nuevos: cuota por nivel; 1 nuevo cada 3–5 repasos; si backlog > 1,5 días de capacidad → 0 nuevos salvo Urgente
5. Módulo de refuerzo de la ruta si toca hoy
6. Final drill (opcional / modo urgente): todo lo calificado Again/Hard hoy vuelve al final
Sobrecarga: si vencidos × tiempo_mediano > presupuesto → auto-postpone empezando por Mantenimiento y mayor S;
   mostrar "hoy hiciste 80 %, pospuse 40 tarjetas de mantenimiento".
```

Presentación: "hoy: 35 repasos (~12 min) + 8 nuevos + refuerzo" y el streak goal para los días malos.

Métricas que la app debe mostrar

Métrica	Definición operativa	Precedente
True retention	% de aciertos sobre el primer repaso del día de tarjetas en Review con intervalo ≥ 1 d; young (< 21 d) vs mature; ventanas día/semana/mes/año	Anki
Desired vs true retention	Comparación por nivel; alerta si difieren > 5 pp (re-optimizar o ajustar DR)	fsrs4anki
Retención media hoy	$\text{mean}(R_i(\text{hoy}))$ sobre Review: "hoy recordarías el X % de todas tus tarjetas"	Expertium
Conocimiento memorizado	$\Sigma R_i(\text{hoy})$ y su serie temporal, por tema/ruta	FSRS Helper
Distribución de S y D	Histogramas; % con $S > 21$ d y > 365 d; D promedio por tema	Anki
Forecast	Tarjetas y minutos por día a 30/90 días, por nivel, con y sin nuevas	Anki / RemNote
Simulación de workload	reviews/día y minutos/día a 1 año para un DR dado	fsrs-rs
Heatmap y rachas	Días con repaso; color por meta (streak / daily / stretch); racha actual y máxima	RemNote, Review Heatmap
Tiempo por tarjeta y por hora	Mediana y p90 por tipo; aciertos por hora del día	Anki

Métrica	Definición operativa	Precedente
Answer buttons	% Again/Hard/Good/Easy por estado; alerta si Again > 20 % o Hard se usa como fallo	Anki
Knowledge decay forecast	$\sum R_i(\text{hoy} + k)$ para $k = 7, 30, 90$ si no se repasa nada: "si parás hoy, en un mes recordarás X %"	derivado de $R(t, S)$
Mastery por tema / lección	Etapas New · Acquiring · Growing · Solidifying · Retaining · Stale definidas por S y R (Retaining: $S \geq 90$ d y $R \geq 0,85$; Stale: $R < 0,7$)	RemNote
Readiness de examen	$\sum w_{\text{tema}} \cdot \text{mean } R_E$ y proyección; comparación con puntaje real del simulacro (calibración)	propio
Leeches y calidad del modelo	Lapsos $\geq$ umbral, pares con interferencia, tiempo anómalo; log loss / RMSE del optimizador y fecha de última optimización	Anki

Modelo de datos del sistema de memoria

```
-- Unidad de conocimiento (≈ "note" de Anki / Rem de RemNote)
CREATE TABLE items (id TEXT PK, lesson_id TEXT, topic_id TEXT, kind TEXT /*fact, concept,
procedure...*/,
    fields JSON, source_id TEXT, locator JSON /*página, timestamp, selector*/, as_of TEXT,
    importance TEXT CHECK(importance IN ('urgent','high','normal','maintenance','paused')),
    created_by TEXT, created_at INT, updated_at INT, deleted_at INT);
-- Unidad programable: cada tarjeta/ejercicio tiene su propio estado FSRS
CREATE TABLE cards (id TEXT PK, item_id TEXT, template TEXT /*basic, reverse, cloze:c1,
occlusion:3, mcq, order_steps...*/,
    payload JSON, state INT DEFAULT 0, due INT, stability REAL DEFAULT 0, difficulty REAL DEFAULT 0,
    scheduled_days INT DEFAULT 0, learning_steps INT DEFAULT 0, reps INT DEFAULT 0, lapses INT
DEFAULT 0,
    last_review INT, suspended INT DEFAULT 0, buried_until INT, leech INT DEFAULT 0,
    importance_override TEXT, exam_id TEXT);
CREATE INDEX cards_due ON cards(due) WHERE suspended = 0;
-- Historial inmutable: fuente de verdad para optimizer, stats, rollback y sync
CREATE TABLE review_logs (id INTEGER PK, card_id TEXT, reviewed_at INT, rating INT /*0
Manual,1..4*/,
    state_before INT, due_before INT, stability_before REAL, difficulty_before REAL, elapsed_days
INT,
    scheduled_days INT, learning_steps INT, duration_ms INT,
    context TEXT /*daily, lesson, reinforcement, exam_sim, cram, manual_postpone*/, exercise_score
REAL, device TEXT);
CREATE TABLE scheduler_profiles (id TEXT PK, scope TEXT, w JSON, decay REAL, trained_at INT,
n_reviews INT, log_loss REAL, rmse REAL);
CREATE TABLE importance_levels (name TEXT PK, desired_retention REAL, max_interval_days INT,
order_rank INT,
    postpone_allowed INT, new_per_day INT, leech_action TEXT);
CREATE TABLE exams (id TEXT PK, title TEXT, date TEXT, scope JSON, blueprint JSON,
target_retention REAL,
    final_window_days INT, study_days_mask INT, status TEXT);
CREATE TABLE exam_attempts (id TEXT PK, exam_id TEXT, started_at INT, finished_at INT, score REAL,
by_topic JSON, items JSON, readiness_predicted REAL);
```

Guardar `reviewed_at` en UTC y un `day_start_hour` (Anki y fsrs-optimizer usan las 4 a. m.) para que "mismo día" sea consistente.
`review_logs` + `cards` bastan para reconstruir el estado con `f.reschedule(card, history)` tras cambiar parámetros o importar.

Interfaz del scheduler (pluggable)

```
interface SchedulingOptions { desiredRetention: number; maxIntervalDays: number; learningSteps:
StepUnit[]; relearningSteps: StepUnit[];
  fuzz: boolean; loadBalance?: (candidates: Date[]) => Date; easyDays?: Record<0|1|2|3|4|5|6,
'normal'|'reduced'|'minimum'>; }
interface Scheduler { id: 'fsrs6' | 'sm2' | string;
  preview(card, now, opts): Record<Grade, { card; log }>; apply(card, now, grade, opts): { card;
log };
  retrievability(card, at): number; intervalFor(retention, state): number; reschedule(card,
history, opts): Card;
  rollback(card, log): Card; forget(card, now, resetCounts): Card; }
interface Optimizer { train(logs, opts): Promise<{ w: number[]; logLoss: number; rmse: number }>;
simulate(cfg): SimulationResult; }
```

El load balancer y los easy days se aplican *después* del intervalo (como Anki): dentro de [min_ivl, max_ivl] del fuzz se elige el día con menos vencimientos, evitando los días "mínimo" del usuario.

Notificaciones y ciclo de vida del optimizador

- Recordatorio diario a la hora preferida con conteo de vencidos y tiempo estimado (toast de Windows, respetando Focus Assist): "12 tarjetas del examen de Fisiología (3 min) — mantén tu racha". Alertas de examen (atraso vs daily goal, inicio de ventana final, readiness bajo umbral), de backlog (> 2× el promedio de 30 días) y de racha en riesgo. Nunca más de 2 al día; easy days también para notificaciones.
- Optimizador: arrancar con los parámetros por defecto; a partir de ~400–1.000 reviews ofrecer "optimizar" en un worker mostrando log loss/RMSE antes y después (aceptar solo si mejora, como el "health check" de Anki); re-optimizar cada 2^ª reviews o mensualmente; parámetros por usuario (opcionalmente por dominio con ≥ 1.000 reviews); nunca reprogramar en masa salvo acción explícita; exportar/importar el historial en el CSV de fsrs-optimizer y en .apkg.

RIESGOS Y DECISIONES ABIERTAS

(1) Los mantenedores de FSRS anunciaron ralentizar cambios y explorar variantes ("-S" para repases del mismo día, "-F" con fatiga) en lugar de FSRS-7: abstraer el scheduler y guardar `algorithm_version`. (2) El modo examen es una síntesis propia (RemNote + helper Advance + fórmula cerrada): validar con el simulador. (3) Los umbrales Hard/Easy de los ejercicios automáticos son heurísticos: medir true retention por tipo y ajustar. (4) Verificar soporte win32-x64 del binding del optimizador antes de decidir entre napi y WASM.

6 Catálogo de actividades interactivas

Todo lo que existe en Articulate Storyline/Rise, Adobe Captivate, Moodle (core + plugins + H5P), Duolingo, Khan Academy Perseus, Wordwall, Educaplay, LearningApps, Quizizz, Kahoot y las apps de idiomas, reducido a una taxonomía unificada de 98 tipos con calificación, mapeo a memoria y complejidad.

Lo que se aprendió de cada catálogo

Fuente	Aporte verificado
Storyline 360	11 tipos de pregunta calificados (T/F, MC, multiple response, fill-in-the-blank, word bank, matching drag-and-drop / drop-down, sequence drag-and-drop / drop-down, numeric, hotspot), 9 de encuesta (Likert, pick one/many, which word, short answer, essay, ranking x2, how many) y 6 freeform (drag and drop, pick one/many, text entry, hotspot, shortcut key). Mecánica: sliders, dials, markers, estados, triggers, variables, capas, lightbox, question banks, result slides, simulaciones de software, imágenes 360° con variables de exploración.
Rise 360	Bloques de microaprendizaje: accordion, tabs, labeled graphic, process, escenario (ramificado con personajes), sorting activity, timeline, flashcards (grid/stack), button stack; 4 knowledge checks (MC, multiple response, fill in the blank, matching); quiz con feedback ampliado y reintentos. Demuestra que 8 bloques interactivos + 4 checks bastan para un producto líder de microlearning.
Adobe Captivate	Classic: 8 tipos de pregunta + pretest/knowledge check/random; ~40 learning interactions incl. 7 juegos (Hangman, Jigsaw, Memory, Millionaire, Word Search, Catch AlphaNums, Jeopardy) cuyo puntaje es "binario por jugar" — los juegos son engagement, no assessment; video interactivo con overlay slides y bookmarks (branching dentro del video); VR/360 con hotspots evaluables; simulaciones demo/training/assessment. All-new Captivate 13: componentes + widgets (Click to Reveal, Carousel, Hotspot, Drag-and-Drop, Timeline, Slider, Stack Card) y GenAI.
Moodle	17 tipos core (calculated x3, drag and drop into text / markers / onto image, description, essay, matching, Cloze embebido, MC, numerical, ordering, random short-answer matching, select missing words, short answer, T/F) + comportamientos (deferred, adaptive, interactive with multiple tries, immediate, certainty-based marking); ~35 tipos de terceros (STACK con CAS Maxima, CodeRunner, Formulas, Gapfill, Kprime, Pattern-match, Wordselect, Freehand drawing, GeoGebra, PoodLL recording...); módulos (Lesson ramificado, Workshop, Game con 8 juegos, VPL, PoodLL ReadAloud/Solo/MiniLessons, Mootyper, Level Up XP, Stash, Board).
H5P	53 content types oficiales (Interactive Video, Course Presentation, Branching Scenario, Drag and Drop, Drag the Words, Fill in the Blanks, Mark the Words, Memory Game, Image Hotspots, Find the Hotspot, Image Sequencing/Pairing/Juxtaposition, Flashcards, Dialog Cards, Speak the Words, Audio Recorder, Dictation, Essay, Interactive Book, Timeline, Virtual Tour 360, Arithmetic Quiz, Crossword, Sort the Paragraphs, Summary, Single Choice Set, Game Map...). Reproducible en Electron con h5p-standalone (MIT) capturando eventos xAPI; editar con la librería de Lumi arrastra GPL-3.0. "H5P" es marca registrada.
Duolingo	~18 tipos de ejercicio de idiomas (traducir con word bank o teclado, seleccionar imagen, marcar significado, missing word, tap/type what you hear, speak, match pairs, arrange words, read and respond, complete the chat, character tracing), 9 tipos en Stories, Math con manipulativos y puzzles (Cash Dash, Secret Equation, Magic Squares), Music con teclado en pantalla, Chess con puzzles; principios: Birdbrain + Session Generator, HLR, generación de ejercicios con LLM tipo "Mad Lib" + revisión humana.
Perseus (Khan)	30 widgets MIT en TypeScript/React con esquema público (expression, grapher, interactive-graph, number-line, numeric-input, orderer, matcher, categorizer, sorter, plotter, matrix, label-image, radio, dropdown...), con render y scoring separados (@khanacademy/perseus / perseus-score): reutilizable como librería para matemáticas.
Wordwall / Educaplay / LearningApps / Quizizz / Kahoot	34 / 20 / 21 plantillas y ~20 tipos de pregunta (draw, hotspot, labeling, graphing, math response, audio/video response, slider, drop pin...). Lección clave de Wordwall: el mismo contenido (pares término-definición) se re-renderiza en decenas de plantillas — contenido ≠ presentación .
Código y voz	Exercism/Boot.dev/Codecademy/DataCamp: tests ocultos, fix-the-bug, Parsons, fill-in code, predict output; ELSA/Speak/Praktika/BoldVoice: repeat after me por fonema, stress e intonation, minimal pairs, roleplay, read aloud con WPM (PoodLL); Duolingo English Test: c-test, highlight the answer, interactive listening.

Primitivas, envoltorios y calificadores

Todos los tipos se descomponen en **25 primitivas** (lo que hace el usuario: elegir uno/varios, V/F, escribir texto corto/número/expresión/largo, rellenar huecos, ordenar, emparejar, clasificar, señalar en imagen, etiquetar, arrastrar a zonas, marcar segmentos, construir frase con fichas, valor continuo, hablar, escuchar-y-X, trazar/dibujar, manipular objeto matemático, escribir/ordenar código, decidir en rama, explorar-y-completar, recordar libremente) × **8 envoltorios** (pregunta suelta, cloze en pasaje, tarjeta, juego con tiempo/vidas, media con puntos de control, escenario

ramificado, presentación/libro, mapa de juego) × **7 calificadores**: **DET** determinista, **FUZ** difuso (normalización Unicode, distancia Damerau-Levenshtein relativa $\leq 0,2$, sinónimos, regex), **CAS** numérico/simbólico con tolerancia, **CODE** tests, **SPEECH** ASR/pronunciación, **AI** rúbrica, **SELF** autoevaluación.

Propiedades comunes (no tipos nuevos): barajado, partial credit, negative scoring, intentos máximos con pistas progresivas, typo tolerance, certainty-based marking, límite de tiempo, "sin pistas" (Legendary), feedback por opción, "Explain my answer", salto adaptativo según respuesta, switch template.

Mapeo a la escala 1-4 del scheduler: **M-bin** 1 = fallo; 2 = acierto tras pista, 2.º intento o $> 2 \times$ el tiempo esperado; 3 = acierto limpio; 4 = limpio en $< 50\%$ del tiempo o marcado "fácil". **M-pct** $p < 0,5 \rightarrow 1$; $0,5-0,8 \rightarrow 2$; $0,8-1 \rightarrow 3$; 1 sin pistas $\rightarrow 4$. **M-self** el usuario elige. **M-ai** la rúbrica devuelve rating y el usuario puede corregirlo. **M-speech** $\geq 0,9 \rightarrow 4$; $\geq 0,75 \rightarrow 3$; $\geq 0,5 \rightarrow 2$. **M-none** no alimenta memoria.

Tabla maestra: 98 tipos

Calif.: DET / FUZ / AI / SELF / SPEECH / CODE / CAS / none. Repaso: S = apto para el sistema de memoria (con su regla M-*), N = solo lección. Gen. IA: facilidad de generar solo desde texto fuente. Media: qué hay que generar (TTS, img, vid). Compl.: S/M/L/XL de implementación (render + grader + generador).

#	Tipo (id)	Categoría	Calif.	Repaso	Gen. IA	Media	C.
1	flashcard_basic (anverso $\rightarrow$ reverso)	recall	SELF	S · M-self	fácil	no	S
2	flashcard_reverse	recall	SELF	S · M-self	fácil	no	S
3	dialog_cards ("lo sabía / no")	recall	SELF	S · M-self	fácil	no	S
4	cloze_typed (hueco tipeado)	recall	FUZ	S · M-bin/pct	fácil	no	S
5	short_answer	recall	FUZ (+AI respaldo)	S · M-bin	fácil	no	S
6	numeric_answer (tolerancia, unidades)	recall/math	DET	S · M-bin	fácil	no	S
7	list_recall (enumerá los N)	recall	FUZ set-match	S · M-pct	fácil	no	M
8	free_recall (explicá con tus palabras)	recall/production	AI	S · M-ai	fácil	no	M
9	self_check_statement (escribí, compará, puntuá)	recall	SELF	S · M-self	fácil	no	S
10	image_occlusion (ocultar etiquetas)	recall/visual	SELF	S · M-self	difícil	img + coords	M
11	spell_the_word (escuchá y deletreá)	production	DET	S · M-bin	fácil	TTS	S
12	dictation / listen_type	listening	FUZ	S · M-bin	medio	TTS	M
13	speak_repeat (read aloud / repeat after me)	speaking	SPEECH	S · M-speech	fácil	TTS modelo (opc.)	L
14	pronunciation_word (fonema/palabra con score)	speaking	SPEECH (API)	S · M-speech	fácil	TTS	L
15	mcq_single	recognition	DET	S · M-bin	fácil	no	S
16	mcq_multi (multiple response)	recognition	DET (partial / all-or-nothing)	S · M-pct	fácil	no	S
17	true_false	recognition	DET	S · M-bin	fácil	no	S
18	statement_set (Kprime / multiple T-F)	recognition	DET partial	S · M-pct	fácil	no	S
19	single_choice_set (ráfaga 1-de-N)	recognition	DET	S · M-pct	fácil	no	S
20	cloze_dropdown (select missing words)	recognition	DET	S · M-pct	fácil	no	S
21	cloze_wordbank (word bank / drag the words)	recognition/procedural	DET	S · M-pct	fácil	no	M
22	image_choice (elegí la imagen)	recognition/visual	DET	S · M-bin	medio	img	S
23	summary_builder (afirmación correcta por bloque; H5P Summary)	comprehension	DET	S · M-pct	fácil	no	S
24	mark_the_words / highlight	comprehension	DET partial	S · M-pct	fácil	no	M
25	odd_one_out	recognition	DET	S · M-bin	fácil	no	S

#	Tipo (id)	Categoría	Calif.	Repaso	Gen. IA	Media	C.
26	confidence_mcq (MCQ + certeza; CBM)	recognition/metacog.	DET ponderado	S · M-bin ajustado	fácil	no	S
27	matching_pairs (arrastrar / unir con líneas)	procedural	DET parcial	S · M-pct	fácil	no	M
28	matching_dropdown	procedural	DET parcial	S · M-pct	fácil	no	S
29	tap_pairs_timed (Match Madness)	game/recognition	DET + tiempo	S · M-pct	fácil	no	M
30	ordering_sequence (pasos, párrafos, cronología)	procedural	DET (exacto / adyacentes / Kendall)	S · M-pct	fácil	no	M
31	timeline_build (eventos en línea de tiempo)	procedural/visual	DET	S · M-pct	fácil	no	M
32	categorize (group sort)	procedural	DET parcial	S · M-pct	fácil	no	M
33	sentence_builder (fichas + distractores)	production	DET (varias válidas)	S · M-bin	fácil	no	M
34	anagram / unscramble	game/production	DET	S · M-bin	fácil	no	S
35	table_completion (celdas con huecos)	recall	DET/FUZ por celda	S · M-pct	medio	no	M
36	number_line_place	math/visual	DET tolerancia	S · M-bin	fácil	no	M
37	estimate_slider (slider / dial)	math/recall	DET gradual	S · M-pct	fácil	no	S
38	hotspot_click (find the hotspot)	visual-spatial	DET zona	S · M-bin	difícil	img + coords	M
39	hotspot_multi	visual-spatial	DET parcial	S · M-pct	difícil	img + coords	M
40	label_image (labelled diagram)	visual/recall	DET parcial	S · M-pct	difícil	img + coords	M
41	drop_pin (markers / mapa)	visual-spatial	DET distancia	S · M-pct	difícil	img/mapa	M
42	drag_drop_zones (objetos → zonas, N:M)	visual/procedural	DET	S · M-pct	difícil	img	L
43	image_sequencing	visual/procedural	DET	S · M-pct	medio	img	M
44	image_pairing (imagen ↔ imagen/texto)	visual	DET	S · M-pct	medio	img	M
45	image_hotspots_explore (labeled graphic informativa)	teoría/visual	none	N	difícil	img	M
46	image_juxtaposition (antes/después)	teoría	none	N	medio	2 img	S
47	character_tracing (orden de trazos)	production/visual	DET tolerante	S · M-bin	medio	trazos	XL
48	freehand_drawing (dibujá el esquema)	production/visual	SELF / AI-visión	S · M-self/ai	fácil	no	L
49	geo_map_click (región en mapa vectorial)	visual-spatial	DET	S · M-bin	medio	GeoJSON	L
50	reading_passage_qs (pasaje + preguntas)	comprehension	DET/FUZ	S · M-pct	fácil	no	S
51	c_test / read-and-complete (letras faltantes)	comprehension/production	DET	S · M-pct	fácil	no	M
52	complete_the_chat (mejor réplica)	comprehension	DET	S · M-bin	fácil	no	S
53	word_in_context (significado de la palabra resaltada)	comprehension	DET	S · M-bin	fácil	no	S
54	main_idea_title (título / idea principal / evidencia)	comprehension	DET	S · M-bin	fácil	no	S
55	essay_rubric (respuesta breve con rúbrica)	production	AI	S · M-ai	fácil	no	M
56	structure_strip (escritura andamiada)	production	SELF/AI	N/S · M-ai	fácil	no	M
57	notes_reflection (Cornell / documentation tool)	teoría/production	none	N	fácil	no	M
58	worked_example_steps (paso a paso con decisión)	procedural/math	DET por paso	S · M-pct	medio	no	M
59	listen_select (¿qué oís?)	listening	DET	S · M-bin	medio	TTS	S
60	listen_reconstruct (tap what you hear)	listening	DET	S · M-bin	medio	TTS	M
61	listen_comprehension_qs (audio + preguntas)	listening	DET	S · M-pct	medio	TTS	S
62	minimal_pairs	listening	DET	S · M-bin	medio	TTS ×2	S
63	listening_cloze	listening	FUZ	S · M-pct	medio	TTS	M

#	Tipo (id)	Categoría	Calif.	Repaso	Gen. IA	Media	C.
64	speak_free_prompt (hablá sobre el tema/foto)	speaking	SPEECH + AI	S · M-ai	fácil	img opc.	L
65	roleplay_chat (conversación con objetivo; texto o voz)	speaking/production	AI (objetivo, precisión, complejidad)	S · M-ai (limitado)	medio	TTS/ASR	L
66	shadowing_intonation (curva de entonación)	speaking	SPEECH (pitch)	S · M-speech	fácil	audio modelo	XL
67	expression_input (expresión algebraica)	math	CAS	S · M-bin	medio	no	L
68	calculated_variant (parámetros aleatorios + fórmula)	math	DET fórmula + tolerancia	S · M-bin	medio	no	M
69	matrix_input	math	DET	S · M-pct	medio	no	M
70	interactive_graph (punto/recta/función)	math	DET geométrico	S · M-bin	difícil	no	XL
71	plotter (histograma / dot plot / barras)	math	DET	S · M-pct	medio	no	L
72	arithmetic_sprint (cálculo mental contra reloj)	math/game	DET + tiempo	S · M-pct	fácil	no	S
73	secret_equation (Wordle matemático)	game/math	DET	S · M-pct	fácil	no	M
74	compare_fast (comparación binaria cronometrada)	game/math	DET	S · M-pct	fácil	no	S
75	manipulative (reloj, regla, transportador, fracciones)	math/simulation	DET	S · M-bin	medio	no	L
76	code_tests (función con tests visibles/ocultos)	code	CODE	S · M-pct	medio	no	L
77	fix_the_bug	code	CODE	S · M-pct	medio	no	L
78	parsons_problem (ordenar / indentar bloques)	code/procedural	DET	S · M-pct	fácil	no	M
79	code_fill_blanks	code	DET/regex	S · M-pct	fácil	no	S
80	predict_output	code/comprehension	DET (stdout exacto)	S · M-bin	fácil	no	S
81	sql_query (SQLite embebido; comparar result-set)	code	CODE	S · M-bin	medio	no	L
82	regex_task	code	DET	S · M-pct	fácil	no	S
83	terminal_task (CLI simulada)	code/simulation	DET estado	S · M-bin	difícil	no	XL
84	typing_drill (WPM / precisión)	procedural	DET	N	fácil	no	M
85	crossword	game/recall	DET	S · M-pct	fácil	no	L
86	word_search	game/recognition	DET	N	fácil	no	M
87	hangman	game/recall	DET ($\leq k$ fallos)	S · M-bin	fácil	no	S
88	memory_game (concentration)	game/recognition	DET	N	fácil	img opc.	M
89	arcade_select (whack-a-mole, balloon pop, maze chase, shooter)	game/recognition	DET	S · M-pct (sin azar)	fácil	no	L-XL
90	gameshow_ladder (Millionaire con comodines)	game/recognition	DET	S · M-pct	fácil	no	M
91	branching_scenario (árbol de decisiones con finales)	simulation/comprehension	DET (score de ruta) + AI opc.	S limitado · M-pct	medio	personajes/img opc.	L
92	media_checkpoints (video/audio con preguntas y saltos)	comprehension	por sub-actividad	S (sub-ítems)	difícil	vid/audio	L
93	software_simulation (clic/teclea sobre capturas)	simulation/procedural	DET (zonas + secuencia)	S · M-pct	difícil	capturas	XL
94	virtual_tour_360	simulation/visual	DET/completitud	N	difícil	img 360	XL
95	disclosure_block (accordion / tabs / process / timeline / flip / stack)	teoría	none (completitud)	N	fácil	no	S
96	likert_poll (encuesta)	survey	none	N	fácil	no	S
97	board_puzzle (ajedrez: mejor jugada)	procedural/game	DET (motor)	S · M-bin	difícil	no	XL

#	Tipo (id)	Categoría	Calif.	Repaso	Gen. IA	Media	C.
98	play_notes_rhythm (teclado musical, ritmo)	procedural/listening	DET (tono + tiempo)	S · M-pct	difícil	síntesis audio	XL

De las 98 filas, **89 son aptas para el repaso de memoria**; las 9 restantes (bloques de teoría, encuesta, mecanografía y juegos con azar o sin señal fiable de recuerdo) forman parte del catálogo de lección.

Reglas para el repaso espaciado

- **Núcleo de repaso** (alto valor por segundo, generables desde texto, sin media): tipos 1–9, 15–21, 24, 27–28, 30, 32–33, 50, 52–55, 58, 68, 78–80. Unos 30 tipos permiten repasar un mismo skill con variedad de formato y evitar memorizar la forma de la pregunta.
- **Progresión por skill** (Quizlet Learn + Duolingo Legendary): 1.ª exposición → reconocimiento (mcq / true_false / cloze_dropdown); estabilidad media → producción asistida (cloze_wordbank, sentence_builder, matching); estabilidad alta → producción libre (cloze_typed, short_answer, free_recall) y variantes "sin pistas".
- **Un ítem de memoria ≠ una actividad**. El scheduler programa skills; el generador de sesión elige en tiempo de ejecución qué tipo renderizar según estabilidad y modalidad disponible (¿micrófono? ¿imagen?), y el rating se calcula con la regla M-* de ese tipo.
- **Los tipos con azar o ruido** (memory game, word search, arcade con distractores en movimiento, tablero) no alimentan el scheduler; sirven como recompensa y variedad, como hacen Captivate y Blooket.

Priorización

MVP · 21 TIPOS · 10 FAMILIAS

Solo texto, sin media generada

choice: mcq_single, mcq_multi, true_false, statement_set, complete_the_chat · **cloze**: typed, dropdown, wordbank · **text_input**: short_answer, numeric_answer · **cards**: basic, reverse, dialog · **long_text**: free_recall, essay_rubric (AI) · **pairs**: matching_pairs · **ordering**: ordering_sequence, sentence_builder · **categorize** · **text_mark**: mark_the_words · **disclosure_block** para la teoría. Infraestructura: dnd, Markdown + KaTeX, graders DET/FUZ, un grader AI, FSRS, un esquema por familia por llamada.

FASE 2 · MODALIDADES Y MEDIA

Audio, imagen, código, juegos simples

listen_select, listen_reconstruct, dictation, spell_the_word, listening_cloze (TTS); speak_repeat (ASR local); image_choice, hotspot_click, label_image, drop_pin, image_occlusion (imágenes del documento o subidas; coordenadas asistidas por visión); code_fill_blanks, parsons, predict_output, code_tests (JS / Pydide); tap_pairs_timed, arithmetic_sprint, hangman, crossword; worked_example_steps, calculated_variant, estimate_slider, number_line; branching_scenario en texto; media_checkpoints sobre videos del usuario; confidence_mcq; reading_passage_qs, c_test, word_in_context, main_idea_title, summary_builder.

FASE 3 · MATEMÁTICA RICA, VOZ AVANZADA, PUENTES

Perseus, pronunciación, simulaciones, H5P

expression_input (CAS por muestreo con math.js), interactive_graph y plotter (reutilizando @khanacademy/perseus, MIT), matrix_input; roleplay_chat, speak_free_prompt, pronunciation_word, shadowing_intonation; freehand_drawing, character_tracing, geo_map_click; arcade, gameshow, memory, word_search; software_simulation, virtual_tour_360, manipulative; **h5p-bridge** (h5p-standalone en iframe sandbox, xAPI → rating); board_puzzle y play_notes solo si el producto entra en esos dominios.

Esquema JSON: envoltorio común + 22 familias de payload

Un `oneOf` de 90 ramas es frágil y caro en tokens. Se separa **type** (tipo concreto, decide renderer, prompt y estrategia de rating) de **payload.family** (22 familias de datos, deciden grader y validación), igual que Perseus (`WidgetOptions<Type, Options>`) y H5P (`library + params`). En cada llamada al LLM se pasa **solo el esquema**

de la familia que se quiere generar, con el `enum` de `type` reducido a los tipos permitidos.

```
interface ActivityBase<T, P> {
  id: string /*ULID*/; schemaVersion: 1; type: T; family: P['family']; lang: string /*BCP-47*/;
  prompt: RichText /*Markdown + $TeX$ + [[media:ID]] + ``code``*/; instructions?: string;
  media?: MediaRef[] /*{id, kind, src?, alt?, generate?: {by:'tts'|'image'|'user-upload', ...}}*/;
  hints?: RichText[]; explanation?: RichText /*"Explain my answer" estático*/; sources?:
  SourceRef[] /*{docId, span, quote}*/;
  skills: string[] /*conceptos que el scheduler programa*/; difficulty: 1|2|3|4|5; tags?:
  string[];
  grading: { method: 'det'|'fuzzy'|'ai'|'self'|'speech'|'code'|'cas'|'none'; partialCredit?;
  negativeScoring?; maxAttempts?; hintPenalty?; timeLimitSec?; shuffle?;
    fuzzy?: { caseSensitive?; ignoreDiacritics?; maxRelativeEditDistance?; synonyms?:
  string[][] }; numeric?: { absTol?; relTol?; units? } };
  review: { eligible: boolean; ratingStrategy: 'bin'|'pct'|'self'|'ai'|'speech'|'none';
  expectedSeconds?: number };
  payload: P;
}
// Familias: choice · text_input · cloze · long_text · pairs · ordering · categorize ·
image_target · text_mark · scale · speech
//          dialogue · branching · media_checkpoints · code · math · graph · grid_game · arcade ·
cards · disclosure · draw · (simulation)
interface GradeResult { score: number /*0..1*/; correct: boolean; perItem?: {id, correct,
  expected?, got?}[]; feedback: RichText;
  rating: 1|2|3|4|null; meta: { timeMs; attempts; hintsUsed; confidence?; engine? } }
```

Familia	Tipos que la usan	Datos clave del payload
choice	mcq_single/multi, true_false, statement_set, single_choice_set, image_choice, odd_one_out, complete_the_chat, word_in_context, main_idea_title, listen_select, minimal_pairs, confidence_mcq, compare_fast	sets[{stem, options[{id,text,media,correct,feedback}], multiple, min/maxSelect}] , layout, askConfidence
text_input	short_answer, numeric_answer, spell_the_word, dictation, predict_output, expression_input, regex_task	inputKind (text/number/math/letters/regex), answers[{value, isRegex, feedback}], numeric{value, tol, unit}, regexCases
cloze	cloze_typed/dropdown/wordbank, listening_cloze, code_fill_blanks, c_test, table_completion	mode, layout (inline/table/code), segments[text gap{id, answers, options, visiblePrefix}], bankDistractors, singleUseDraggables
long_text	free_recall, essay_rubric, structure_strip, self_check_statement, list_recall, notes_reflection	min/maxWords, sections, modelAnswer, keyPoints[{text, weight, aliases}], rubric[{criterion, weight, levels}]
pairs	matching_pairs/dropdown, tap_pairs_timed, image_pairing, memory_game	presentation (drag/dropdown/lines/tap-timed/memory), pairs[{left, right}], rightDistractors, timeLimitSec
ordering	ordering_sequence, timeline_build, image_sequencing, sentence_builder, anagram, parsons_problem, listen_reconstruct	items[{id, text, media, date, indent}], correctOrder, alternativeOrders, distractors, scoring (exact/adjacent-pairs/kendall/position), axis, checkIndentation
categorize	categorize, group_sort	categories[], items[{categoryIds}]
image_target	hotspot_click/multi, label_image, drop_pin, drag_drop_zones, image_hotspots_explore, image_occlusion, geo_map_click	image, mode, shapes[{kind rect/circle/polygon/region, coords, correct, label, info}], draggables[{targetShapelds}], tolerancePx
text_mark · scale · speech · dialogue · branching · media_checkpoints	mark_the_words; estimate_slider, number_line, likert; speak_*, pronunciation, shadowing; roleplay_chat; branching_scenario; video/audio con checkpoints	tokens/correctIds; min/max/correct{value, tolerancias}; mode + targetText + engine + thresholds; persona + scenario + goal + rubric + mustUse; nodes/choices/endings + scoring; media + checkpoints[{atSec, activity, onWrong}]
code · math · graph · grid_game · arcade · cards · disclosure · draw	code_tests, fix_the_bug, sql_query; calculated_variant, matrix, worked_example_steps; interactive_graph, plotter; crossword, word_search, hangman, secret_equation, sprint; arcade/gameshow; flashcards; accordion/tabs/process/timeline; freehand/tracing	language + runner + starterCode + tests[{code, expectedStdout, hidden, weight}] + limits; variables + answerFormula + tolerance; range + correct{type, coords} (compatible con Perseus); entries[{answer, clue}] + settings; questions[] + lives/lifelines; cards[{front, back, media}]; items[{title, body}]; strokes + evaluation

Arquitectura del "activity engine" (React)

```
packages/
  activity-schema/    zod por familia → JSON Schema (zod-to-json-schema); migraciones
  schemaVersion; fixtures
  activity-graders/  graders puros por familia + utilidades: normalización Unicode, Damerau-
  Levenshtein, set-match,
                      métricas de orden (exact/adjacent/Kendall), geometría de zonas, math.js
  equivalence por muestreo
  activity-ui/       React: <ActivityHost/>, un Renderer por familia, componentes compartidos
  (DragLayer, TokenBank,
                      MathField, AudioButton, ImageStage, CodeEditor)
  activity-ai/       prompts por type, generación con structured outputs, validación + repair, AI
  graders, "blind solve", media jobs
  activity-speech/   ASR local o Web Speech; pronunciation scoring (proveedor enchufable); TTS
  activity-code/     runners: JS en worker aislado / QuickJS-WASM, Python con Pyodide, SQL con
  sql.js, regex nativo
  memory-core/       FSRs (ts-fsrs); adaptador rating; relación skill ↔ actividades
  h5p-bridge/        (fase 3) h5p-standalone en iframe sandbox; xAPI → GradeResult
```

- **Registro de tipos:** cada tipo es un archivo (`types/mcq_single.ts`) que declara `{ type, family, Renderer, grader, validate, generator: { promptTemplate, schemaRef, needsMedia, itemsPerCall, sourceMode }, review: { strategy, expectedSeconds, progression }, capabilities: { offline, needsMic, needsSandbox } }`. Añadir un tipo = un archivo + un prompt + fixtures.
- **<ActivityHost/>**: máquina de estados `idle → presenting → answering → (hinting)* → checking → feedback → (retry | completed)` (XState o reducer); expone `useActivity()`; emite eventos xAPI-like (`activity.presented / answered / graded / completed / skipped` con verb, object.id, result.score/success/duration, context.skills) — lo que además hace trivial mapear eventos H5P al mismo bus. Responsabilidades: barajado determinista (seed por sesión), cronómetro, intentos, pistas con penalización, alternativa por teclado para todo drag-and-drop (como exigen Rise/H5P), feedback por opción, botón "Explicame" (llama a la IA con actividad + respuesta + explicación), cálculo del rating con `toRating(result, review)`.
- **Graders:** DET puros y testeables con fixtures; FUZ = `normalize → exact → synonyms → regex → edit distance` relativa ($\leq 0,2$) → (opcional) AI fallback; CAS = `math.js` evaluando ambas expresiones en N puntos aleatorios + simplificación básica (STACK/Maxima queda fuera de escritorio); CODE en proceso aislado con límites, nunca en el renderer; SPEECH = `asr-match` viable offline con whisper, scoring por fonema vía API; AI = rúbrica con salida JSON `{score, rating, feedback, evidence[]}`, siempre mostrando la respuesta modelo y permitiendo sobrescribir el rating.
- **Generación desde fuentes:** ingesta → chunks con anclas → extracción de skills (conceptos, definiciones, procesos, pares término-definición, listas, fórmulas, código) → matriz skill-kind × type (definición → flashcard/cloze/mcq; proceso → ordering/worked_example; clasificación → categorize/matching; fórmula → numeric/calculated; código → code_fill/parsons/predict_output; idioma → sentence_builder/complete_the_chat/dictation) → generación con esquema de familia + reglas fijas (nº de opciones, longitud, "la respuesta no puede aparecer en el enunciado", distractores plausibles) → validación en tres capas: zod (forma) → reglas por tipo (exactamente 1 correcta en `mcq_single`, ids únicos, todo gap referenciado, correctOrder es permutación, targetShapelds existen) → **"blind solve"** (otro modelo resuelve el ítem sin ver la clave; si discrepa, `needsReview`) → bucle de repair (máx. 2) → media asíncrona (`pending-media` no entra en sesión) → SQLite.
- **Sesiones:** lección = teoría (disclosure, cards, hotspots informativos, texto) + práctica (6–12 actividades con variedad de familias, ≤ 2 con media, 1 de producción al final); repaso = el scheduler devuelve skills vencidos y el generador elige tipo según progresión y modalidad, intercala, evita repetir forma y aplica M-*. "Mistakes review" y "Legendary" (sin pistas ni banco de palabras) son políticas, no tipos nuevos.

DECISIONES QUE CONVIENE TOMAR YA

(1) Adoptar "contenido ≠ presentación": los skills se generan con datos crudos (pares, listas, pasos) y las actividades se derivan por plantilla además de la generación directa. (2) Rating por reglas explícitas desde el día 1, registrando `timeMs` / `attempts` / `hintsUsed` en cada intento para recalibrar. (3) Un esquema por familia por llamada al LLM y validación "blind solve". (4) Registro de tipos abierto para que la fase 3 y el contenido H5P no obliguen a tocar el núcleo.

7 Generar rutas de aprendizaje con IA

El botón "Generar con IA": cómo convertir un libro, un curso en video y unos links en una ruta fija, con lecciones de teoría y práctica, refuerzos, examen final y diagnóstico de conocimientos previos, sin alucinar y por unos pocos dólares.

Lo que enseñan los productos de referencia

- **Nadie genera "rutas fijas completas"** desde fuentes: Gemini Notebook produce artefactos sueltos; los course builders (Mindsmith, Coursebox, Thinkific, LearnWorlds) generan outlines y borradores para creadores, con quejas de contenido genérico, "recall-only" y alucinaciones; Oboe genera un curso desde un prompt pero es superficial y repite las preguntas del quiz en el examen.
- **Learn Your Way** (Google Research) es la mejor evidencia: misma fuente, múltiples formatos, chequeo de comprensión por sección → +9 % inmediato y +11 pp de retención a 3–5 días (RCT, N = 60).
- **Duolingo** genera ejercicios con un prompt "Mad Libs" (reglas fijas + variables de nivel/tema/tipo), pide 10 y los diseñadores eligen 3: **generar de más y filtrar**. Estima la dificultad de ítems con ML alineado a CEFR sin pilotaje (Settles 2020).
- **LearnLM** (Google) documenta los dos obstáculos que nos afectan: "dificultad para verbalizar intuiciones pedagógicas en prompts" y "falta de buenas prácticas de evaluación" → rúbricas explícitas y jueces con anclas.
- **Sana Learn** es el único producto enterprise con placement test para "no estudiar lo que ya sabés"; **Kinnu** (10.000 personas) muestra que las preguntas embebidas y los pathways por concepto funcionan y el chatbot suelto no.
- **Khanmigo** hace revisar los ítems MCQ por el docente antes de asignarlos: *human-in-the-loop antes de publicar* — nuestro "preview editable".

El pipeline en 10 etapas

1 • INGESTA Y NORMALIZACIÓN (LOCAL, GRATIS)

PDF (pdf.js + heurística de headings; OCR si escaneado), DOCX/EPUB/MD (headings nativos), web (Readability/Defuddle), imágenes (OCR + descripción), audio (Whisper local), video/cursos (ASR + keyframes en cambios de escena + OCR de slides). Todo termina en un `SourceDoc` con árbol de secciones y bloques con `locator` (página, timestamp, anchor) y hash.

2 • CHUNKING CON ESTRUCTURA + CONTEXTO

Nunca cortar por tamaño fijo si hay estructura: chunk = sección; si > ~1.200 tokens, dividir por párrafos con 10–15 % de solapamiento; si < ~150, fusionar. Transcripciones por ventanas de 60–90 s. "Contextual retrieval" (Anthropic): 50–100 tokens de contexto por chunk con modelo cheap (~35/–49/–67 % de fallos de recuperación con embeddings / +BM25 / +reranker).

3 • EXTRACCIÓN POR CHUNK (MAP, CHEAP, BATCH)

Haiku 4.5 o Gemini Flash-Lite con schema estricto: `concepts[]` (nombre canónico, alias, definición, tipo concept/procedure/fact/principle/example/misconception), `claims[]` citables con `block_ids`, objetivos candidatos (verbo Bloom), prerequisites mencionados, dificultad 1–5, ejemplos, figuras, ejercicios.

4 • SÍNTESIS DEL GRAFO Y EL OUTLINE (REDUCE, STRONG)

Sonnet 5 (u Opus 5) con el TOC + conceptos deduplicados por embeddings (coseno > 0,9): `KnowledgeGraph` (nodos con bloom_target, difficulty, importance, source_refs; aristas PREREQ_OF con confidence, RELATED_TO, PART_OF) y `Outline` (secciones → módulos → lecciones con 1–3 objetivos). Debe ser un DAG; se piden `misconceptions[]` por concepto (distractores y remediación). "Outline first, then expand" (Skeleton-of-Thought).

5 · SECUENCIACIÓN DETERMINISTA (CÓDIGO)

Topological sort estable sobre prerequisites con desempate por orden en la fuente primaria e importancia (mantener la narrativa del libro); spiral: conceptos importantes reaparecen como "retrieval warm-up"; lección = 600–1.200 palabras de teoría (3–6 min) + 4–8 actividades (4–8 min) = 1–3 objetivos, 2–5 conceptos; módulo = 3–7 lecciones + refuerzo (10–15 ítems, interleaving); cada 3–4 módulos un refuerzo acumulativo; examen final por blueprint.

6 · PREVIEW EDITABLE → PATHSPEC V1

Árbol sección → módulo → lección con minutos, conceptos, fuentes y avisos ("el capítulo 9 no se incluyó: apéndice"). Renombrar, reordenar (aviso si rompe prerequisites), fusionar/dividir, excluir, "profundizar", marcar "ya lo sé", fijar fuente primaria. Al confirmar, ids y orden se congelan.

7 · EXPANSIÓN EN BATCH (MID) + 2 LECCIONES EN TIEMPO REAL

Por lección: system prompt (rol, principios, contrato de fidelidad, formato), LessonSpec, chunks relevantes (mapeados + top-k por retrieval), resumen de lecciones previas, glosario, few-shots cacheados. Sonnet 5 vía Batch API (~50 %) con prompt caching; las dos primeras lecciones síncronas para que el usuario empiece en < 1 min.

8 · QA GATES

Schema → citas válidas (el span citado existe, fuzzy $\geq 0,85$) → faithfulness por claims (FactScore/RAGAS con modelo cheap: $\geq 0,9$ aprueba, 0,7–0,9 critic-editor, < 0,7 regenerar) → cobertura (conceptos con importancia $\geq 0,5$) → duplicados (coseno > 0,92) → variedad y Bloom → longitud → idioma/glosario → juez pedagógico (rúbrica 1–5 con anclas, modelo distinto al generador, temperatura 0) → self-critique (máx. 1 iteración).

9 · ITEM BANK

Ítems con módulo, conceptos, Bloom, difficulty_logit, usage[] (diagnostic, reinforcement, final_exam_A/B, remediation, mock), exposición y estadísticas; reglas NBME para MCQ; formas paralelas A/B deduplicadas frente a los quizzes de lección.

10 · PERSISTENCIA + MEMORIA

LearningPath v1 con GenerationManifest (hashes de fuentes, versiones de prompts y schemas, modelos, temperatura, tokens, costo, warnings); flashcards a "Need to Learn" con importancia inicial; ítems de módulos "known" sembrados con estado equivalente a un Good y prioridad baja.

Anatomía de una lección (Gagné + Merrill + CLT)

1. **Hook** y objetivos en lenguaje del aprendiz ("al terminar podrás...").
2. **Activación**: 1–2 preguntas de recuperación de lecciones previas.
3. **Explicación** con citas [S3 p. 112] en cada afirmación no trivial; un párrafo por idea; una analogía o ejemplo concreto por concepto.
4. **Worked example** completo y, si aplica, un completion problem.
5. **Diagrama como código** (Mermaid flowchart/sequence/mindmap, tabla) junto al texto que lo explica (sin split-attention).
6. **Misconceptions** frecuentes ("Error típico: ... Por qué está mal: ...").
7. **Resumen** de 3–5 bullets + glosario bilingüe.
8. **Práctica**: ≥ 3 tipos distintos, máximo 40 % MCQ, ≥ 1 actividad de nivel "aplicar" o superior, dificultad creciente; MCQ con 4 opciones, una correcta inequívoca, distractores derivados de misconceptions (generar 6–8 y quedarse con 3 tras el crítico: los distractores "buenos" rondan el 47–59 %, Bitew 2023), feedback por opción, sin "todas las anteriores".
9. **Flashcards**: 3–8 por lección según las 20 reglas (§4), con importance, context_cue, interference_group y source_refs.

Contrato de fidelidad en el system prompt: "solo afirmá lo que está en <sources>"; cada afirmación sustantiva lleva [cite:id]; si necesitás conocimiento general, envuelvo en un bloque general_knowledge — la QA lo somete a verificación por preguntas (Chain-of-Verification).

Costo de generar una ruta desde un libro de 300 páginas

≈ 120.000 palabras ≈ 160k tokens en inglés (≈ 200k con el tokenizer de Claude 4.7+, que produce ~30 % más tokens); salida en español. Precios oficiales del 1-sep-2026.

Etapas	Modelo	Tokens in / out	Tiempo real	Batch
Extracción por chunk (≈ 180 chunks)	Haiku 4.5	220k / 60k	USD 0,52	USD 0,26
Contextualización de chunks	Haiku 4.5 + cache	≈ 200k	0,20	0,10
Síntesis grafo + outline (1–2 llamadas)	Opus 5 (Sonnet 5)	80k / 20k	0,90 (0,36)	— (síncrono)
Lecciones (≈ 40 × 14k in, 5k cacheados / 4k out)	Sonnet 5	560k / 160k	2,36	1,18
Faithfulness + dedupe + variedad	Haiku 4.5	340k / 40k	0,54	0,27
Critic-editor en ≈ 30 % de lecciones	Sonnet 5	170k / 50k	0,84	0,42
Item bank (≈ 150 ítems: diagnóstico, refuerzos, examen A/B)	Sonnet 5	100k / 40k	0,60	0,30
Total		≈ 1,7M / 0,37M	≈ USD 6,0	≈ USD 3,4

Con Opus 5 también en las lecciones: ≈ USD 10–12 en tiempo real, 6–7 en batch. Con Gemini Flash-Lite o GPT-5.6 Luna en las etapas cheap, esas filas bajan 3–5×. Un curso de video de 20 h (transcript ≈ 240k tokens + OCR de ≈ 800 keyframes ≈ 100k) queda en el mismo orden (USD 4–8) más el ASR (gratis local). El pipeline alternativo "single-shot 1M + cache" en Sonnet 5 (escribir cache 1 h del libro y leerlo en 31 llamadas) cuesta ≈ USD 3,46 y da mejor visión global para el outline.

Determinismo, versionado y multi-fuente

- **Manifest por ruta:** hashes de fuentes, versiones de prompts y schemas, modelo y versión, temperatura (0 en extracción, jueces y calificación; 0,5–0,7 en redacción), seed si existe, tokens y costo, warnings.
- Los LLM no son deterministas ni a temperatura 0: la reproducibilidad viene de **persistir salidas** y de que la secuenciación sea código puro. Regenerar = nueva `PathSpec.version` con diff por lección; el progreso migra por `concept_id`, no por posición.
- **Idempotencia:** cada llamada tiene `custom_id = hash(stage, input_ids, prompt_version)`; si existe resultado, no se repite (clave con Batch API y con reanudar tras cerrar la app).
- **Multi-fuente:** el grafo es la unión; la fuente primaria (la más estructurada o la elegida) fija la narrativa; las demás aportan ejemplos, videos y ejercicios ("ver también: lección 4 del curso, 12:30"); contradicciones entre fuentes se detectan en la QA de claims y se muestran ("las fuentes difieren...").
- **Idioma:** generar directamente en español desde fuentes en inglés, con glosario bilingüe y citas textuales en el idioma original + traducción; para aprender inglés, la lección va en español y los ítems en inglés con can-do CEFR.

Esquemas (extracto)

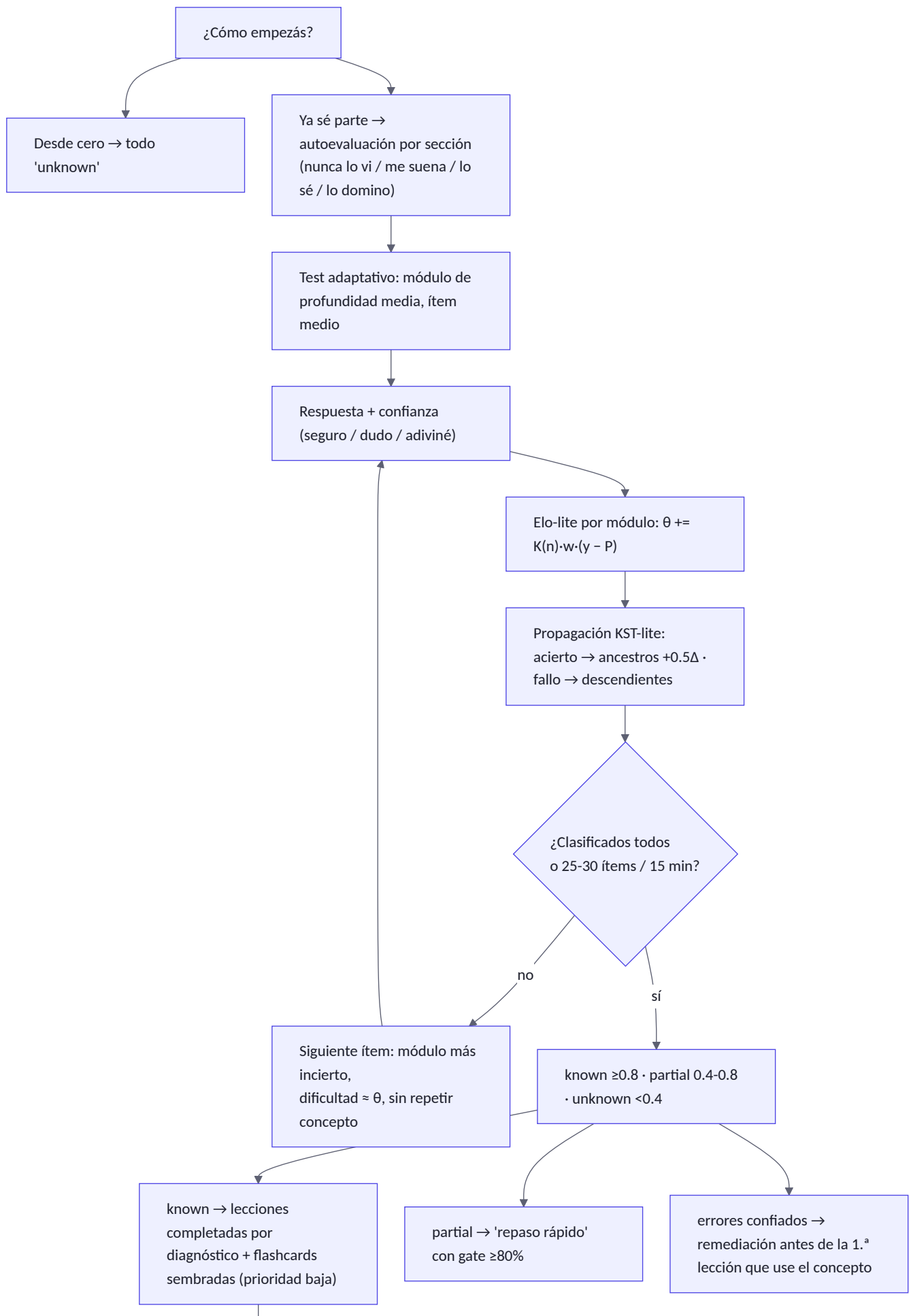
```
LearningPath.v1 { id, version, title, language (BCP-47), level, goal, target_date?, sources[],
knowledge_graph,
    sections[{ id, title, modules[{ id, title, objectives[], lessons[LessonRef],
reinforcement: AssessmentRef, diagnostic_items[] }] }],
    final_exam: AssessmentRef, manifest: GenerationManifest }
Lesson.v1      { id ("L07" | "L07.r1"), module_id, kind: core|remediation, title, objectives[{
text, bloom, abcd{audience,behavior,condition,degree} }],
    concept_ids[], prerequisite_lesson_ids[], estimated_minutes,
    theory: { blocks[{ type:
hook|activation_question|explanation|example|worked_example|diagram|misconception|summary|glossary
|general_knowledge, content (Markdown), citations[] }] },
    activities[Activity.v1], flashcards[Flashcard.v1], citations[{ id, source_id,
block_ids[], locator, quote }],
    qa: { faithfulness, pedagogy_score, coverage_ok, warnings[] } }
Activity.v1     { id, type, bloom, difficulty 1-5, concept_ids[], misconception_ids[], prompt,
options[{ id, text, correct, feedback, misconception_id }],
    answer, grading: { mode:
exact|numeric_tolerance|symbolic|regex|tests|rubric_llm|reference_llm, rubric[], reference_answer,
tests[], partial_credit },
    hints[], explanation, source_refs[], estimated_seconds, memory_item: { create,
importance } }
Flashcard.v1    { id, type: basic|reverse|cloze|image_occlusion|example_to_concept|contrast,
front, back, cloze_text, context_cue,
    concept_ids[], importance 0-1, interference_group, source_refs[], lesson_id }
ItemBankItem.v1 { id, activity,
usage[diagnostic|reinforcement|final_exam_A|final_exam_B|remediation|mock], module_id,
    difficulty_logit ((difficulty-3)·0,8, ajustado por Elo), discrimination_hint,
exposure, stats{ n, p_correct } }
GenerationManifest.v1 { created_at, source_hashes, prompt_versions, schema_versions, models{
stage: { provider, model, temperature, seed } },
    cost{ input_tokens, output_tokens, cached_tokens, usd }, warnings[] }
```

Structured outputs de Claude: `output_config.format = json_schema` y `strict: true` en tools; no admite `min/max`, `pattern` ni referencias recursivas (el SDK las pasa a descripciones); límites ≤ 20 tools estrictas, ≤ 24 parámetros opcionales, ≤ 16 uniones; incompatible con la feature de citations → las citas viajan como ids dentro del JSON y se verifican en código. Compatible con Batch (~50 %).

Los 11 prompts del pipeline

Prompt	Modelo · temp.	Entradas → salida
P1_extract_chunk	cheap · 0	chunk + contexto → conceptos, claims, objetivos, misconceptions, dificultad
P2_synthesize_outline	strong · 0,3	TOC + conceptos consolidados + config → KnowledgeGraph + Outline + warnings (DAG, tamaño de lección, spiral, cobertura, excluir front-matter)
P3_write_lesson	mid · 0,6	LessonSpec + chunks + resumen previo + glosario → Lesson (principios pedagógicos + contrato de fidelidad + presupuesto)
P4_make_activities	mid · 0,7	lección + misconceptions → 2-3× actividades, luego filtro; restricciones de variedad
P5_make_flashcards	mid · 0,3	20 reglas operativas → Flashcard[]
P6_faithfulness	cheap · 0	claims + chunks → soportado / no soportado / contradice, con cita
P7_pedagogy_judge	mid (distinto) · 0	rúbrica 1-5 con anclas → puntuación + edits concretas
P8_edit	mid	aplica solo las edits sin tocar citas verificadas
P9_items	mid · 0,7	blueprint → ítems por celda, formas A/B, dificultad estimada, reglas NBME
P10_grade	mid · 0	rúbrica + referencia + fuentes → puntuación por criterio, evidencia, feedback, uncertain
P11_remediation	mid	concepto + misconception + errores + chunks → mini-lección de 3-5 min con 1 worked example + 3 ítems

Diagnóstico de conocimientos previos



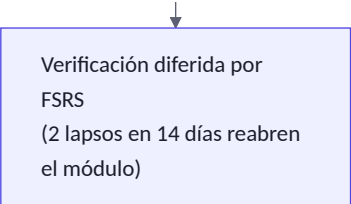


Figura 7.1 · "Outline-driven adaptive quiz": Elo-lite por módulo con propagación por prerequisites.

Enfoque	Qué aporta	Qué se descarta
CAT + IRT (1PL/2PL/3PL)	Selección adaptativa por información, criterios de parada, arranque medio, control de exposición; reduce ~50 % la longitud	Calibración con ~1.000 examinados por ítem: imposible con ítems recién generados
Knowledge Space Theory / ALEKS	El DAG de prerequisites infiere: acierto → evidencia sobre ancestros; fallo → sobre descendientes; 25-30 preguntas localizan el estado entre millones	Construir la estructura con datos de miles de alumnos
BKT	Formalizar guess/slip: una respuesta "seguro" pesa más que una "adiviné"	Estimar parámetros por skill sin datos
DKT / SAKT	Nada en v1	Todo (overkill sin datos)
Elo (Pelánek 2016)	$\theta \leftarrow \theta + K \cdot (y - P)$, $P = \sigma(\theta - d)$, K por incertidumbre $U(n) = a/(1 + b \cdot n)$; rinde como IRT bajo selección adaptativa y funciona online; dificultad inicial = estimación del LLM (viable según DET)	Con un solo usuario la dificultad de ítems se calibra lento: aceptar ruido
Duolingo / Sana / autoevaluación	La UX "¿empiezo de cero o hago el test?"; saltar módulos; no preguntar lo que el usuario dice no saber	Fiarse solo de la autoevaluación (sobreconfianza)

Algoritmo: (1) "Desde cero" → todo unknown; "ya sé parte" → autoevaluación por sección en 4 niveles; lo marcado "nunca lo vi" no se pregunta. (2) Arranque en el módulo de profundidad media del DAG con su ítem medio. (3) Respuesta + confianza → peso $w = 1,0 / 0,6 / 0,3$; $\theta_m += K(n) \cdot w \cdot (y - P)$ con $K(n) = 1/(1 + 0,05 \cdot n)$ escalado a $\approx 0,8$ en el primer ítem; un error "seguro" se registra como **misconception confiada**. (4) Propagación: ancestros $+0,5 \cdot \Delta\theta$ si $\Delta\theta > 0$; descendientes $+0,5 \cdot \Delta\theta$ si $\Delta\theta < 0$; máximo dos niveles. (5) Siguiente ítem: módulo más incierto ponderado por importancia; dificultad $\approx \theta$; sin repetir concepto; máximo 3 por módulo. (6) Clasificación: $P = \sigma(\theta)$; known $\geq 0,8$ con $n \geq 2$ (o un ítem "apply" correcto y seguro con ancestros known); partial 0,4-0,8; unknown $< 0,4$. (7) Parada: todos clasificados, 25-30 ítems, 12-15 min o abandono (estado parcial guardado). (8) DiagnosticResult con acciones: `mark_completed`, `seed_memory` (estado "reviewed_good", prioridad baja), `insert_remediation`. **Verificación diferida:** 2 lapsos en 14 días sobre ítems del mismo módulo, o R media $< 0,7$, reabren el módulo.

Política de remediación

- **Disparadores:** refuerzo de módulo $< 70\%$ en un concepto; ≥ 2 lapsos en 14 días o R media $< 0,7$ en tarjetas del concepto; error confiado en diagnóstico o examen; el mismo `misconception_id` fallado 2 veces; el usuario pide "no lo entiendo".
- **Acción:** `Lesson(kind = remediation)` anclada al concepto, id derivado `Lxx.rN`, opcional, inmediatamente después de la lección actual (o antes de la primera que dependa del concepto); reutiliza flashcards (solo añade tarjeta de contraste); 3-5 min.
- **Límites:** 1 remediación activa por módulo y 3 por semana; dedupe por `concept_id`; a la tercera remediación de un concepto, sugerir volver a la lección core y bajar la estimación de dominio del módulo.
- **Trazabilidad:** cada remediación registra el disparador y su efecto (aciertos posteriores) para ajustar umbrales.

Calificación de respuestas libres

GPT-4 con few-shot alcanzó $\kappa = 0,70$ vs $0,75$ humano en respuestas cortas (Henkel 2024): adecuado para evaluación formativa con feedback; en el examen final la nota se muestra como "provisional" y el usuario puede pedir re-evaluación con otro modelo. Rúbrica por actividad (2-4 criterios con anclas 0/1/2), respuesta de referencia, `must_include[]` y `must_not[]`, `source_refs`; el juez devuelve puntuación por criterio, evidencia citada de la respuesta y feedback de 2-3 líneas; temperatura 0; evaluar dos veces con criterios permutados y promediar si difieren;

"Explain my answer" para cada error (por qué está mal, qué misconception activa, cita). Código: tests deterministas primero, luego el LLM explica fallos. Matemática: verificación simbólica/numérica (math.js; SymPy vía Pyodide). Speaking: STT → transcript visible → rúbrica CEFR. Guardas: el grader solo usa referencia, rúbrica y chunks; ante duda declara `uncertain` (no afecta Elo ni FSRS); detección de inyección en la respuesta del alumno.

UX del flujo "Generar con IA"

1 Panel de generación

con campos tipo plantilla: objetivo en una frase, nivel, idioma de las lecciones, ¿para un examen? fecha, ritmo (horas/semana), fuente primaria, alcance (todo / capítulos). Estimación en vivo de tiempo y costo ("≈ 6 min y USD 3,40").

2 Progreso por etapas

con streaming: "Leyendo 14 fuentes (9/14)", "Detectando 212 conceptos", "Construyendo mapa de prerrequisitos", "Proponiendo ruta". Cancelable y reanudable.

3 Preview editable del outline

(obligatoria): editar, excluir, marcar "ya lo sé", fijar fuente primaria; botón "¿Empiezo de cero o hago el diagnóstico?".

4 Diagnóstico

(12–15 min, barra de ítems restantes, confianza por respuesta, saltable) → resumen de lo que se marca completado, reversible.

5 Expansión:

las lecciones aparecen progresivamente; la primera lista en < 1 min; cada lección con "Regenerar", "Más ejemplos", "Reportar error" (abre la cita) e indicadores de QA (fidelidad, fuentes).

6 Finalización:

resumen (n lecciones, minutos, ítems creados en memoria, plan hasta la fecha X); versión 1 congelada; "Regenerar ruta" crea v2 con diff.

VEINTE TRAMPAS CONOCIDAS

Fidelidad (verificar spans, no solo ids) · recall-only sin restricciones de Bloom · examen que repite quizzes · sobre-generación de flashcards (30 por lección hunde la memoria) · chunking por tamaño que rompe tablas y listas · front/back matter que contamina el outline · ciclos y prerrequisitos inventados · dificultad estimada por LLM (prior, no verdad) · diagnóstico largo o punitivo · autoevaluación sobreconfiada (confirmar con ≥ 1 ítem apply) · no determinismo y drift de modelos · costos ocultos (reintentos, critic loops, 1M de contexto por lección) · mezcla de idiomas en términos · errores de ASR en términos técnicos (post-corregir con el glosario) · structured outputs sin `pattern / min/max` y JSON truncado por `max_tokens` · jueces sesgados (modelo distinto, anclas, temperatura 0) · privacidad local-first (las fuentes salen a la API; ofrecer "solo proveedor X" y redacción de PII) · latencia de Batch (hasta 24 h: generar las primeras lecciones síncronas) · ruta fija vs realidad (ofrecer "regenerar afectadas") · contenido médico/legal (advertencia: deriva de tus fuentes y puede tener errores).

8 Ingesta de fuentes y RAG local

Cómo entra cada tipo de fuente (PDF, DOCX, EPUB, PPTX, imágenes, audio, video, YouTube, web), qué se procesa gratis en tu PC y qué se manda a una API, y el diseño del índice local (SQLite + FTS5 + sqlite-vec) que alimenta el chat, la corrección y la generación.

Estrategia por tipo de fuente

Fuente	Herramienta (gratis, local, Node)	Cuándo escalar a OCR / VLM / API
PDF nativo (born-digital)	<code>pdfjs-dist</code> 6 en Node (texto + posiciones por página → citas navegables); <code>unpdf</code> como wrapper; <code>@hyzyla/pdfium</code> (MIT) para renderizar páginas a PNG (thumbnails, OCR). <code>PyMuPDF</code> es mejor pero AGPL.	Si una página devuelve < 50 caracteres o texto desordenado → escaneada → OCR. Solo páginas con figuras van al modelo con visión.
PDF escaneado / fotos de apuntes manuscritos	<code>Tesseract.js</code> 7 (Apache 2.0, WASM, 100+ idiomas) para impreso; débil en manuscrito y layout.	Siempre VLM/OCR: Gemini 2.5 Flash-Lite como OCR (≈ USD 0,25 por 1.000 páginas), LlamaParse (10.000 créditos gratis/mes), Mistral OCR 4.1 (USD 4 / 2 batch por 1.000 páginas, con bounding boxes). Local avanzado: PaddleOCR-VL 1.6 o Docling en sidecar Python (v2).
DOCX	<code>mammoth</code> (BSD-2) → HTML/Markdown con headings, listas, tablas, imágenes.	Ecuaciones OMML: <code>mammoth</code> las ignora → extraer del XML o Pandoc.
EPUB	<code>spine</code> + XHTML por capítulo (<code>fflate</code> + parser); lector con <code>foliate.js</code> (vendorizado) o <code>epub.js</code> .	Nunca.
PPTX	Descomprimir y parsear <code>ppt/slides/slideN.xml</code> (≈ 150 líneas propias) u <code>officeparser</code> ; imágenes vía unzip.	Slides con diagramas como imagen → VLM opcional.
Imágenes con texto	<code>Tesseract.js</code> (impreso); descripción con modelo multimodal barato (Qwen3.7-flash, Gemini Flash-Lite: 258 tokens por imagen).	Manuscrito o mixto → Gemini / Mistral.
Fórmulas y tablas	<code>pix2tex/LaTeX-OCR</code> (MIT) en sidecar pequeño; <code>pdf.js</code> posiciones + heurística (frágil).	Math-heavy → Mistral OCR, Marker balanced (83,9 % en math), Gemini; tablas complejas → Docling/Marker/Mistral.
Audio (clases, podcasts)	Whisper local: Parakeet-TDT-0.6b-v3 (25 idiomas europeos, WER medio 6,34 %, CC-BY-4.0) o <code>whisper-large-v3-turbo</code> vía <code>sherpa-onnx</code> (Node addon, CUDA) o <code>whisper.cpp</code> CLI bundleado; timestamps por segmento.	Fallback cloud: AssemblyAI Universal (USD 0,15/h + 0,02 diarization; 185 h gratis), ElevenLabs Scribe v2 (0,22/h, archivos de 10 h), Groq whisper-turbo (0,04/h, trozos < 25 MB).
Video / cursos tipo Udemý	<code>ffmpeg</code> (build LGPL) extrae audio 16 kHz mono para Whisper y keyframes por cambio de escena (<code>select='gt(scene,0.3)'</code>) o muestreo cada 10 s + dHash (Hamming < 8 = duplicado) → 20–60 keyframes/hora; OCR/descripción de cada keyframe (258 tokens en Gemini/Qwen); fusión "lo que dice + lo que muestra" en chunks de 2–5 min; el índice del curso (carpetas/módulos) ya es un outline candidato.	Nunca enviar el video entero: 20 h = 72.000 s × 263 tokens/s ≈ 18,9M tokens (no cabe en 1M; ≈ USD 14 en Gemini 3.7 Flash). Transcripción cloud opcional: Gemini audio 32 tokens/s → 20 h ≈ 2,3M tokens ≈ USD 1,7.
YouTube	1) <code>youtube-transcript</code> (npm, gratis; endpoints internos; funciona mejor con IP residencial); 2) si falla, Gemini con la URL pública (8 h de video/día gratis; ~100 tokens/s en baja resolución + 32 de audio: 10 min ≈ USD 0,024 con Flash, 0,008 con Flash-Lite) pidiendo transcript con timestamps + capítulos.	Data API <code>captions.download</code> solo para videos propios. yt-dlp viola los ToS de YouTube : fuera del producto base (como mucho, plugin opcional bajo responsabilidad del usuario).
Web	Defuddle (MIT, de Obsidian; conserva notas al pie, ecuaciones y bloques de código) o <code>Readability</code> + <code>JSDOM</code> → Markdown con <code>Turndown</code> ; <code>BrowserWindow</code> oculto para SPAs (si el HTML estático trae < 500 palabras).	Jina Reader (gratis 20 RPM; 10M tokens con key) o Firecrawl (1.000 créditos/mes) para sitios de documentación completos.
Udemý / Coursera / Vimeo	Sin API de descarga y con términos que la prohíben: el usuario aporta sus archivos locales (grabaciones propias, PDFs del curso) y la app transcribe local.	Documentar que el material con copyright se procesa localmente y no se redistribuye.

Regla: born-digital → extracción local gratis; escaneado, manuscrito, fórmulas o layout complejo → OCR con VLM. Detectar automáticamente por densidad de texto extraíble y presencia de imágenes a página completa. Con Node puro (`pdf.js`, `mammoth`, `epub`, `pptx`, `Tesseract.js`, `Transformers.js`, `sqlite-vec`, `Mermaid`, `Defuddle`) se resuelve el 80 % de las

fuentes sin instalar nada; el sidecar Python (Docling/PaddleOCR-VL, ComfyUI, pix2tex, faster-whisper) es un "paquete avanzado" descargable, no parte del instalador (PyInstaller es el número uno en falsos positivos de antivirus).

OCR y parsing cloud: precio por 1.000 páginas

Servicio	USD / 1.000 págs	Salida	Math / manuscrito	Gratis
Gemini 2.5 Flash-Lite como OCR	≈ 0,23	Markdown/JSON/LaTeX a pedido	Muy bueno en manuscrito; math bueno	Free tier
Gemini 3.1 Flash-Lite / 3 Flash	≈ 0,8 / ≈ 1,6	ídem	Mejor (Flash 3.5: 76,4 olmOCR-bench)	Free tier
Mistral OCR 4.1	4 (batch 2)	Markdown + imágenes + bounding boxes + confianza	170 idiomas; olmOCR-bench 85,2	No
LlamaParse	1,25 Fast · 3,75 Cost-effective · 12,50 Agentic	Markdown/JSON con layout	Agentic usa VLMs	10.000 créditos/mes
Datalab (Marker API)	4 (accurate 10)	Markdown/HTML/JSON	Chandra 2: 85,8	USD 10–20 de crédito
Claude Haiku 4.5 como OCR	≈ 4,4 (batch ~50 %)	Markdown	Muy bueno en manuscrito	No
Azure Document Intelligence / Google Document AI / Reducto	1,5–10 / 1,5–30 / 10–20	JSON	Manuscrito sí	500–1.000 págs/mes
Local: Docling (MIT) · PaddleOCR-VL 1.6 (Apache) · Marker v2 · MinerU · Chandra 2	0	Markdown/JSON	Docling 50,3 (flojo en escaneos); PaddleOCR-VL 96,3 % OmniDocBench; Marker 76 (pesos con licencia comercial sobre USD 2M–5M de ingresos)	Sidecar Python

Dos libros escaneados de 300 páginas al mes: Gemini Flash-Lite ≈ USD 0,15 · LlamaParse USD 0 (dentro de cuota) · Mistral batch USD 1,20.

Embeddings, rerankers y vector store

Modelo	USD / 1M tokens	Dims	Contexto	Multimodal	Gratis
OpenAI text-embedding-3-small / large	0,02 / 0,13 (batch 0,01)	1536 / 3072 (Matryoshka)	8.191	No	No
Google gemini-embedding-2 (preview)	0,15 (Vertex) · no leído en Gemini API	128–3072	8.192	Sí (texto, ≤ 6 imágenes, audio ≤ 180 s, video ≤ 120 s, PDF ≤ 6 págs)	Free tier
Voyage voyage-4 / 4-lite / 4-large (recomendado por Anthropic)	0,06 / 0,02 / 0,12 (batch ~33 %)	flexible	32K	voyage-multimodal-3.5 aparte	200M tokens por familia
Cohere Embed 4	0,12 (imagen 0,47)	256–1536	128K	Sí	Trial no comercial
Jina v4 / v5-omni	no publicado por token	2048	32K	Sí (v5-omni: audio y video)	10M tokens
Local: EmbeddingGemma-300M (768 dims, Matryoshka, 100+ idiomas, ≈ 300–600 MB) · bge-m3 (567M, 1024 dims, 8K, dense + sparse) · multilingual-e5-small (384 dims, 120 MB) · qwen3-embedding 0.6B/4B	0	Vía @huggingface/transformers 4.x (ONNX; WebGPU/WASM en renderer, CPU/CUDA en Node; tutorial oficial para Electron) u Ollama /v1/embeddings . Un libro de 300 páginas: 1–3 min en GPU, 10–20 en CPU.			

Embeber un libro de 300 páginas (~160–200k tokens) cuesta **USD 0,004** con text-embedding-3-small y USD 0 con Voyage (dentro de los 200M gratis) o local: los embeddings son irrelevantes en el presupuesto. El costo real del RAG está en la contextualización (Haiku 4.5 batch ≈ USD 0,30 por libro; Gemini Flash-Lite ≈ 0,10) y en el reranking (Cohere

Rerank 4: USD 2–2,5 por 1.000 búsquedas; Voyage rerank-3-lite: 0,02 por 1M tokens con 200M gratis; local: mxbai-rerank-base-v2 o bge-reranker-v2-m3, gratis, 0,2–1 s por 20 documentos en CPU). Guardar `model_id` por embedding y nunca mezclar espacios; reindexar como job.

Vector store	Tipo	Índice	FTS / híbrido	Estado y licencia	Veredicto
sqlite-vec (<code>vec0</code>) 0.1.9	Extensión de SQLite (mismo archivo que la app)	Fuerza bruta (sin ANN); int8/binary; partition keys y metadata	Con FTS5 en la misma DB (RRF en SQL)	Pre-v1 (breaking changes posibles), Apache/MIT, financiado por Mozilla; binarios por plataforma	v1 si el corpus < ~200k chunks: un solo archivo, transacciones, backups triviales
LanceDB 0.38 (<code>@lancedb/lancedb</code>)	Embebido, archivos Lance	IVF-PQ, HNSW	BM25 (Tantivy) + híbrido + rerankers integrados; blobs	Estable, Apache 2.0, Windows x64	Cuando > 200k chunks o búsqueda multimodal
Vectra · Orama · hnswlib-node · Chroma · pgvector · sqlite-vss					
Prototipos · búsqueda instantánea en UI · si sqlite-vec resulta lento · requiere servidor Python · para el SaaS futuro · abandonado (descartar)					

Diseño del RAG local

- Parser** (tabla anterior) → Markdown + metadata → **chunker** semántico por sección de 300–500 tokens con solapamiento 10–15 %, ruta de encabezados como metadata (`Libro > Cap. 3 > 3.2`), tablas y bloques de código enteros, fórmulas KaTeX inline; transcripciones por ventanas de 60–90 s alineadas a pausas con `start/end` ("saltar al minuto"); páginas web por H2/H3.
- Contextualización** opcional (toggle "índice mejorado" con costo mostrado): 50–100 tokens de contexto por chunk con Haiku 4.5 Batch o Gemini Flash-Lite.
- Embeddings**: EmbeddingGemma/bge-m3 local por defecto; text-embedding-3-small o voyage-4-lite en la nube (casi gratis).
- Almacenamiento**: SQLite (better-sqlite3) con `chunks` , `chunks_fts` (FTS5, tokenizer `unicode61_remove_diacritics 2` + trigram para español) y `chunks_vec` (sqlite-vec, int8, partition por `source_id`); un solo archivo por biblioteca.
- Búsqueda híbrida**: top-50 BM25 U top-50 vectorial → Reciprocal Rank Fusion → reranker local → top-10–20 al LLM, con `chunk_id` → `block_ids` para citas exactas.
- Late chunking / ColBERT**: no en v1 (índices 10–50× más grandes); `voyage-context-4` como alternativa cloud si hiciera falta.

Benchmarks de chunking (NVIDIA 2024, Chroma): page-level y recursive ~400–512 tokens rinden bien (recall 88–89 %); el semantic chunking gana 2–3 puntos a mucho más costo; la brecha entre mejor y peor estrategia llega al 9 %. Para nosotros el chunk sirve para citar y para alimentar prompts de lección, no solo para recuperar: priorizar límites semánticos (headings) sobre tamaño.

Costo mensual de ingesta (usuario intensivo)

Rubro	Ruta barata	Ruta premium
OCR de 600 páginas escaneadas	Gemini Flash-Lite USD 0,15 (o LlamaParse gratis)	Mistral OCR batch 1,20 / estándar 2,40
Embeddings (840k tokens: 2 libros + 20 h de transcripciones + 50 webs + 10 videos)	Local USD 0 (cloud 3-small 0,017)	voyage-4 0,05
Contextual retrieval (640k tokens)	Off: 0	Flash-Lite ≈ 0,15 / Haiku 4.5 batch ≈ 0,65
Reranking	Local 0	Cohere ≈ 1 (500 búsquedas)
Web (50 páginas) y YouTube (10 videos)	USD 0 + ≈ 0,10 (5 sin captions vía Gemini URL)	Firecrawl / Supadata free tiers
Transcripción de 20 h de audio propio	Whisper local 0	gpt-4o-mini-transcribe 3,60 / AssemblyAI 3,00
Total ingesta	≈ USD 0,25	≈ USD 6–8

9 Inteligencia artificial: proveedores, precios y costos

Qué IA hace cada cosa, cuánto cuesta (precios oficiales verificados el 1-sep-2026) y cómo se abstrae para poder cambiar de proveedor sin tocar la app. Incluye texto, voz, pronunciación, imagen, diagramas y video, y el presupuesto mensual consolidado.

Modelos de texto: la foto de septiembre de 2026

El mercado cambió mucho respecto a 2025. Anthropic vende Claude **Sonnet 5 (USD 2/10)**, **Opus 5 (5/25)**, **Haiku 4.5 (1/5)** y el tier superior **Fable 5.1 / Mythos 5.1 (10/50)**; todos los 4.6+ tienen 1M de contexto a precio plano y 128K de salida. OpenAI reemplazó GPT-5.x por **GPT-5.6 Sol / Terra / Luna (4/20 promocional hasta el 21-nov, 2/12, 0,20/1,20)**. Google: **Gemini 3.7 Flash (0,75/3,75 hasta el 31-dic-2026; 1,50/7,50 desde 2027)**, 3.5 Flash-Lite (0,30/2,50), 3.1 Pro Preview (2/12). DeepSeek pasó a V4 Flash/Pro con tarifa pico/valle desde el 16-ago (Flash valle 0,22/0,66; la jornada argentina completa cae en valle).

Proveedor · modelo	ctx	In	Cache read	Out	Batch	Modalidades de entrada	JSON estricto
Anthropic · Fable 5.1 / Mythos 5.1	1M	10	0,25	50	50 %	texto, imagen, PDF nativo	✓
Anthropic · Opus 5	1M	5	0,50	25	50 %	texto, imagen, PDF	✓
Anthropic · Sonnet 5	1M	2	0,20	10	50 %	texto, imagen, PDF (600 págs por request)	✓ (sin beta header)
Anthropic · Haiku 4.5	200K	1	0,10	5	50 %	texto, imagen, PDF	✓
OpenAI · GPT-5.6 Sol / Terra / Luna	1,05M	4 / 2 / 0,20	0,40 / 0,20 / 0,02	20 / 12 / 1,20	50 %	texto, imagen, PDF (input_file)	✓ strict
Google · Gemini 3.1 Pro Preview	1M	2 (≤ 200K) / 4	0,20 / 0,40 + storage	12 / 18	50 %	texto, imagen, PDF (258 tok/pág), audio (32 tok/s), video (263 tok/s)	✓
Google · Gemini 3.7 Flash	1M	0,75 (2026) / 1,50	0,075	3,75 / 7,50	50 %	todo (imagen, PDF, audio, video)	✓
Google · Gemini 3.5 Flash-Lite · 2.5 Flash-Lite	1M	0,30 · 0,10	—	2,50 · 0,40	50 %	todo	✓
DeepSeek · V4 Flash (valle / pico)	1M	0,22 / 0,44	0,007	0,66 / 1,32	✗ (off-peak)	texto (visión solo -vision-exp)	JSON mode
Moonshot · Kimi K3 · K2.6	1M · 256K	3 · 0,95	0,30 · 0,16	15 · 4	— · 60 %	texto, imagen (K2.6: video)	JSON mode
Alibaba · qwen3.7-flash (≤ 32K) · qwen3.6-plus	1M	0,03 · 0,5	0,006 · —	0,13 · 3	?	imagen, texto, video · texto	OpenAI-compatible
Z.ai · GLM-5.3 · GLM-5.3-Flash (promo)	1M	1,4 · 0,075	0,26 · 0,015	4,4 · 0,25	—	texto · video/imagen/archivo	✓
xAI · Grok 4.6 · Grok 4.1 Fast	500K · 2M	2 · 0,20	0,50 · —	6 · 0,50	—	imagen	✓
Mistral · Medium 3.5 / Large 3 / Small 4	—	1,5 / 0,5 / 0,15	—90 %	7,5 / 1,5 / 0,6	50 %	imagen (OCR aparte USD 4/1.000 págs)	✓
OpenRouter (agregador)	—	precio del proveedor + 5,5 % de fee al comprar créditos (tarjeta, AliPay, USDC); routing con fallback; cache passthrough			✗	según modelo	✓

Mecánica relevante: prompt caching de Anthropic (escritura 1,25× a 5 min o 2× a 1 h; lectura 0,1×; mínimo 1.024 tokens en Sonnet 5, 4.096 en Haiku); Batch API 50 % en todo (hasta 100.000 requests, "la mayoría termina en menos de 1 h", máximo 24 h) compatible con caching y structured outputs; PDF nativo de Claude convierte cada página a imagen (300 páginas ≈ 0,9–1,3M tokens → extraer texto localmente y mandar solo páginas con figuras); Gemini cobra 258 tokens por página de PDF y su Files API retiene 48 h; el free tier de Gemini usa los datos para mejorar productos (el pago, no).

Calidad

Arena (ex LMArena, 1-sep-2026, texto): claude-fable-5 1507, claude-opus-4-6-high 1505, claude-opus-4-7-high 1502, claude-opus-5-high 1492 (#7), **gemini-3.7-flash-high 1491 (#9)**, kimi-k3-max 1489, gemini-3.1-pro-preview 1487, glm-5.3-max 1483, gpt-5.6-sol 1483. Artificial Analysis Intelligence Index v4.1.1: Opus 5 63 (USD 2,34 por tarea del índice), Fable 5 62, GPT-5.6 Sol 61, Grok 4.6 61, Kimi K3 60, GLM-5.3 60, Qwen3.8 Max 58, **GLM-5.3-Flash 57 (USD 0,09 por tarea)**, Gemini 3.7 Flash 56 (0,40), Sonnet 5 55 en modo max (1,72 por el thinking: usar `effort: low/medium` en tareas simples), DeepSeek V4 Pro 53. Lectura: Anthropic domina el top; **Gemini 3.7 Flash está a 1 punto de Opus 5 en preferencia humana costando 7× menos**; GLM-5.3-Flash y Qwen3.8-Flash son la mejor "inteligencia por dólar". No hay tabla pública por idioma: armar un eval propio de 50 ítems en español (teoría, corrección, tono) antes de fijar el modelo comercial (< USD 2 por corrida).

Matriz de proveedores por tarea

Tarea	Smart tier	Cheap tier	Fallback	Racional
Ingesta + ruta estructurada con citas	Claude Sonnet 5 (batch + cache 1 h; Opus 5 opcional)	Gemini 3.7 Flash (batch)	GPT-5.6 Terra; DeepSeek V4 Pro vía OpenRouter	1M plano, PDF nativo, JSON estricto, batch 50 %; USD 1,4–2,8 por libro
Teoría, flashcards, ejercicios JSON	Claude Sonnet 5 (effort medium)	Gemini 3.7 Flash / GLM-5.3-Flash	GPT-5.6 Terra	Structured outputs + streaming por ítem
Corrección de respuestas libres + explicación	Sonnet 5 (rúbrica cacheada)	Gemini 3.7 Flash	GPT-5.6 Luna	Volumen alto (500/mes): Flash ≈ USD 1,7; Sonnet ≈ 3,5
Chat / tutor con RAG	Sonnet 5 (cache de system + fuentes)	Gemini 3.7 Flash	DeepSeek V4 Flash / Qwen3.6-plus	Latencia y costo; citas desde el índice local
Diagnóstico de conocimientos previos	Sonnet 5	Gemini 3.7 Flash	GPT-5.6 Terra	Juicio pedagógico; bajo volumen
Clasificación, tagging, pistas	Qwen3.7-flash / GPT-5.6 Luna / DeepSeek V4 Flash	Local: qwen3.5:9b / gemma4:12b	Gemini 3.5 Flash-Lite	Centavos; local si el usuario lo activa
Visión (keyframes, diagramas)	Gemini 3.7 Flash	Qwen3.7-flash / Gemini 3.5 Flash-Lite	GLM-5.3-Flash	258 tokens por imagen en Gemini; Claude es 5–6× más caro por imagen
Audio (transcripción)	Gemini 3.7 Flash (32 tok/s)	Whisper local	Gemini 3.5 Flash-Lite	Claude y GPT-5.6 no aceptan audio
Embeddings	text-embedding-3-small / voyage-4-lite	Local EmbeddingGemma / bge-m3	Mistral embed	Gratis y privado; un solo espacio
OCR de escaneados	Mistral OCR 4.1	Gemini 2.5 Flash-Lite	LlamaParse free tier	USD 0,25–4 por 1.000 páginas

Costo mensual de texto (usuario intensivo)

Supuestos: 2 rutas nuevas (2 × 740k/130k tokens), 30 lecciones sueltas (510k/90k), 500 correcciones (1,25M/0,2M, ~60 % cacheable), 300 turnos de chat RAG (1,8M/0,15M, ~50 % cacheable) ≈ **5,0M de entrada / 0,7M de salida**.

Configuración	USD / mes
Todo Opus 5 sin optimizar / con batch + cache	42,7 / ≈ 33
Todo Sonnet 5 sin optimizar / con batch + cache	17,1 / ≈ 13
Todo GPT-5.6 Terra · Todo Gemini 3.1 Pro	17,9 · 18,5
Todo Gemini 3.7 Flash (2026 / 2027)	6,4 / 12,8
Todo GPT-5.6 Luna · Todo DeepSeek V4 Flash (valle / pico)	1,85 · 1,6 / 3,1
Híbrido recomendado (rutas y lecciones en Sonnet 5 batch; corrección y chat en Gemini 3.7 Flash; tagging local o Qwen)	≈ 8,8 (+30 % margen por thinking/reintentos ≈ 11,5)

Configuración	USD / mes
Híbrido premium (Opus 5 para rutas y explicaciones de errores)	≈ 18-22

Structured outputs, AI SDK 7 y abstracción de proveedores

- **Soporte:** Anthropic `output_config.format = json_schema` (gramática compilada, cache 24 h); OpenAI `text.format + strict: true` (Responses API, refusals explícitos); Gemini `responseJsonSchema` (subset amplio); DeepSeek/Kimi/Qwen/GLM JSON mode (validar con Zod y reintentar); Ollama/LM Studio con gramática local (JSON siempre válido).
- **Vercel AI SDK 7** (ESM-only, Node ≥ 22): `generateText / streamText + Output.object({ schema }) , Output.array` con `elementStream` (cada ítem llega completo y validado), `partialOutputStream`, errores `NoObjectGeneratedError`; proveedores first-party `@ai-sdk/anthropic`, `openai`, `google`, `deepseek`, `mistral`, `xai`, `Moonshot`, `Alibaba`; `@openrouter/ai-sdk-provider`; `@ai-sdk/openai-compatible` para LM Studio (`http://localhost:1234/v1`), Ollama (`:11434/v1`), Z.ai, MiniMax, Kimi, Qwen; `transcribe()` y `generateSpeech()` estables; políticas de aprobación de tools ("la IA quiere crear 40 cards, ¿confirmás?").
- **Outputs largos:** nunca la ruta como un JSON gigante: esqueleto (< 8K tokens) → una llamada por lección (2-6K) en paralelo/batch → flashcards/ejercicios con `Output.array` persistiendo cada ítem; si `finishReason === 'length'`, continuar "desde el ítem N" con el JSON parcial válido. Máximo de salida: 128K en Claude 5.x / GPT-5.6; 64K en Haiku 4.5 y Gemini 3.x.
- **Abstracción (packages/ai):** perfiles { `id`, `kind`, `baseUrl`, `keyRef`, `models[]`, `pricing {in, out, cachedIn, batch, date}`, `caps {pdf, image, audio, video, jsonStrict, maxOutput, ctx}` }; **roles** `smart | cheap | vision | audio | embed | local` mapeados por el usuario; toda llamada pasa por `runJob(role, schema, prompt, { batchable, cacheKey })`; adaptadores "premium" solo en tres puntos (cache, batch, PDF/Files); validación y reparación uniforme con Zod; middleware que registra `usage` (incl. `cachedInputTokens`, `reasoningTokens`) y calcula costo con una tabla de precios versionada y editable; fallback ordenado ante 429/5xx/timeout (o OpenRouter `:floor / :exacto`); tests de contrato con un golden set de 20 lecciones en español; concurrencia con `p-queue`; presupuesto mensual con alertas al 80/100 % y bloqueo opcional; el futuro proxy comercial es un perfil más (`kind: 'proxy'`).

Modelos locales en la RTX 4070 Super (12 GB)

Modelo (Ollama)	Descarga	Modalidades	Uso recomendado
qwen3.5:9b	6,6 GB	texto + visión, 201 idiomas, 256K	Todo-terreno: tagging, pistas, flashcards de notas, chat sobre notas (contexto ≤ 16K)
gemma4:12b	7,6 GB	texto + imagen, thinking, function calling	Mejor en español/redacción; corrección simple
gemma4:e4b	9,6 GB	texto + imagen + audio	Notas de voz locales
qwen3.5:4b / 2b	3,4 / 2,7 GB	texto + visión	Clasificación ultrarrápida
qwen3-embedding:0.6b / 4b · bge-m3 · embeddinggemma	0,6-2,5 GB	100+ idiomas	Embeddings del RAG
gpt-oss:20b · qwen3.8:27b	14 / 18 GB	—	No entran en 12 GB sin offload pesado

Throughput esperado (estimación): 9B ≈ 50-70 tok/s, 12B ≈ 35-50 tok/s; prefill 1-2K tok/s → un contexto de 30K tarda 15-30 s antes del primer token: mantener `num_ctx ≤ 16-32K`. **Qué no mandar local:** rutas desde libros completos, corrección con explicación matizada, JSON largos (> 4K tokens), diagnóstico. Regla: "local" es un proveedor más, opt-in, con fallback a nube.

Privacidad, límites y Argentina

- **Entrenamiento con datos de API:** Anthropic no ("we will not use your inputs or outputs... to train"); OpenAI no (abuse logs 30 días; store: false); Gemini **free tier sí** (con revisión humana), pago no; DeepSeek almacena en la RPC y puede usar los datos salvo opt-out; OpenRouter no loguea por defecto; Kimi/Z.ai/MiniMax/Qwen no verificados. Para el producto comercial: proveedores por defecto Anthropic/Google pago/OpenAI; los chinos como opción explícita del usuario y nunca para contenido sensible.
- **Límites para un usuario individual:** Anthropic 1.000 RPM / 2M ITPM / 400K OTPM desde el tier Start (tope USD 500/mes; los tokens cacheados no cuentan); OpenAI Tier 1 (USD 5) = 100/mes, Tier 2 (USD 50) = 500/mes; Gemini Tier 1 con USD 10 por ventana de 10 min de gasto (usar Batch para lotes grandes); Kimi 200 RPM con USD 10 acumulados.
- **Argentina** figura en las listas de países soportados de Anthropic, OpenAI y Gemini. Todos cobran créditos prepagos en USD con tarjeta; OpenRouter acepta tarjeta, AliPay y USDC (5,5 % de fee). No verificado: percepciones impositivas sobre consumos en moneda extranjera y compatibilidad de tarjetas locales con Stripe en cada consola → probar; si falla, OpenRouter con USDC o tarjeta prepaga internacional. Para el modelo comercial, contemplar que un usuario BYOK argentino paga 21–30 % extra de impuestos sobre créditos: argumento para un futuro "plan con créditos" en pesos.

Voz: evaluación de pronunciación

DECISIÓN

Azure AI Speech Pronunciation Assessment es el único proveedor que cubre los 6 idiomas de la v1 con scores de accuracy, fluency, completeness y salida por fonema. Precio verificado en la Azure Retail Prices API: **USD 1,00/h** (mismo meter que STT en tiempo real) **+ 0,30/h** si se activa prosodia; free tier F0 con 5 h/mes. SDK JavaScript (microsoft-cognitiveservices-speech-sdk) que funciona en el renderer (micrófono directo, con tokens de 10 min emitidos por main) o en main (push stream PCM 16 kHz / 16-bit / mono).

Locales con PA	Scores	Solo en-US
en-US, en-GB, en-AU, en-CA, en-IN · pt-BR, pt-PT · fr-FR, fr-CA · de-DE · it-IT · es-ES, es-MX (+ ar, ca, zh, da, nl, fi, hi, ja, ko, ms, nb, pl, ru, sv, ta, th, vi). No existen en-IE, en-NZ, es-AR como locales de evaluación.	AccuracyScore (fonemas vs nativo), FluencyScore, CompletenessScore (scripted), miscue (omisión/inserción/mala pronunciación), ProsodyScore (stress, entonación, velocidad, ritmo), PronScore agregado; granularidad Phoneme / Word / FullText con offset y duración; modos scripted (con referenceText) y unscripted; audio > 30 s en modo continuo (sin miscue).	Prosodia (+USD 0,30/h), content assessment (vocabulario/gramática/tema; retirado del SDK ≥ 1.46 → se hace con nuestro LLM), alfabeto IPA, nBestPhonemeCount (qué fonema dijiste en lugar del esperado: p. ej. [t] en vez de [θ]), syllable groups. En otros locales los fonemas vienen con nombre interno (no verificado el formato).

Proveedor	Idiomas v1	Fonema · fluency · prosodia · content	Precio efectivo	Piso mensual
Azure PA	6/6	✓ · ✓ · en-US · vía LLM	USD 1,00–1,30/h	0 (F0 5 h)
Speechace	en, fr, es	✓ + sílaba · Pro+ · stress/intonation · IELTS/CEFR/PTE/TOEIC	1,92/h overage	USD 40–125
SpeechSuper	en (US/UK/IN), fr, de, es	✓ · ✓ · stress · IELTS unscripted	≈ 1,4/h	USD 20
ELSA API	en	✓ · ✓ · pitch/volume · grammar/vocab, IELTS	1,92/h scripted · 4,80/h unscripted	0 (self-serve)
Language Confidence	en	✓ IPA · ✓ · — · IELTS/CEFR	no público	—
Google / Amazon	No ofrecen pronunciation scoring como API.			
GOP open source (wav2vec2-espeak + alineación)	6/6	✓ (PCC ≈ 0,5 vs 0,69 SOTA) · ✗ · ✗ · ✗	0	semanas de I+D (v2, offline fallback)

Proveedor	Idiomas v1	Fonema · fluency · prosodia · content	Precio efectivo	Piso mensual
LLMs de audio (Gemini, GPT-4o-audio)	todos	✗ como motor de scoring: en 1.800 utterances diagnostican por estereotipo del L1 (marcan errores de stress en 82-96 % de los audios cuando solo 4 % los tenía; solo 15,8 % combina razonamiento coherente y rating correcto). Sirven como verbalizadores del JSON de Azure + F0 (correctness de pitch pasa de 0,18 a 0,45-0,62 si se les dan valores de F0 en texto).		

"Quiero sonar irlandés": el Accent Lab

Ningún proveedor evalúa contra en-IE. La solución por capas: (1) scoring segmental con el locale más cercano (en-GB, o en-US si se quieren NBest y prosodia), mostrado como "inteligibilidad", no como "irishness"; (2) reglas fonológicas por acento curadas (Irish: /θ, ð/ → [t̪, d̪], /r/ rótico, FACE/GOAT monoptongales, /t/ final lenitado) para decidir con nBestPhonemeCount si [t] por [θ] es error o rasgo del acento objetivo; (3) voces de referencia nativas: Azure en-IE-EmilyNeural / en-IE-ConnorNeural, Polly Niamh, Deepgram Aura-2 Irish; (4) shadowing con comparación acústica propia: extraer F0 del TTS y del usuario (pitchy, McLeod Pitch Method; ventanas de 2.048 muestras a 16 kHz; descartar clarity < 0,9; semitonos respecto a la mediana), alinear con DTW sobre los timestamps de palabra y graficar ambos contornos superpuestos (correlación, rango tonal, ratio de duraciones); (5) minimal pairs por acento (tin/thin es casi homófono en irlandés → ejercicio de percepción); (6) opcional, un clasificador de acento (SpeechBrain CommonAccent, 16 acentos incl. Irlanda; no verificado) para una "distancia al acento objetivo" estilo BoldVoice (embeddings + regresión PLS); (7) el LLM redacta el coaching articulatorio en español. UX a copiar (ELSA): escuchar referencia → grabar → score global + palabras coloreadas → tap = fonema + consejo + "escuchar solo esta palabra" (SSML <phoneme>) → reintentar; sesiones ≤ 5 frases.

Voz: STT, TTS y tiempo real

STT	USD / h pregrabado	Streaming	Notas
Groq whisper-large-v3-turbo	0,04	—	216× realtime; 25 MB por archivo (trocear)
AssemblyAI Universal-2 / 3.5 Pro	0,15 / 0,21	0,15	Diarization +0,02/h; 185 h gratis
Azure batch / fast / real-time	0,18 / 0,36 / 1,00	1,00	Mismo recurso que PA; 5 h/mes gratis
OpenAI gpt-4o-mini-transcribe / gpt-4o-transcribe	0,18 / 0,36	1,02 (live)	25 MB por archivo
ElevenLabs Scribe v2	0,22	0,39	Archivos hasta 3 GB / 10 h; diarization 32 hablantes
Deepgram Nova-3 (mono / multi)	0,26 / 0,31	0,29 / 0,35	Keyterms; USD 200 de crédito; ~300 ms
Gemini 3.x Flash (audio-in)	≈ 0,12	—	Hasta 9,5 h por prompt; ideal para resúmenes, no SRT precisos
Local (RTX 4070 Super)	0	sí	Parakeet-TDT-0.6b-v3 (25 idiomas, WER 6,34 %; 20 h en 2-5 min), whisper-large-v3-turbo (10-15 min), whisperX con diarization (40-60 min); sherpa-onnx (Apache-2.0, Node addon, CUDA, trae Parakeet, Whisper, Moonshine, Silero VAD, Kokoro y Piper) es la vía más limpia; whisper.cpp CLI bundleado (CPU + CUDA) como alternativa sin compilar en el equipo del usuario

TTS	USD / 1M caracteres	Gratis	Acentos / locales	Notas
Azure Neural (default)	15 (Dragon HD 22)	0,5M chars/mes	154 locales, 767 voces: en-IE 2, en-GB 20, en-AU 19, en-IN 20, pt-BR 28, es-AR 2, es-ES 24, it-IT 26, de-DE 23, fr-FR 23	SSML completo (<phoneme> , <prosody> , estilos), visemes, streaming; voces MAI-Voice-2 con 18 estilos
ElevenLabs (premium, ya contratado)	≈ 100 (Multilingual v2 / v3) · ≈ 50 (Flash)	10k créditos	Voice Library filtrable por acento; voces irlandesas comunitarias	Planes Starter 6 / Creator 22 (220k chars v3) / Pro 99; relación créditos ↔ caracteres no verificada
Amazon Polly Neural / Generative	16 / 30	1M neural (12 meses)	en-IE Niamh; es-US, pt-BR	Speech marks

TTS	USD / 1M caracteres	Gratis	Acentos / locales	Notas
Google Chirp 3 HD / Neural2	30 / 16	1M	sin en-IE	—
OpenAI gpt-4o-mini-tts	≈ 15–20 (0,015/min; estimación)	—	instructions controla "accent" sin garantía	No usar como referencia de acento
Deepgram Aura-2	30	USD 200	EN 35 voces (US/UK/AU/Irish), ES 17; sin portugués	Latencia baja
Local: Kokoro-82M (Apache-2.0)	0	—	en-US 20, en-GB 8, es 3, fr 1, it 2, pt-BR 3	Corre en CPU en tiempo real; Chatterbox Multilingual (MIT, 23 idiomas, GPU) si se quiere más naturalidad; Piper es GPL; XTTS no comercial

Tiempo real (fase 2): Gemini 3.1 Flash Live (USD 0,005/min in + 0,018/min out ≈ 0,012–0,023/min) y gpt-realtime-mini (10/20 por 1M audio tokens ≈ 0,015–0,03/min) son los más baratos; gpt-realtime ≈ 0,05–0,10/min; ElevenLabs Agents 0,08/min + LLM; Azure Voice Live ≈ 0,03–0,06/min; pipeline propio con Pipecat/LiveKit Agents (open source) + Deepgram + LLM barato + Azure TTS ≈ 0,01–0,02/min. 10 h/mes de roleplay ≈ USD 8–18. Empezar con Gemini Live o gpt-realtime-mini por WebRTC directo desde Electron.

Implementación en Electron: `getUserMedia mono + AudioContext({ sampleRate: 16000 }) + AudioWorklet Float32 → Int16 LE` en frames de 20–100 ms (no confiar en el constraint de sampleRate: Windows entrega 48 kHz); MediaRecorder (WebM/Opus) solo para archivar; VAD con `@ricky0123/vad-web` (Silero v5) para recortar silencios (ahorra dinero) y detectar fin de frase; latencia esperada: Azure PA 0,5–1,5 s tras el fin de la frase, TTS Azure primer byte 200–400 ms, Kokoro CPU ≈ 0,3× realtime; guardar WAV 16 kHz + JSON de Azure (5–30 KB por frase) en `userData/audio/<yyyy-mm>/`; offline: STT/TTS locales y scoring aproximado marcado como estimado, encolando el audio para re-evaluar. Anti-sorpresas: tope diario de segundos evaluados, prosodia solo en en-US, caché de TTS por hash(texto, voz, estilo).

Imagen, diagramas y video

Generación de imagen	USD / imagen (~1K)	Texto legible	Notas
Google Nano Banana 2 Lite (<code>gemini-3.1-flash-lite-image</code>)	0,034	Bueno	Default barato; Interactions API; SynthID
Google Nano Banana 2 (<code>gemini-3.1-flash-image</code>)	0,067 (2K 0,101; 4K 0,151)	Diseñado para "infografías, menús, diagramas"	Edición conversacional multi-turno; hasta 14 referencias; batch ~50 %
Google Nano Banana Pro	0,134	Excelente	Premium
BFL FLUX.2 [pro]	0,03 por MP	4,83/5	Mejor relación precio/texto; multi-referencia
Seedream 4.5 (fal)	0,04	4,93/5 (#1 en texto)	—
Recraft V4 Vector	0,08 (Pro 0,30)	Bueno	Único con SVG editable nativo: iconos de ejercicios
OpenAI gpt-image-2	≈ 0,02–0,05 medium (terceros; OpenAI publica USD 30/M tokens de salida)	Muy bueno	Requiere verificación de organización; gpt-image-1.5/1-mini deprecados
xAI Grok Imagine · Qwen-Image · Ideogram	0,02 · 0,003–0,035 · 0,03–0,10 (no verificado)	Bueno · 4,64/5 · históricamente el mejor en tipografía	—
Local: FLUX.2 Klein 4B (Apache 2.0, FP8 ≈ 7 GB) · Z-Image Turbo (6B, Apache 2.0, 8 pasos)	0	Aceptable · bueno	ComfyUI como sidecar opcional; el usuario "normal" no tendrá esta GPU
Descartados	Midjourney (sin API, ToS prohíbe automatizar), Imagen 4 (apagado el 17-ago-2026), Sora (API se apaga el 24-sep-2026), FLUX.2 dev 32B (no comercial, no entra en 12 GB)		

Diagramas como código (default): un diagrama Mermaid son 150–600 tokens (≈ USD 0,0015 con Gemini Flash, 0,006 con Sonnet 5): texto siempre legible, editable, temas claro/oscuro, y acceso programático a nodos para hotspots y "completá el diagrama". Fiabilidad alta en flowchart, sequence, class, mindmap, timeline, ER (modelos de 7B ya logran ~91 % de sintaxis válida en sequence diagrams); baja en xychart/block/architecture. Patrón: pedir Mermaid con structured output (`mermaid + alt_text + nodes[]`) → `mermaid.parse()` → reenviar el error al LLM (máx. 2) → fallback a Graphviz DOT (`@viz-js/viz`) → última instancia imagen. Complementos: Markmap (mapas mentales desde Markdown), KaTeX (fórmulas; nunca imágenes), Vega-Lite/Chart.js (datos), Excalidraw (MIT; vía "element skeleton"; tldraw descartado por watermark obligatoria), Kroki solo si hay backend, Manim (fiabilidad baja-media; feature experimental).

Video (1 minuto de explainer, con 2-3 reintentos por clip)	USD	Notas
Gemini Omni 1.1 Flash 720p (0,10/s; borrador 360p 0,03/s)	≈ 15	Escenas extensibles hasta 40 s
Veo 3.1 Fast con audio (0,15/s) · Veo 3.1 estándar (0,40/s)	≈ 22 · ≈ 60	8 s por clip + extensiones
Kling 3.0 con audio (0,126/s) · Hailuo 2.3 (≈ 0,05/s) · Runway Gen-4 Turbo (0,05/s)	≈ 19 · ≈ 8 · ≈ 7,5	Sin audio salvo Kling
HeyGen avatar (USD 3/min)	3–4	Busto parlante; no ilustra conceptos
Slides + TTS + Remotion (default)	0,05–0,20	LLM → guion por escenas → Mermaid/SVG + ilustraciones opcionales → TTS (Kokoro local o Azure) → Remotion compone MP4 con subtítulos; Remotion es gratis para individuos y equipos ≤ 3

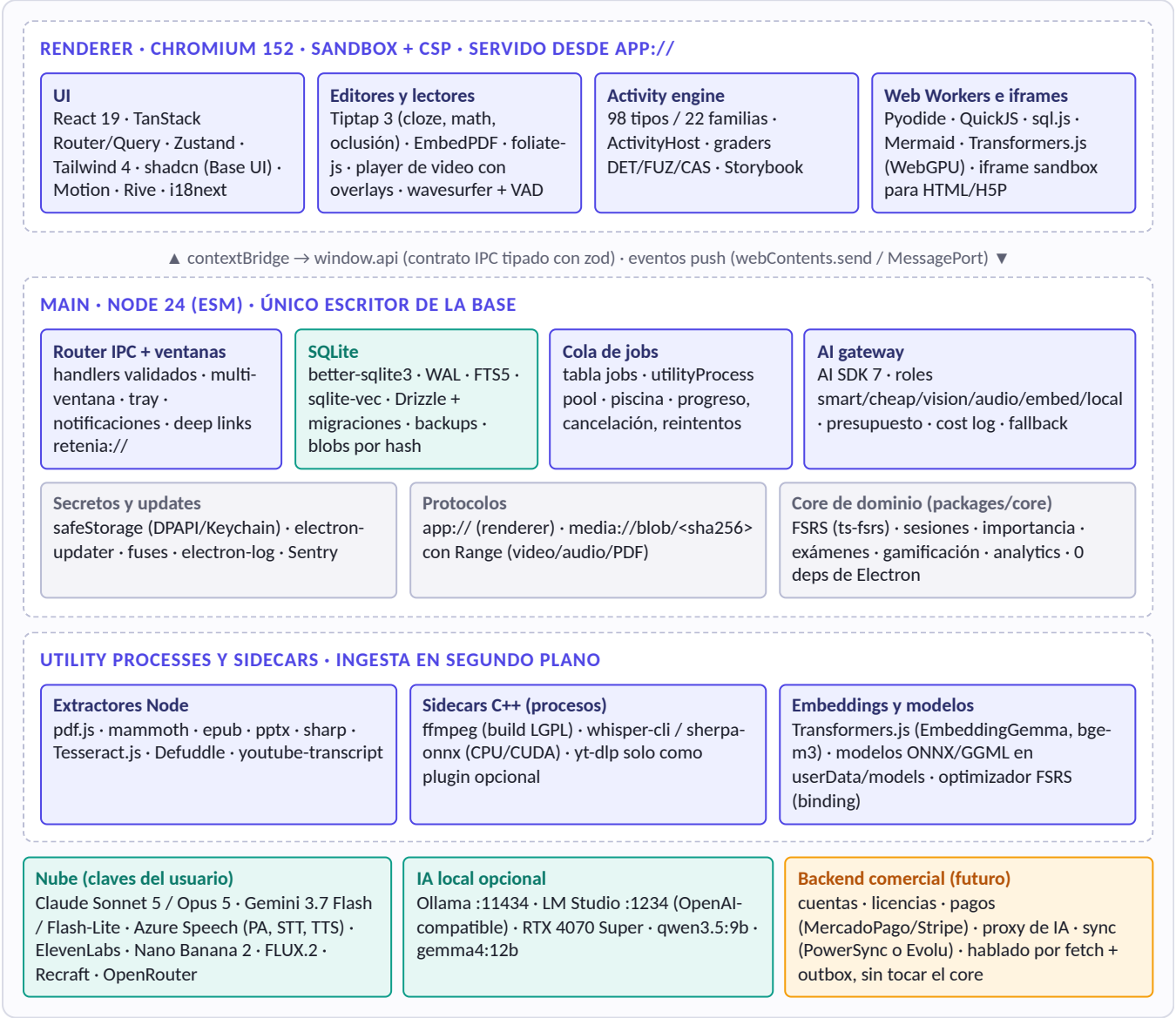
Un solo video generativo de 1 minuto consume del 30 % al 100 % del presupuesto mensual: en v1 el video es **opcional/premium con costo por clip visible**. Misceláneos: DeepL API Free (500k chars/mes, no verificado hoy) o LLM barato para traducción (Gemini Flash-Lite ≈ USD 0,13 por millón de caracteres); ElevenLabs SFX 0,12/min y Music 0,15/min para jingles (< USD 0,50/mes); Wolfram|Alpha LLM API gratis no comercial (SymPy vía Pyodide en v1); sandboxes de código: Pyodide (Python en WASM, gratis) + QuickJS-WASM; E2B (USD 100 de crédito) o Judge0/Piston autohospedados para lenguajes compilados en la versión con backend.

Presupuesto mensual consolidado

Rubro	Ruta recomendada	Premium	Notas
Texto (rutas, lecciones, corrección, chat)	USD 9–14	18–22	Sonnet 5 + Gemini 3.7 Flash + local · Opus 5 en rutas y explicaciones
Voz (pronunciación 7–15 h, STT corto, TTS 200k chars)	6–14	+ real-time 8–18	Azure PA (–5 h gratis) + Whisper local + Azure TTS dentro del free tier; ElevenLabs ya contratado aparte
Media (100 imágenes, 20 iconos, 30 diagramas, OCR, embeddings, web, YouTube)	5–6	20–30	Nano Banana 2 Lite + Recraft Vector + Mermaid; premium con Nano Banana Pro y Mistral OCR
Video generativo (2 min)	0,20 (slides + TTS)	25–45	Solo opt-in
Total	≈ USD 20–34	≈ 70–115	Dentro de 20–50 si el video queda opt-in; recalcular en Q4-2026 por la suba de Gemini en 2027

10 Arquitectura técnica

Stack verificado (versiones y fechas de npm al 1-sep-2026), estructura del monorepo, seguridad, datos, procesamiento en segundo plano, sandboxes de código, importación/exportación, firma y distribución, calidad y riesgos.



Área	Elección (versión)	Por qué	Alternativa
Firma Windows	Certum Cloud Code Signing Individual (~USD 139/año, sin token) o SSL.com IV/OV (65–75/año) + eSigner (20/mes)	Disponibles para un individuo en Argentina; eSigner tiene CLI y GitHub Action para CI	Azure Artifact Signing (USD 9,99/mes) no disponible para individuos fuera de EE. UU./Canadá ; EV ya no da reputación instantánea en SmartScreen
IPC	Contrato tipado propio + zod (~150 líneas)	Cero dependencias, Structured Clone nativo, validación en main	trpc-electron (fork para tRPC 11); electron-trpc está en tRPC 10 y sin releases hace 21 meses
UI	React 19.2 + TanStack Router 1.170 + TanStack Query 5.102 + Zustand 5	Rutas y search params tipados; Query envuelve <code>window.api</code> ; <code><Activity></code> para mantener montado el player	React Router 8, Jotai
Estilo y componentes	Tailwind 4.3 + shadcn/ui sobre Base UI 1.7 (default desde jul-2026) + Motion 13 + Rive 4.33 + dotLottie + canvas-confetti	Pulido "Linear/Duolingo"; mascota con state machines; tokens OKLCH; dark mode sincronizado con el SO	Radix, React Aria, Mantine 9, Ark UI 5
Editor de notas	Tiptap 3.31 (MIT) + extensiones propias (cloze, math, occlusion, callout) + <code>@tiptap/markdown + static-renderer</code>	Custom nodes, round-trip Markdown, Yjs-ready; nada Pro necesario	BlockNote 0.54 (MPL + XL de pago), Lexical, Plate
Código y matemática	CodeMirror 6 (default) + Monaco lazy (IDE); KaTeX 0.18 + MathLive 0.110	Peso vs IntelliSense; input matemático con teclado virtual	—
PDF / EPUB	EmbedPDF 2.15 (MIT, PDFium WASM, anotaciones) + pdfjs-dist 6 (texto en Node) + <code>@hyzyla/pdfium</code> ; foliate-js vendorizado	Highlights → cards; render de páginas para OCR	react-pdf; epub.js; Nutrient (comercial, descartado)
Video / audio	media-chrome 4 (o Vidstack 1.x si su actividad se confirma) + capa propia de overlays; wavesurfer.js 7 + RecordPlugin + Silero VAD	Markers, pause-and-ask, subtítulos VTT de Whisper, clip a card, waveform y grabación	video.js 8
Oclusión y diagramas	Konva 10 + react-konva; Mermaid 11.17 (en worker/iframe, <code>securityLevel: strict</code>); Excalidraw 0.18 (fuentes locales)	Editor declarativo; JSON serializable	Fabric 7; tldraw (watermark)
Datos	better-sqlite3 13 (N-API prebuilds) + Drizzle 0.45 → 1.0 + FTS5 + sqlite-vec 0.1.9 + WAL	Síncrono en main, extensiones, migraciones SQL versionadas	Kysely; LanceDB si > 200k chunks; <code>node:sqlite</code> no verificado en Electron 44
Cifrado	better-sqlite3-multiple-ciphers (opcional) + safeStorage (DPAPI / Keychain)	AES-256 en reposo; claves nunca en el renderer	keytar está abandonado desde 2022
Scheduler	ts-frs 5.4 + <code>@open-spaced-repetition/binding</code> (optimizador en worker)	FSRS-6 estándar	frs-browser (WASM)
Jobs	<code>utilityProcess</code> + piscina 5 + tabla <code>jobs</code> en SQLite	Progreso, cancelación y reintentos persistentes; huérfanos re-encolados al arrancar	BullMQ (requiere Redis: no)
Media tooling	ffmpeg build LGPL bundleado (no <code>ffmpeg-static</code> , que es GPL-3), yt-dlp binario con auto-update solo como plugin opcional, whisper-cli (CPU/CUDA) o sherpa-onnx	Licencias limpias; sin Python en v1	faster-whisper sidecar (v2)
Embeddings locales	<code>@huggingface/transformers</code> 4.2 (WebGPU/ONNX) u Ollama	e5/bge/EmbeddingGemma cuantizados; GPU sin CUDA vía WebGPU/DirectML	onnxruntime-node directo
IA	Vercel AI SDK 7.0.x + providers oficiales + openai-compatible + OpenRouter	Una API, structured output, tools, costos	SDKs nativos
Sandboxes	Pyodide 314 (Python 3.14 en WASM, MPL-2.0), QuickJS-WASM, sql.js, iframe <code>srcdoc</code> sandbox para HTML/CSS (Sandpack con bundler self-hosted si hace falta React/Vue)	Offline, sin licencias	WebContainers (licencia comercial), isolated-vm (maintenance mode), Judge0/Piston (server)
Calidad	TypeScript 7.0 (compilador Go; API programática estable en 7.1) + Biome 2.5 + Vitest 4 + Playwright 1.62 (<code>_electron</code>) + Storybook 10	Rápido y moderno; Biome no depende de la API de TS	ESLint 10 con <code>@typescript/typescript6</code>
Observabilidad	Sentry Electron 7 + electron-log 5 + PostHog opt-in	Minidumps, logs rotativos, eventos agregados sin contenido	—
i18n / repo	i18next 26 + react-i18next 17 (es-AR default, ICU plurals); pnpm 11 workspaces + Turborepo 2.10	Namespaces por feature; caché de builds	Lingui 6; Nx

Monorepo

```

retenia/
  apps/desktop/          # electron-vite: src/main, src/preload, src/renderer; electron-
builder.yml
  packages/
    core/                # dominio puro: entidades, FSRS (ts-fsrs), grading, gamificación,
analytics; puertos (repos, clock, ids); zod. 0 deps de Electron
    db/                  # Drizzle schema + migraciones SQL + repos better-sqlite3 (adapter
Node); futuro: expo-sqlite / sqlite-wasm / powersync
    ipc-contract/        # canales IPC tipados (zod in/out) y eventos push
    ui/                  # design system: tokens Tailwind, shadcn (Base UI), presets Motion,
iconos, wrappers Rive/Lottie
    activities/          # activity-schema, activity-graders, activity-ui, activity-ai,
activity-speech, activity-code (+ Storybook)
    editor/              # kit Tiptap (cloze/math/occlusion/callout), Markdown round-trip,
static renderer
    readers/             # pdf (EmbedPDF), epub (foliate-js), video (player + overlays), audio
(wavesurfer)
    ai/                  # AI SDK providers, routing por rol, prompts versionados, pricing,
budgets, claves (interfaz)
    ingest/              # extractores, chunking, embeddings, jobs (Node only), sidecar manager
(ffmpeg / whisper / yt-dlp)
    importers/           # anki, remnote, obsidian, csv; exporters markdown / apkg
    i18n/ config/        # recursos es-AR/en; tsconfig base, biome.json, preset tailwind
    tooling/             # scripts de release, firma, descarga de sidecars y modelos

```

Dependencias permitidas: `core` no importa nada interno; `db`, `ai`, `ingest`, `importers` importan `core`; `ui`, `activities`, `editor`, `readers` importan `core` (tipos) y `ui`; `apps/desktop` importa todo. Un cliente Expo o web futuro reutiliza `core`, `ai`, `activities` y `editor` con otro adapter de `db`.

Fundación Electron

- **Cadencia:** una major cada ~8 semanas (40.0 ene-16, 44.0 ago-25); se soportan las tres últimas. Fijar la major por release y probar el arranque en CI en Windows.
- **Auto-update:** provider `github` (release draft → publicar) con `latest.yml` / `beta.yml`; blockmaps para descargas diferenciales; `quitAndInstall`; provider `generic` para migrar a servidor propio sin tocar el cliente. macOS después: DMG/ZIP separados por arquitectura (los nativos complican `lipo`), notarización con `@electron/notarize` y API key de App Store Connect; Apple Developer USD 99/año.
- **Seguridad (checklist oficial):** `contextIsolation` y `sandbox` activos, `nodeIntegration` off, sin `@electron/remote`; preload mínimo con `contextBridge.exposeInMainWorld('api', ...)` (nunca `ipcRenderer` crudo); validar `event.senderFrame` en cada handler; CSP estricta (`default-src 'self'`; `script-src 'self' 'wasm-unsafe-eval'`; `connect-src` con allowlist de proveedores desde settings, `http://127.0.0.1:11434` y `:1234`); renderer servido desde `app://` (esquema privilegiado, no `file://`); `will-navigate` bloqueado y `setWindowOpenHandler` → `deny`; `setPermissionRequestHandler` que solo permite `media` (micrófono) al origen `app://`; fuses (`RunAsNode: false`, `EnableNodeOptionsEnvironmentVariable: false`, `EnableEmbeddedAsarIntegrityValidation`, `OnlyLoadAppFromAsar`); contenido no confiable (H5P, previews HTML, Sandpack) en `<iframe sandbox>` de origen distinto sin el preload.
- **IPC:** `packages/ipc-contract` define `{ 'cards.review': { input: z.object(...), output: z.object(...) } }`; main registra `ipcMain.handle(channel, (e, raw) => handler(schema.parse(raw)))`; el preload genera `api` desde el contrato; TanStack Query envuelve `window.api.*` con invalidación desde eventos de main (`api.events.on('cards.changed', ...)`); push main → renderer con `webContents.send` o `MessageChannelMain` para streams (transcripción, tokens del LLM).

- **Ventanas, deep links, media:** ventana principal + secundarias (player pop-out, modo examen) como clientes de la misma API; `app.setAsDefaultProtocolClient('retenia')` + single instance (`retenia://import?src=...`, `retenia://review`, futuro OAuth); `protocol.handle('media', ...)` con `net.fetch` sobre `file://` (soporta Range para seek) mapeando `media://blob/<sha256>` → `userData/blobs/ab/cd/...` con validación anti-traversal; FS solo en main/utility; drag & drop con `webUtils.getPathForFile`.
- **Rendimiento:** lazy loading agresivo (Monaco ~5 MB gz, Excalidraw, pdf.js, Pyodide 10+ MB, Mermaid, fuentes KaTeX) por ruta/actividad; workers con `onnxruntime` + modelo (+300–600 MB) se crean bajo demanda y se matan; toggle "desactivar aceleración GPU"; memoria de referencia: main 60–90 MB, renderer React 80–120 MB + GPU 90 MB; Monaco + pdf.js + video simultáneos pueden llegar a 500–800 MB.

Capa de datos

- **Operación:** `PRAGMA journal_mode=WAL; synchronous=NORMAL; foreign_keys=ON; busy_timeout=5000; cache_size=-64000; temp_store=MEMORY`; un único escritor (main o "db worker"); los workers devuelven resultados por mensaje. Backups: `db.backup()` online diario y al cerrar, rotación 7 en `userData/backups/`, "Exportar copia" (zip de DB + blobs), `integrity_check` semanal, advertencia si `userData` está en una carpeta sincronizada por OneDrive/Dropbox.
- **Layout:** `userData/retenia.db` (+ wal/shm), `blobs/<sha256[0:2]>/<sha256>.ext` (content-addressed: dedupe gratis, integridad, sync futuro trivial), `cache/` (regenerable), `models/` (ONNX/GGUF descargados con hash), `bin/` (sidecars actualizables), `backups/`, `logs/`. Windows: `path.join`, nombres cortos por hash (`MAX_PATH`), NFC en nombres importados, reemplazo atómico, cerrar la DB antes de actualizar.
- **Convenciones sync-ready:** ids UUIDv7 (texto, ordenables), `created_at` / `updated_at` (ms), `deleted_at` (soft delete), `device_id`, `version` por fila, sin AUTOINCREMENT, JSON en TEXT con `json_valid`, blobs fuera de la DB, tabla `outbox` vacía en v1.

Tabla	Campos clave
<code>sources</code> · <code>source_units</code> · <code>chunks</code> · <code>embeddings (vec0)</code> · <code>annotations</code>	<code>kind</code> , <code>title</code> , <code>origin_uri</code> , <code>blob_sha256</code> , <code>status</code> , <code>language</code> , <code>meta</code> · <code>page/slide/keyframe/segment</code> con <code>t_start/t_end</code> · <code>text</code> , <code>char_start/end</code> , <code>token_count</code> , <code>hash</code> (+ <code>chunks_fts</code>) · <code>embedding float[N]</code> , <code>model_id</code> , <code>partition</code> <code>source_id</code> · <code>highlight/note/region/clip</code> con <code>anchor (rects/CFI/t)</code>
<code>knowledge_items</code> · <code>cards</code> · <code>review_logs</code> · <code>scheduler_profiles</code> · <code>importance_levels</code>	Ver §5 (campos FSRS 1:1 con <code>ts-fsrs</code>)
<code>paths</code> · <code>sections</code> · <code>modules</code> · <code>lessons</code> · <code>activities</code> · <code>attempts</code> · <code>lesson_sessions</code>	Árbol de la ruta con <code>version</code> , <code>unlock_rule</code> , <code>xp_reward</code> ; <code>activities</code> con <code>type/family/config/grading</code> JSON; <code>attempts</code> con <code>score</code> , <code>answer</code> , <code>feedback</code> , <code>ai_eval_call_id</code> ; <code>sesiones</code> con <code>xp</code> y <code>accuracy</code>
<code>exams</code> · <code>exam_items</code> · <code>exam_attempts</code> · <code>item_bank</code>	Blueprint, formas A/B, <code>difficulty_logit</code> , <code>exposure</code> , <code>stats</code>
<code>jobs</code> · <code>ai_calls</code> · <code>settings</code> · <code>gamification (xp_events, streaks, achievements)</code> · <code>blobs</code> · <code>outbox</code>	<code>status/priority/payload/progress/attempts/run_after/locked_by</code> · <code>provider/model/purpose/tokens (input, output, cached, reasoning)/cost_usd/latency</code> · <code>key/value</code> JSON · <code>eventos append-only</code> → <code>agregados</code> · <code>sha256/mime/bytes</code> · (vacía en v1)

Sync futuro (menor arrepentimiento): ninguna librería en v1; `packages/core` con casos de uso puros (`reviewCard(cardId, rating, now)`) sobre interfaces `CardRepository`, `Clock`, `IdGenerator`. Candidatos cuando llegue el momento: **PowerSync** (SQLite en cliente sobre better-sqlite3, backend Postgres, SDK Node en beta, Apache-2.0; tu backend sigue siendo tuyo) o **Evolu** (MIT, CRDT propio, E2E encryption, relay self-hosted; pero exige su esquema desde el día 1). Descartados: Zero (sin escrituras offline), cr-sqlite (dormido), Triplit (AGPL), Jazz (lock-in de modelo), Electric (duplicaría motor con PGlite). Yjs/Automerger solo para colaboración en el editor.

Procesamiento en segundo plano y sidecars

`utilityProcess.fork()` para jobs pesados (parsing, embeddings, spawn de binarios), `worker_threads` + piscina para CPU puro, `child_process.spawn` con `windowsHide` y `kill-tree`. Cola persistida: `UPDATE jobs SET status='running', locked_by=? WHERE id = (SELECT id FROM jobs WHERE status='queued' AND run_after<=? ORDER BY priority DESC, created_at LIMIT 1) RETURNING *`; progreso por mensajes → `webContents.send('jobs:progress')`; cancelación con `AbortController`; backoff 2" minutos; panel "Procesando" con barra por fuente. Binarios: `ffmpeg` LGPL en `resources/bin` con `asarUnpack` (invocado como proceso, con aviso y enlace a fuentes: no contamina la licencia); `whisper-cli` precompilado (CPU + CUDA, detectar GPU NVIDIA) con modelos GGML descargados bajo demanda (`nodejs-whisper` compila en postinstall con MinGW: inaceptable para usuarios finales); `yt-dlp` solo como plugin opcional con auto-update semanal y verificación SHA256. Keyframes: `-vf "select='gt(scene,0.3)'"` o `fps=1/10 + dHash`. Sidecar Python (PyInstaller) diferido a v2: +300 MB–2 GB, falsos positivos de antivirus, firma de cientos de DLLs.

Sandboxes de código

Caso	Solución	Notas
Python	Pyodide 314 (CPython 3.14 en WASM, MPL-2.0) en Web Worker con timeout	~10–15 MB core + paquetes on-demand (numpy, pandas, SymPy); pre-bundlear wheels para offline; mini-runner de asserts (pytest funciona pero lento)
HTML/CSS/JS (ruta "HTML & CSS")	<code><iframe sandbox srcdoc></code> + CodeMirror para HTML/CSS puro; Sandpack 2.20 para React/Vue con bundler self-hosted (por defecto carga desde codesandbox.io)	Grading: comparar DOM y estilos computados con reglas
JS/TS con tests	QuickJS-WASM (<code>quickjs-emsripten</code> , MIT) con límite de memoria e <code>interruptHandler</code>	isolated-vm en maintenance mode; WebContainers requiere licencia comercial
SQL	<code>sql.js</code> (SQLite WASM) con fixtures por ejercicio; comparar result-sets	—
C, Java, Go, Rust	Toolchain local detectado (<code>where gcc/java/go</code>) en cwd temporal con timeouts; marcado "avanzado"	Sin sandbox real en Windows; aceptable porque el código es del propio usuario. Judge0 (GPL) / Piston (MIT) / E2B en la versión con backend

Importación y exportación

- **Anki .apkg:** Legacy 1 (`collection.anki2` , schema 11, JSON en `col`), Legacy 2 (`collection.anki21` + dummy), Latest (`collection.anki21b` comprimido con `zstd`, schema 18 con tablas `notetypes/fields/templates/decks` , config protobuf, `media` protobuf `MediaEntries`). Estrategia: importar Legacy 2 primero (Anki ofrece "Support older Anki versions" al exportar), luego `anki21b` con `fzstd` + `protobufjs` (mensaje pequeño copiado de `import_export.proto`); leer `notes` (campos separados por `\x1f`), `cards`, `revlog` (→ `review_logs` con conversión SM-2 → FRSR o "reaprender"); soportar templates `{{Field}}`, `{{cloze:}}`, `{{hint:}}`, `{{type:}}`, condicionales, `[sound:]`, `[latex]`. Exportar .apkg propio (Legacy 2) ≈ 300 líneas (genanki-js es AGPL: no usar).
- **RemNote:** exporta "RemNote (complete)" JSON, OPML, Anki .apkg, HTML, Markdown, Text, PDF; importar el .apkg (fiable para cards) + OPML/Markdown para la jerarquía; el JSON completo requiere ingeniería inversa con un export real (no documentado).
- **Obsidian / Markdown:** `frontmatter` (`gray-matter`), wikilinks y embeds, callouts, tags; sintaxis del plugin Spaced Repetition: `Pregunta::Respuesta`, `:::` reversa, multi-line con `? / ??`, `cloze ==texto==` o `{{1::respuesta::pista}}`, decks por tag `#flashcards/...`, scheduling en comentario `<!--SR: !2025-01-01,3,250-->`. Export propio: una carpeta por ruta/módulo, un `.md` por ítem con frontmatter, `cloze {{c1:...}}`, media en `attachments/` por hash.
- **CSV/TSV, H5P, SCORM:** mapeo de columnas guiado con preview; `h5p-standalone` 3.8 (MIT) en `iframe sandbox` capturando xAPI; `scorm-again` 3.3 (MIT) para reproducir SCORM 1.2/2004/cmi5 en B2B; empaquetar lecciones propias como SCORM/xAPI queda para después.

Calidad, CI y release

TypeScript strict con `tsc 7` para typecheck y Biome para lint/format; Vitest 4 + RTL con 100 % de la lógica de `core` (FSRS, grading, sesiones) cubierta; Storybook 10 con un story por tipo de actividad × estado (idle/correcto/incorrecto) y regresión visual opcional; Playwright 1.62 con `_electron.launch` sobre la app empaquetada (instalar NSIS silencioso → lanzar → repasar → cerrar) en un runner Windows; GitHub Actions con `matrix windows-latest` (build NSIS + firma con secrets `CSC_LINK / CSC_KEY_PASSWORD` o credenciales eSigner) y `macos-latest` después; releases como draft; canales beta/latest con toggle "Recibir betas"; Sentry (main, renderer, utility; source maps; incompatible con bytecode); electron-log con "Exportar diagnóstico"; PostHog opt-in con id anónimo y sin contenido; feature flags en `settings` ; auditoría de licencias con `license-checker` (allowlist MIT/Apache/BSD/ISC/MPL) en CI.

Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación
Módulos nativos (better-sqlite3, sharp, onnxruntime, sqlite-vec) al subir Electron	Build falla / crash	Preferir N-API; <code>@electron/rebuild</code> solo para lo no-N-API; fijar mayor; <code>asarUnpack</code> ; test de arranque en CI
Firma / SmartScreen	Usuarios asustados en la primera semana	Certificado desde el primer beta; mismo cert por años; firmar también <code>ffmpeg/whisper-cli</code> ; página de descarga con instrucciones
Antivirus (NSIS sin firma, DLLs CUDA, binarios)	Instalación bloqueada	Firma; sidecars descargados bajo demanda; enviar muestras a Microsoft; sin sidecar Python en v1
Tamaño (Monaco, Pyodide, modelos 100 MB–1,5 GB, CUDA 500 MB)	Instalador de 300 MB+	Lazy loading; modelos y sidecars en <code>userData</code> con hash; "Paquete GPU" opcional
GPU (drivers, pantallas negras, WebGPU ausente)	Crash de render / inferencia lenta	Toggle de aceleración; feature detection con fallback WASM/CPU
Vidstack con estado incierto; EmbedPDF 3.0, Drizzle 1.0, electron-vite 6, sqlite-vec pre-v1 con breaking changes; AI SDK con major cada ~6 meses	Trabajo de migración	Wrappers finos; <code>save-exact</code> ; Renovate mensual agrupado; todo el uso de IA encapsulado en <code>packages/ai</code> ; codemods oficiales
TypeScript 7 sin API programática estable hasta 7.1	Lint roto	Biome; o <code>@typescript/typescript6</code> para typescript-eslint
Licencias contaminantes (ffmpeg-static GPL-3, genanki-js AGPL, tldraw watermark, BlockNote XL, WebContainers comercial, h5p-server GPL, PyMuPDF AGPL)	Producto no comercializable	Evitados en la tabla del stack; auditoría automática en CI
Corrupción de SQLite (apagones, OneDrive sobre %APPDATA%)	Pérdida de datos	WAL + backups + <code>integrity_check</code> + advertencia de carpeta sincronizada
Seguridad de claves (DPAPI no protege de malware del mismo usuario)	Robo de claves	Documentarlo; presupuesto de gasto; opción de passphrase maestra (Argon2) que cifra claves + DB
Sync tardío	Rewrite	UUIDv7, soft deletes, outbox y repos por puerto desde el día 1; spike temprano de PowerSync
Memoria en sesiones largas (video + PDF + editor + workers)	OOM	Reciclar utility processes; <code><Activity></code> para desmontar vistas pesadas; monitoreo opt-in

Esfuerzo estimado

Área	Semanas-persona	Área	Semanas-persona
Esqueleto Electron (vite, builder, NSIS, updater, firma, CSP, IPC, protocolos, logging, Sentry, CI)	2–3	Monorepo + design system (tokens, shadcn, dark mode, Motion, iconos, Storybook)	2–3
Data layer (schema, migraciones, repos, FTS5, sqlite-vec, backups, jobs)	2–3	Core de dominio (FSRS, colas, gamificación, analytics, tests)	2–3
Editor Tiptap (cloze, math, occlusion, callouts, Markdown round-trip)	3–4	Paths, lecciones y actividades (21 tipos + grading + Storybook)	5–7
Lector PDF con highlights → cards; EPUB	2–3	Video interactivo (player, markers, pause-and-ask, subtítulos, clips)	3–4
Audio y pronunciación (grabación, waveform, VAD, Azure, accent lab)	2–3	Ingesta (extractores, chunking, embeddings, ffmpeg/whisper, keyframes)	4–6

Área	Semanas-persona	Área	Semanas-persona
RAG + IA (providers, routing, structured output, costos, presupuestos, claves) + "Generar con IA"	5-7	Sandboxes de código (Pyodide, QuickJS, HTML/CSS preview, sql.js, Parsons)	3-4
Importadores/exportadores	2-3	Exámenes, notificaciones, estadísticas, gamificación, onboarding, i18n	4-5
E2E Playwright + hardening + beta	2-3	macOS (firma, notarización, arm64/x64, QA)	1-2
Total v1 (Windows), 1 desarrollador senior con IA de apoyo			≈ 40-55
MVP usable (repaso + editor + PDF + rutas generadas básicas)			≈ 12-16

11 Experiencia de usuario y gamificación

Cómo se siente Retenia: mapa de pantallas, el camino estilo Duolingo, el reproductor de lecciones, la sesión de repaso, el centro de exámenes y una gamificación con evidencia y modo sobrio. Referencias de pulido: Duolingo (camino, feedback, mascota), Brilliant (interactividad en la explicación), Linear (densidad, command palette, atajos), Notion (editor de bloques), Anki/RemNote/Mochi (repaso), Speak/ELSA (voz), Khan Academy (mastery), Readwise Reader (PDF/EPUB con highlights → cards).

Principios de UX

- 1. **Una acción principal por pantalla.** El home responde "¿qué hago hoy?" con un botón: repasar (35 repasos, ~12 min), seguir la ruta (lección 7 de 40) o prepararme para el examen del 14.
- 2. **Feedback inmediato y explicable.** Cada respuesta muestra correcto/incorrecto, la respuesta modelo, "¿por qué?" (Explain my answer) y la cita a la fuente. Nunca se castiga: los errores son datos.
- 3. **El scheduler es visible.** Cada tarjeta muestra su fuerza (R), estabilidad (S) y por qué apareció hoy; los 4 botones muestran el próximo intervalo; "Urgente" muestra el costo en repasos antes de aplicarse.
- 4. **Teclado primero.** Command palette (Ctrl+K), atajos reconfigurables (1–4 para calificar, Espacio para revelar, J/K/L en video), toda interacción drag-and-drop con alternativa por teclado.
- 5. **Densidad configurable.** Modo compacto (Linear) para adultos; modo amplio y animado para escolares.
- 6. **Offline sin sorpresas.** Todo lo local funciona sin red; lo que requiere IA muestra costo estimado y estado de conexión; las operaciones largas viven en un panel de progreso con cancelar/reanudar.
- 7. **Accesibilidad de serie.** WCAG 2.2 (foco visible, target ≥ 24 px, alternativas al arrastre, contraste AA, reduced motion), lectores de pantalla (árbol de accesibilidad de Electron), fuentes opcionales (Atkinson Hyperlegible, OpenDyslexic — sin prometer milagros: la evidencia sobre fuentes "para dislexia" es mixta), TTS de cualquier texto.

Mapa de pantallas

Pantalla	Contenido y comportamiento	Referencia
Home / Hoy	Tarjeta "Hoy": vencidos por nivel, nuevos, refuerzo de ruta, examen más cercano con readiness; racha y meta con anillo de progreso; heatmap de 12 semanas; rutas activas con progreso; "Continuar donde estaba" (lección, PDF, video); insights de IA ("tus temas débiles esta semana").	RemNote Flashcard Home, Duolingo home, Anki Today
Camino (ruta)	Scroll vertical serpenteante de nodos: lección (círculo), refuerzo (escudo), checkpoint (trofeo), remediación (desvío punteado), examen (bandera); completados en dorado y reabribles; bloqueo suave ("se apoya en 4 ítems al 60 %"); mapa de conceptos alternativo (V2).	Duolingo path, Kinnu pathways
Reproductor de lección	Dos mitades: teoría (bloques: hook, activación, explicación con citas clicables que abren la fuente en la página/timestamp exacto, worked example, diagrama, misconceptions, resumen, glosario; "explicámelo de otra forma / más simple / con analogía") y práctica (barra de progreso de actividades, feedback por actividad, XP al cierre, resumen con lo que entra a memoria).	Brilliant, Learn Your Way, Duolingo lesson
Sesión de repaso	Una actividad por pantalla, sin distracciones; 4 botones con intervalos; Skip; "Explicame"; "Editar tarjeta" inline; "Marcar como leech / bajar importancia"; final drill al cierre; resumen: repasadas, precisión, tiempo, racha, "pospuse 40 de mantenimiento".	Anki, RemNote queue, Mochi
Biblioteca de fuentes (notebook)	Colecciones por tema/examen; tarjetas de fuente con estado (pendiente / procesando / lista / error), páginas/duración, idioma, "índice mejorado" (contextualización) con costo; botón Generar con IA ; lector PDF/EPUB con highlights → ítems; player de video con transcript clicable, markers y "clip a card"; chat del notebook con citas.	Gemini Notebook, Zotero, Readwise Reader
Generar con IA	Ver \$7: plantilla → progreso por etapas → preview editable → diagnóstico → expansión progresiva → resumen. Estimación de costo en vivo y contador de gasto.	RemNote Guided Learn, Thinkific wizard

Pantalla	Contenido y comportamiento	Referencia
Centro de exámenes	Lista con fecha, readiness (%), tarjetas pendientes, plan semanal; asistente de creación (material, fecha, blueprint, días de estudio, ensure mastery); simulacro en modo "a ciegas" (temporizador, sin pistas, UI bloqueada) y revisión posterior con item analysis; informe post-examen por tema y tipo de error.	RemNote Exam Scheduler V2, Knowt practice tests
Idiomas / Accent Lab	Escuchar referencia (voz del acento elegido) → grabar (waveform + VAD) → score global + palabras coloreadas → tap = fonema + consejo + "escuchar solo esta palabra" → curva de entonación superpuesta → reintentar; progreso por sonido; minimal pairs; roleplay por texto (voz en fase 2).	ELSA, BoldVoice, Speak
Notas y conocimiento	Editor de bloques (Tiptap) con >> , {{cloze}} , LaTeX, código, imágenes con oclusión, referencias [[]] y backlinks; "crear tarjeta desde cualquier bloque"; daily notes (V1).	RemNote, Obsidian, Logseq
Estadísticas	Panel diario; true retention; retención media y memorizado; forecast 30/90 días; distribución S/D; heatmap; answer buttons; rendimiento por hora; mastery por tema/ruta; readiness por examen; calidad del modelo (última optimización).	Anki stats, SuperMemo Analysis
Ajustes	Proveedores de IA (claves cifradas, roles smart/cheap/vision/audio/embed/local, presupuesto mensual con alertas, panel de gasto por llamada), voz (locale, acento, prosodia), scheduler (DR por nivel, pasos, easy days, optimizar), gamificación (arcade / sobrio), notificaciones, tema y tipografía, atajos, backups, importar/exportar, telemetría opt-in, "Paquete GPU".	—

El camino, en detalle

ANATOMÍA DE UN MÓDULO EN EL CAMINO

L1

 →

L2

 →

L3

 →

Refuerzo

 →

L4

 →

L4.r1 (desvío opcional)

 →

L5

 →

Refuerzo

 →

Checkpoint acumulativo

 → ...
→

Simulacro (forma A)

 →

Examen final (forma B)

Cada nodo de lección muestra minutos estimados, conceptos y estado (bloqueada suave / disponible / completada / completada por diagnóstico). El refuerzo compone vencidos de la ruta + ítems con $R < 0,8 + 2-3$ ejercicios de aplicación intercalados + mini-quiz; su gate es suave ($\geq 80\%$ sugiere seguir; menos, ofrece remediación sin bloquear). El checkpoint cada 3–4 módulos es un refuerzo acumulativo con interleaving. Completar nodos da XP; "Legendary" (sin pistas ni banco de palabras) es una variante de política, no un nodo nuevo.

Gamificación con evidencia

Mecánica	Diseño en Retenia	Evidencia / lección
Racha	Meta diaria (calculada) y meta de racha separadas (≈ 10 tarjetas); "Forgot" no cuenta; freezes ganados (1 cada 6 días, banco de 2) y "vacaciones" programadas; calendario y hitos.	Duolingo: los usuarios con racha ≥ 7 pasaron de $\sim 17\%$ a $> 50\%$ del DAU; la notificación "streak saver" fue el mayor salto; RemNote separa metas para evitar rachas frágiles.
XP y niveles	XP por lección, repaso, refuerzo y simulacro; boosts temporales por logros; mastery por tema (Attempted → Familiar → Proficient → Mastered) siempre al lado del XP.	Preferir mastery real a niveles vacíos (Khan); métricas de sustancia junto a las de vanidad.
Misiones	Daily quests (3/día: "repasá 20", "completá 1 lección", "acertá 5 seguidas"), semanales, y cooperativas cuando haya amigos (V2).	Cooperativas > competitivas para retención social (Duolingo Friend Quests).
Logros	Ligados a hitos de retención real (100 ítems "Retaining", 30 días de true retention $\geq 90\%$, examen aprobado) y no solo a actividad.	Badges por actividad tienen efecto pequeño si no están ligados a progreso real.
Mascota	Personaje con state machine (Rive): reacciona a aciertos, errores, rachas y cierres; tono configurable; celebraciones de fin de sesión ("Done for today") con confetti moderado.	Personalidad consistente = memorabilidad (Duo, Lily); evitar sobreuso.
Ligas y leaderboards	Opcionales, por nivel de actividad, cuando exista backend (V2); leaderboard de amigos.	Ligas: +17 % de tiempo de aprendizaje; desmotivan a quien está abajo → emparejar por actividad.
Vidas / energía	No hay corazones. Energía solo como límite suave opt-in (Later).	Los corazones fueron criticados por frenar el aprendizaje y empujar a pagar; Duolingo los reemplazó por Energy en 2025.
Notificaciones	Máximo 2 por día, hora preferida, "streak saver", digest; copys con personalidad pero nunca culpabilizadores; easy days.	"Proteger el canal": timing y copy optimizados valen más que volumen.

Mecánica	Diseño en Retenia	Evidencia / lección
Timed challenges	Match Madness (tap_pairs_timed), sprint aritmético, lightning round: recompensa y variedad, sin alimentar el scheduler.	Un contador de "movimientos" estilo Gardenscapes no movió retención: mecánicas sin relación con el aprendizaje no sirven.
Modo sobrio	Toggle que deja rachas, metas, heatmap y mastery; oculta XP, mascota, misiones y ligas; densidad compacta.	Meta-análisis (Sailer & Homner 2020): efectos pequeños-medios con efecto novedad; críticas de "electronic whip" y overjustification. Gamificación por perfiles.

Detalles de plataforma esperados en 2026

- Command palette con búsqueda difusa; hoja de atajos; hotkey global de captura rápida a un inbox (V1); web clipper (V1).
- Tray con pendientes y badge en el icono con las tarjetas de hoy; inicio con el sistema opcional.
- Multi-ventana, split view (PDF a la izquierda, notas a la derecha con `react-resizable-panels`), pestañas; modo enfoque a pantalla completa; Pomodoro opcional.
- Temas claro/oscuro/sistema/alto contraste; tamaño, interlineado y ancho de columna ajustables; reduced motion.
- Backups rotativos, papelera restaurable, historial de versiones (V1); exportación completa (JSON + Markdown + .apkg + CSV).
- Localización es-AR primero, en, pt-BR; RTL preparado; formato de fechas local.
- Onboarding en 3 pasos: elegir modo (arcade / sobrio), cargar claves (o "probar sin IA"), importar (Anki / RemNote / Markdown) o crear la primera ruta.
- MCP server / CLI local para que Claude, ChatGPT o Cursor lean y creen contenido en tu base (V2).

12 Catálogo maestro de funciones

125 funciones candidatas agrupadas por módulo, con etiqueta de fase (MVP necesario para validar · V1 primer año · V2 consolidación · Later visión) y de dónde se toma prestada la idea. Las fases de prompts.md implementan MVP y V1; V2 y Later quedan diseñadas en el esquema.

1 · Fuentes e ingesta

1	Importar PDF, DOCX, PPTX, Markdown, TXT, EPUB con estructura (títulos, capítulos)	MVP	RemNote, Gemini Notebook, Mindgrasp
2	Pegar texto/URL y web clipper de navegador con lectura limpia	MVP / V1	RemNote, Recall, Readwise
3	YouTube / video local con transcripción y anotación por tiempo	V1	RemNote YouTube Annotator, Knowt, LingQ
4	Grabación de clases (Lecture Recorder) con transcripción local y enlace a diapositivas	V1	RemNote, Knowt, Turbo
5	OCR de imágenes, capturas y apuntes manuscritos	V1	RemNote Image-to-Text, Quizlet Magic Notes
6	Importar highlights de Kindle/Readwise/Zotero y bibliografías	V2	Readwise, Heptabase, Zotero
7	Importar Anki (.apkg), Quizlet, CSV/Excel, Markdown/Obsidian/Notion	MVP (Anki/CSV) · V1 resto	Anki, Traverse, Knowt
8	Biblioteca de fuentes con metadatos, tags, carpetas y colecciones por examen	MVP	Zotero, RemNote
9	Lector integrado de PDF/web con highlights → notas/cards enlazadas a la página	V1	RemNote Reader, Zotero
10	Incremental reading: extractos, cola de lectura priorizada y read-points	V2	SuperMemo
11	Detección automática de preguntas tipo examen dentro de la fuente	V1	RemNote
12	Quick-capture global (hotkey del sistema) a un inbox	V1	Obsidian/RemNote clippers

2 · Generación de rutas

13	Ruta tipo Duolingo: Secciones → Módulos → Lecciones generadas desde las fuentes con objetivos y prerequisites	MVP	Duolingo, RemNote Guided Learn, Kinnu
14	Baseline / placement por módulo que salta contenido conocido	MVP -lite / V1	RemNote, Lingvist, Duolingo, Sana
15	Personalización por objetivo, fecha de examen, tiempo diario y nivel	MVP	RemNote Exam Scheduler, StudyFetch, Speak
16	"Customize topics": incluir/excluir/saltar secciones; regenerar; generar próximas secciones progresivamente	V1	RemNote
17	Mapa visual de la ruta (grafo de conceptos y dependencias)	V2	Traverse, Heptabase, Kinnu
18	Guidebook por módulo (resumen de teoría + vocabulario clave)	MVP	Duolingo Guidebook, RemNote Study Guide
19	Rutas prehechas / comunitarias y marketplace de rutas verificadas	Later	Brainscape, SuperMemo.com, Anki shared decks
20	Multi-ruta simultánea con "daily mix" que prioriza por examen más cercano	V1	Duolingo, RemNote

3 · Lecciones y teoría

21	Página de teoría por lección con resumen, imágenes de la fuente, términos clave clicables	MVP	RemNote Learn PDF
22	"Explicámelo de otra forma / más simple / con ejemplo / con analogía" en un clic	MVP	Ghostreader, Explain My Answer

23	Citas a la fuente (página/timestamp) en cada afirmación	MVP	Gemini Notebook, RemNote AI Tutor
24	Study Guide consolidado (conceptos, hechos, procesos, comparaciones, errores comunes, trampas de examen)	V1	RemNote
25	Audio overview / podcast de la lección y TTS de la teoría	V1	Gemini Notebook, Knowt Kai, StudyFetch
26	Explainer video / infografía / mind map generados	V2 / Later	StudyFetch, Gemini Notebook
27	Modo socrático: la IA pregunta antes de explicar; knowledge checks dentro de la teoría	V1	ChatGPT Study Mode, Khanmigo, Brilliant Koji
28	Notas del usuario al margen de la lección, enlazables a la base de conocimiento	MVP	Coursera notes, RemNote

4 · Práctica y actividades

29	Catálogo de ejercicios (98 tipos; 21 en MVP): MCQ con explicación, V/F, cloze, emparejar, ordenar, clasificar, respuesta corta con AI grading, dictado, hablar, etiquetar diagrama, tabla, línea de tiempo, code-fill...	MVP / V1	Duolingo, Quizlet, Knowt, RemNote, Storyline, H5P
30	Ejercicios "a partir de tu error": práctica remedial dirigida	V1	Speak, Duolingo Mistakes, Brilliant
31	Modo Study vs modo Test por actividad (feedback inmediato vs diferido)	MVP	RemNote quizzes
32	Dificultad adaptativa dentro de la sesión (más difícil al acertar, scaffolding al fallar)	V1	Brilliant, Quizlet Learn
33	Scaffolding MCQ → abierta: primera exposición como MCQ, luego producción	V1	RemNote MC Learn Mode
34	Juegos rápidos con tiempo (Match, Lightning Round) y multijugador local/online	V2	Quizlet Match/Live, Duolingo, Gizmo
35	Práctica manos libres / auto-advance (cards y quizzes leídos por voz)	V2	Anki auto advance, SuperMemo.com
36	Mini-apps / plugins de actividad (extensible por comunidad)	Later	StudyFetch, RemNote plugins

5 · Memoria y programación

37	FSRS-6 por defecto con desired retention, optimize, health check y simulador de carga; SM-2 solo como compatibilidad	MVP básico · V1 optimize/simulador	Anki, RemNote
38	Todo ítem de práctica (card o ejercicio) tiene estado D/S/R visible y explicable	MVP	Anki FSRS, FSRS Helper
39	Niveles de importancia que modulan retención objetivo, orden y auto-postpone	MVP	SuperMemo priority queue, RemNote priorities
40	Tipos de card: basic, reverse, bidireccional, cloze (pistas, anidado, multi-card), multi-line list/set, image occlusion, type-in, MCQ, audio cloze, pronunciación	MVP 6 · V1 resto	Anki, RemNote
41	Card clusters / hermanas con dispersión para evitar interferencia	V1	RemNote, FSRS Helper
42	Cola de nuevas ("Cards to Learn") separada; el usuario decide cuándo introducir	MVP	RemNote, Anki
43	Postpone / Advance / Flatten / Easy Days / vacaciones / Mercy	V1	FSRS Helper, SuperMemo
44	Final drill de lo fallado en la sesión	MVP	SuperMemo
45	Subset review, search & review, filtered decks y custom study (olvidadas, review ahead, por tag/estado)	V1	SuperMemo, Anki
46	Leech detection con sugerencia de reescritura por IA	V1	Anki, RemNote
47	Hints, extra detail, insights de fondo, explicación IA de la respuesta	V1	RemNote
48	Neural / semantic review: repasar vecinos conceptuales del ítem fallado	V2	SuperMemo, Recall
49	Reset / edición del historial, "Edit Later", cambio de dirección masivo, card table	V1	RemNote, Anki browser
50	Daily goal + streak goal separados; "forgot no cuenta"	MVP	RemNote
51	Modo "Currently studying": spread over week / all due / fit daily target	V1	RemNote

52	Recomendación de horario según rendimiento por hora y, opcional, registro de sueño	V2 / Later	Anki hourly stats, SuperMemo Sleep Chart
----	--	------------	--

6 · Exámenes

53	Exámenes programados con fecha: asistente (material, fecha, cards nuevas por semana, orden, periodo final, meta diaria, días de estudio, ensure mastery)	MVP	RemNote Exam Scheduler V2
54	Periodos Learning / Catch-up / Final review con avisos de retraso y recálculo	V1	RemNote
55	Simulacros de examen por IA con formato configurable, tiempo, cobertura por temas, "solo lo incompleto"	MVP	RemNote, Knowt, Memrise
56	Corrección automática (MCQ) + AI grading con rúbrica para abiertas y ensayos	V1	RemNote, StudyFetch Essay Grader, Knowt FRQ
57	Informe post-examen: por tema, por tipo de error, predicción de nota, "exam traps"	V1	RemNote, ELSA, Knowt
58	Plantillas de exámenes reales por país (ICFES, ENEM, EXANI, PAES, oposiciones)	V2	Knowt AP/IB hubs, ELSA IELTS
59	Modo "examen a ciegas" (sin pistas, temporizador, UI bloqueada) y revisión posterior	V1	RemNote Test Mode, Chess Game Review

7 · Idiomas y habla

60	Cards de vocabulario en contexto con frase, audio TTS, imagen y traducción automática	V1	Migaku, Mochi, Lingvist
61	Cloze de frases reales por frecuencia	V1	Clozemaster
62	Pronunciación con puntuación por fonema/estrés y feedback de nativo	V1 (Azure)	ELSA, BoldVoice, Speak
63	Roleplay / conversación IA por voz con escenarios, tono de feedback (soft/balanced/strict) y transcript	V2	Duolingo Roleplay/Video Call, Praktika, SuperMemo Live
64	Lectura asistida click-to-translate y "known words" / comprehension score	V2	Readlang, LingQ, Migaku
65	Puntaje tipo Score alineado a CEFR	Later	Duolingo Score, ELSA

8 · Notas y base de conocimiento

66	Editor outliner/Markdown con bloques, referencias, backlinks, tags, alias, portales	MVP básico · V1 portales/alias	RemNote, Obsidian, Logseq
67	Crear card desde cualquier bloque (>> , {{ }}) sin salir del editor	MVP	RemNote, Obsidian SR
68	Daily notes / journal con calendario	V1	RemNote, Heptabase
69	Tablas con columnas tipadas, AI autofill y cards por celda; import/export CSV	V2	RemNote Tables V2, Obsidian Bases
70	Search portals / consultas guardadas (query builder visual)	V2	RemNote, Tana
71	Canvas / whiteboard y manuscrito con conversión a texto/cards	V2	RemNote Canvas, Heptabase, Excalidraw
72	Templates y framework concepto/descriptor para generar cards estructuradas	V1	RemNote
73	Grafo de conocimiento auto-etiquetado con resurfacing de relacionados	V2	Recall, Obsidian Graph
74	LaTeX, código, media, embeds	MVP básico	Anki, RemNote

9 · Tutor IA / chat

75	Tutor conversacional con acceso a la base, citas clicables, "lee lo que hay en pantalla"	MVP	RemNote AI Tutor, Gemini Notebook
76	Acciones del tutor: crear cards, quiz, resumen, ruta, tabla, ejercicio; "convertir esta respuesta en cards"	MVP	RemNote, Knowt Kai

77	Modo socrático (no da la respuesta) y modo directo, conmutables	V1	ChatGPT Study Mode, Khanmigo
78	Memoria del alumno (errores, temas débiles, preferencias, tono) y tutor con personalidad	V1	Praktika, Speak, Duolingo Lily
79	Selección de modelo (rápido/equilibrado/frontier) y BYOK (Anthropic, Google, OpenAI, locales)	MVP 1 proveedor · V1 multi	RemNote, Recall
80	Tutor por voz manos libres (quiz oral)	V2	Knowt Kai, SuperMemo MemoChat
81	MCP server local para que agentes externos consulten y creen contenido	V2	RemNote, Readwise, Heptabase CLI

10 · Generación multimedia

82	TTS neuronal multi-voz para cards y teoría, con caché local	MVP voces del sistema · V1 neuronal	HyperTTS, Readwise, Matter
83	Imágenes generadas o buscadas para cards (mnemotecnias visuales)	V1	Mochi, Duolingo
84	Audio cloze y dictados generados	V1	Duolingo, Clozemaster
85	Podcast / audio overview por módulo y explainer video ligero	V1 / V2	Gemini Notebook, StudyFetch
86	Evaluación de pronunciación (STT + scoring)	V1	ELSA, Speak
87	Diagramas para image occlusion generados o limpiados por IA	V2	RemNote Advanced IO

11 · Gamificación

88	Racha con meta de racha separada, freezes ganados (1 cada 6 días, banco de 2) y vacaciones	MVP	RemNote, Duolingo
89	XP, niveles y mastery por tema (Attempted → Familiar → Proficient → Mastered)	MVP mastery · V1 XP	Khan, Duolingo
90	Daily/weekly quests y misiones cooperativas con amigos	V1 / V2	Duolingo
91	Ligas opcionales por nivel de actividad; leaderboard de amigos	V2	Duolingo, SuperMemo.com
92	Badges ligados a hitos de retención real	V1	Duolingo, Khan
93	Mascota / personajes con personalidad (tono configurable, modo sobrio)	V1	Duolingo, Praktika
94	Celebraciones de fin de sesión, "Done for today", resumen de logros	MVP	RemNote, Duolingo
95	Heatmap / calendario de actividad y previsión de carga	MVP	Anki, Review Heatmap
96	Recordatorios inteligentes (hora preferida, streak-saver, digest), limitados y editables	V1	Duolingo
97	Pomodoro / focus mode y sonidos ambiente opcionales	V1	Quizlet, Anki add-ons
98	Widgets: tray/badge en desktop; widget móvil futuro	V1	RemNote, Duolingo
99	Energía como límite suave opt-in (nunca castigo por error)	Later	Duolingo Energy

12 · Estadísticas

100	Panel diario: pendientes, hechas, tiempo, precisión, meta	MVP	Anki, RemNote
101	Future due, intervalos, dificultad, estabilidad, retrievability, true retention, answer buttons, hourly breakdown	V1	Anki
102	Mastery por tema/módulo/fuente y por examen (% listo, tiempo restante)	MVP	RemNote
103	Curvas de olvido y burden/overload; simulación de carga a futuro	V2	SuperMemo Analysis, Anki simulator
104	Informes exportables y comparativa semanal	V1	SuperMemo, RemNote
105	Insights IA ("tus temas débiles esta semana", "mejor hora de estudio")	V1	ELSA, Brilliant

13 · Importación / exportación

106	Export completo JSON + Markdown + CSV + .apkg (con scheduling) + PDF de guías	MVP	JSON/Markdown/.apkg	Anki, RemNote, Readwise
107	Import de .apkg con media y scheduling, Quizlet, CSV/Excel, Markdown	MVP / V1		Anki, Traverse, Knowt
108	Sync opcional E2E y "vault" portable en carpeta	V1		Obsidian
109	API local / CLI (estilo AnkiConnect)	V2		AnkiConnect, Heptabase CLI
110	Publicar / compartir rutas y mazos (enlace, marketplace)	V2 / Later		Mochi, Brainscape, Anki

14 · Ajustes y plataforma

111	Command palette, atajos reconfigurables, hoja de atajos	MVP		Obsidian, RemNote, SuperMemo
112	Multi-ventana, split view, tabs	V1		RemNote 1.28
113	Temas claro/oscuro/alto contraste, fuentes (OpenDyslexic, Atkinson), tamaño/interlineado, reduced motion	MVP		Obsidian, WCAG
114	Accesibilidad: teclado completo, foco visible, lectores de pantalla, target ≥ 24 px	MVP		WCAG 2.2, Electron
115	Localización es-LatAm, en, pt-BR; RTL preparado	MVP	es/en	RemNote, Mindgrasp
116	Backups automáticos rotativos, historial de versiones, papelera	MVP backups · V1	versiones	AnkiDroid, Obsidian
117	Privacidad: local-first, cifrado en reposo opcional, BYOK, sin telemetría sin consentimiento, sanitización de importaciones	MVP		Obsidian, Anki 25.09
118	Auto-update reanudable, instalador sin terminal, firma de código	MVP		Anki 26.05, RemNote
119	Perfiles / espacios (varios alumnos; modo niño)	V2		Khanmigo, Family plan
120	Plugins y temas de comunidad con API sandboxed	Later		RemNote, Obsidian

15 · Comercial (después)

121	Free generoso (todo local, IA con BYOK) + Pro (IA incluida, sync, TTS neuronal) + Pro Max (frontier, voz)	V1		RemNote tiers, Duolingo Super/Max
122	Créditos de IA con contador visible y "runs" por función	V1		RemNote
123	Licencias B2B: seats, SSO, panel docente/empresa con informes de retención y asignaciones	V2		Brainscape, Duolingo Schools, Khanmigo
124	Marketplace de rutas/mazos con reparto de ingresos a creadores	Later		Brainscape
125	Family plan y precios regionales LatAm (pago local)	V2		Duolingo Family

13 Plan de desarrollo y uso de prompts.md

Quince fases (F0 de configuración + F1–F14 de construcción), 76 sub-fases, ordenadas para que cada fase deje algo usable. Cómo se mapean "Haiku 4.5 / Sonnet 5 / Opus 5 / Fable 5.1" y "Bajo / Medio / Alto / Extra / Max / UltraCode" a los controles reales de Claude Code, verificados en la documentación oficial.

Las fases de un vistazo

Fase	Objetivo	Sub-fases	Qué queda usable	Modelo dominante
F0 Setup	Repo, CLAUDE.md, settings y hooks, subagentes, skills, specs/ADRs a partir de este PDF, CI base	0.1–0.4	Un repo donde Claude Code trabaja con verificación automática	Sonnet 5 · Haiku 4.5
F1 Esqueleto	Monorepo + electron-vite, seguridad de procesos, IPC tipado, protocolos app:// y media://, builder/updater/logging, tests	1.1–1.5	App vacía que se instala, actualiza y loguea	Sonnet 5 · Opus 5
F2 Design system y shell	Tokens, Tailwind 4, shadcn/Base UI, dark mode, layout, router, command palette, i18n, Motion, mascota Rive, accesibilidad	2.1–2.5	Cascarón navegable con la identidad visual	Sonnet 5
F3 Datos	Schema Drizzle completo, migraciones, repos y puertos, FTS5 + sqlite-vec, cola de jobs, blobs, backups, claves	3.1–3.5	Base local robusta y sync-ready	Opus 5 · Fable 5.1 (schema)
F4 Memoria	FSRS-6, importancia, sesión diaria, UI de repaso, ejercicio → rating, optimizador y herramientas de sobrecarga	4.1–4.6	Ya podés repasar flashcards de verdad	Fable 5.1 · Opus 5
F5 Activity engine	Esquema por familias, host, 21 tipos MVP, AI grading, generador de sesión	5.1–5.6	Ejercicios interactivos programados por el scheduler	Opus 5 · Sonnet 5 (UltraCode en renderers)
F6 Ingesta y RAG	Parsers, chunking, embeddings, retrieval híbrido, audio/video, web/YouTube, lector PDF/EPUB	6.1–6.6	Biblioteca de fuentes con búsqueda y highlights → cards	Sonnet 5 · Opus 5
F7 Capa de IA	Providers, roles, costos, structured output, batch/cache, local, ajustes y evals	7.1–7.5	Cualquier feature puede pedir IA con costo visible	Opus 5
F8 Generar con IA	Extracción, síntesis, secuenciación, preview, expansión, QA, item bank, diagnóstico, remediación	8.1–8.6	El botón "Generar con IA" funciona	Fable 5.1 · Opus 5
F9 Rutas en la UI	Camino, reproductor de lección, refuerzos, tutor con citas, editor de notas	9.1–9.5	Experiencia Duolingo completa	Sonnet 5 · Opus 5
F10 Exámenes	Scheduler con fecha, simulacros, calificación, analytics, plan y notificaciones	10.1–10.4	Estudiar hacia una fecha	Opus 5
F11 Idiomas y voz	Audio, Azure PA, TTS, Accent Lab, STT y dictado	11.1–11.5	Pronunciación en 6 idiomas	Opus 5 · Sonnet 5
F12 Media y código	Imágenes, diagramas y oclusión, video ligero, sandboxes de código	12.1–12.4	Lecciones con imágenes, diagramas y código ejecutable	Sonnet 5
F13 Gamificación, stats, import/export	XP/rachas/metastats, estadísticas, notificaciones, importadores, onboarding	13.1–13.5	Producto completo para vos	Sonnet 5 · Haiku 4.5
F14 Release	Hardening, rendimiento, firma e instalador, E2E, macOS, documentación	14.1–14.5	Beta instalable y firmada	Fable 5.1 (auditoría) · Opus 5

Hitos: tras F4 ya repasás con FSRS; tras F6 ya importás tus libros y videos; tras F8 ya generás rutas; tras F9 ya estudiás "a lo Duolingo"; tras F10 ya te preparás para un examen; tras F13 tenés el producto entero; F14 lo deja instalable y firmado. F11 y F12 pueden ir en paralelo (worktrees) con F13.

Las 76 sub-fases, con modelo y esfuerzo

Índice de `prompts.md`. Los títulos están en inglés porque así aparecen en los prompts. "Plan" = empezar en plan mode (Shift+Tab) y aprobar el plan antes de implementar; "Directo" = pegar el prompt y dejar que implemente.

Sub-fase	Título (prompts.md)	Modelo	Esfuerzo	Modo	Depende de
0.1	Repository, CLAUDE.md, permissions and hooks	Sonnet 5	Medio	Directo	—
0.2	Subagents and skills	Sonnet 5	Medio	Directo	0.1
0.3	Extract the specification from the research PDF into docs/spec	Opus 5	Alto	Plan	0.1
0.4	CI skeleton and repository hygiene	Haiku 4.5	Bajo	Directo	0.1
1.1	Monorepo packages and electron-vite app	Sonnet 5	Medio	Plan	0.4
1.2	Process security and typed IPC contract	Opus 5	Alto	Plan	1.1
1.3	app:// and media:// protocols, windows, deep links	Sonnet 5	Alto	Directo	1.2
1.4	Packaging, auto-update, logging and crash reporting	Sonnet 5	Medio	Directo	1.3
1.5	Test infrastructure	Sonnet 5	Medio	Directo	1.4
2.1	Design tokens, Tailwind 4, shadcn on Base UI, dark mode	Sonnet 5	Medio	Directo	1.5
2.2	Application shell, router, command palette, i18n	Sonnet 5	Alto	Plan	2.1
2.3	Core composite components	Sonnet 5	Medio	Directo	2.2
2.4	Mascot, celebrations and sounds	Sonnet 5	Medio	Directo	2.3
2.5	Accessibility and keyboard pass	Sonnet 5	Medio	Directo	2.4
3.1	Domain schema and migrations	Fable 5.1	Extra	Plan	1.5
3.2	Repositories and core ports	Opus 5	Alto	Plan	3.1
3.3	Full-text and hybrid vector search	Opus 5	Alto	Directo	3.2
3.4	Persistent job queue and worker pool	Opus 5	Alto	Plan	3.2
3.5	Blob store, backups, settings and secrets	Sonnet 5	Medio	Directo	3.2
4.1	FSRS-6 scheduler wrapper and regression tests against py-fsrs	Fable 5.1	Max	Plan	3.2
4.2	Importance levels and per-item desired retention	Opus 5	Extra	Plan	4.1
4.3	Daily session composer, overload protection, final drill	Opus 5	Alto	Plan	4.2
4.4	Review screen	Sonnet 5	Alto	Directo	4.3, 2.3
4.5	Exercise → rating mapping, review logs and basic stats	Opus 5	Alto	Directo	4.3
4.6	Optimizer, simulator, leeches, bury, load balancer, easy days	Opus 5	Extra	Plan	4.5, 3.4
5.1	Activity schema (zod per family) and grader utilities	Fable 5.1	Extra	Plan	3.1
5.2	ActivityHost, type registry, shared components, Storybook	Opus 5	Alto	Plan	5.1, 2.3
5.3	Families choice, cards, text_input, cloze (renderers + generators stubs)	Sonnet 5	UltraCode	Directo	5.2
5.4	Families pairs, ordering, categorize, text_mark, disclosure	Sonnet 5	UltraCode	Directo	5.2
5.5	long_text family with AI grading and "Explain my answer"	Opus 5	Alto	Directo	5.2
5.6	Session generator: lesson practice and review variety	Opus 5	Alto	Plan	5.3, 5.4, 5.5, 4.3
6.1	Source library and document parsers (PDF, DOCX, EPUB, PPTX, Markdown/TXT, images)	Sonnet 5	Alto	Plan	3.4, 3.5
6.2	SourceDoc normalization and structural chunking	Opus 5	Alto	Directo	6.1
6.3	Embeddings (local + cloud), hybrid retrieval and reranker	Opus 5	Alto	Directo	6.2, 3.3
6.4	Audio and video ingestion (ffmpeg, local Whisper, keyframes, slide OCR)	Opus 5	Extra	Plan	6.2
6.5	Web pages and YouTube	Sonnet 5	Medio	Directo	6.2, 6.4
6.6	PDF/EPUB reader with highlights → memory items	Sonnet 5	Alto	Plan	6.1, 4.4
7.1	Provider profiles, roles, pricing table and cost log	Opus 5	Extra	Plan	3.5
7.2	Structured outputs, validation/repair loop, versioned prompt files	Opus 5	Alto	Directo	7.1
7.3	Batch API and prompt caching	Opus 5	Alto	Directo	7.2
7.4	Local providers, offline behaviour and fallback policy	Sonnet 5	Medio	Directo	7.1
7.5	AI settings, keys, budget, usage dashboard and evals	Sonnet 5	Medio	Directo	7.1–7.4
8.1	Extraction (P1), synthesis (P2) and deterministic sequencing	Fable 5.1	Max	Plan	6.3, 7.3
8.2	Generation wizard, editable preview and PathSpec freeze	Sonnet 5	Alto	Directo	8.1
8.3	Lesson expansion (P3–P5) in batch with progress and resume	Opus 5	Extra	Plan	8.2, 5.6

Sub-fase	Título (prompts.md)	Modelo	Esfuerzo	Modo	Depende de
8.4	QA gates (P6–P8): citations, faithfulness, coverage, dedupe, Bloom variety, judge	Fable 5.1	Extra	Plan	8.3
8.5	Item bank (P9) and prior-knowledge diagnostic (Elo-lite)	Opus 5	Extra	Plan	8.3, 5.6
8.6	Remediation (P11), regeneration and path versioning	Opus 5	Alto	Directo	8.4, 8.5
9.1	Path map and soft unlocking	Sonnet 5	Alto	Directo	8.2, 4.3, 2.3
9.2	Lesson player: theory and practice	Sonnet 5	Alto	Plan	9.1, 5.6, 8.3
9.3	Reinforcement and checkpoint nodes → memory flow	Opus 5	Alto	Directo	9.2, 4.3, 8.5, 8.6
9.4	AI tutor with citations and "Explain my answer"	Opus 5	Alto	Directo	7.2, 6.3
9.5	Notes editor (Tiptap) with "card from any block"	Opus 5	Alto	Plan	4.4, 6.6
10.1	Exam scheduler ("study toward date X") and readiness	Opus 5	Extra	Plan	4.2, 4.3
10.2	Mock exams and blind mode	Sonnet 5	Alto	Directo	10.1, 8.5, 5.2
10.3	AI grading of open answers (P10) and post-exam analytics → memory	Opus 5	Alto	Directo	10.2, 7.2
10.4	Study plan and exam notifications	Sonnet 5	Medio	Directo	10.1, 13.3
11.1	Audio capture, VAD, storage and playback	Sonnet 5	Alto	Directo	1.2, 3.5
11.2	Azure Pronunciation Assessment and results UI	Opus 5	Alto	Plan	11.1, 7.1, 5.2
11.3	TTS with cache and listening activities	Sonnet 5	Medio	Directo	7.1, 5.2
11.4	Accent Lab (target accent coaching, Irish English first)	Opus 5	Extra	Plan	11.2, 11.3
11.5	STT for answers, dictation, speak activities and text roleplay	Sonnet 5	Alto	Directo	11.1, 7.1, 5.6
12.1	Image provider and asset pipeline	Sonnet 5	Medio	Directo	7.1, 3.5
12.2	Diagrams-as-code and image-occlusion editor	Sonnet 5	Alto	Directo	12.1, 9.5
12.3	Light video explainers (Remotion) and optional generative video	Sonnet 5	Medio	Directo	11.3, 12.2
12.4	Code sandboxes and code activities	Opus 5	Alto	Plan	5.2, 1.2
13.1	XP, streaks, goals, quests, achievements and sober mode	Sonnet 5	Medio	Directo	4.5, 9.2
13.2	Statistics screen (full)	Sonnet 5	Medio	Directo	4.5, 10.1
13.3	Notifications, tray, reminders and global hotkey	Haiku 4.5	Bajo	Directo	4.3, 1.3
13.4	Importers and exporters (Anki .apkg, CSV, RemNote, Markdown/Obsidian, full export)	Opus 5	UltraCode	Plan	3.2, 4.1, 9.5
13.5	Onboarding, settings consolidation and opt-in telemetry	Sonnet 5	Medio	Directo	13.1, 13.4, 7.5
14.1	Security hardening and audit	Fable 5.1	UltraCode	Plan	all previous phases
14.2	Performance and memory pass	Opus 5	Alto	Plan	14.1
14.3	Code signing, installer and auto-update end-to-end	Sonnet 5	Alto	Directo	14.2, 1.4
14.4	E2E Playwright suite and beta checklist	Sonnet 5	UltraCode	Directo	14.3
14.5	macOS build and documentation	Sonnet 5	Medio	Directo	14.4

Totales: Sonnet 5 en 37 sub-fases, Opus 5 en 31, Fable 5.1 en 6, Haiku 4.5 en 2. Esfuerzo: Bajo 2 · Medio 21 · Alto 35 · Extra 11 · Max 2 · UltraCode 5 (5.3, 5.4, 13.4, 14.1, 14.4). 31 sub-fases empiezan en plan mode.

Mapeo de modelos y esfuerzo a Claude Code

Verificado en `code.claude.com/docs` (model-config, workflows, sub-agents, hooks, best-practices) el 2-sep-2026:

Etiqueta en prompts.md	Control real	Cómo se activa	Cuándo usarlo
Haiku 4.5	alias haiku → Claude Haiku 4.5 (200K, USD 1/5)	<code>/model haiku · --model haiku · model: haiku</code> en un subagente	Boilerplate, scaffolding, docs, tests mecánicos, fan-out de tareas pequeñas. No admite el parámetro de esfuerzo (usa extended thinking fijo): las etiquetas de esfuerzo en sus sub-fases son nominales.
Sonnet 5	alias sonnet → Claude Sonnet 5 (1M, USD 2/10); <code>sonnet[1m]</code> para 1M explícito	<code>/model sonnet</code>	Implementación de UI y features con especificación clara; el caballo de batalla.

Etiqueta en prompts.md	Control real	Cómo se activa	Cuándo usarlo
Opus 5	alias opus → Claude Opus 5 (1M, USD 5/25); opusplan = Opus en plan mode y Sonnet al ejecutar	/model opus · /model opusplan	Arquitectura, lógica de dominio compleja, integraciones delicadas, bugs difíciles, refactors.
Fable 5.1	alias fable → Claude Fable 5.1 (1M, USD 10/50); best → el Fable más reciente (u Opus si no está disponible)	/model fable	Decisiones fundacionales (esquema, scheduler, pipeline de generación, auditoría de seguridad). Es el más lento y caro: reservarlo.
Bajo	effort: low	/effort low · --effort low · CLAUDE_CODE_EFFORT_LEVEL=low	Tareas simples y paralelizables; ~50 % menos tokens.
Medio	effort: medium	/effort medium	Trabajo agéntico rutinario con spec clara.
Alto	effort: high (default en la mayoría de los modelos)	/effort high	Razonamiento completo; implementación no trivial.
Extra	effort: xhigh (Fable 5.1/5, Opus 5, Sonnet 5, Opus 4.8/4.7)	/effort xhigh	Trabajo de horizonte largo (30+ min), arquitecturas, migraciones.
Max	effort: max (presupuesto de razonamiento sin límite)	/effort max	Problemas de frontera: el diseño del scheduler, el pipeline de generación, la auditoría.
UltraCode	ultracode = esfuerzo xhigh + orquestación automática de <i>dynamic workflows</i> (decenas de subagentes desde un script que Claude escribe)	/effort ultracode (sesión) · claude --effort ultracode · "ultracode": true en settings · o la palabra clave ultracode: al inicio de un prompt (solo ese turno)	Sweeps paralelos: implementar 10 renderers a la vez, auditar todos los handlers IPC, migrar muchos archivos, verificar hallazgos de forma adversarial. Requiere plan pago con workflows habilitados; hasta 16 agentes concurrentes y 1.000 por run; guía de tamaño en /config (small < 5, medium < 15, large < 50). Es caro: probar primero en una porción.

Notas: el esfuerzo se puede fijar por sub-agente y por skill en el frontmatter (`effort: high`); `/effort auto` borra el nivel guardado; `effortLevel` y `modelSettings` en `settings.json` fijan defaults por modelo; `CLAUDE_CODE_EFFORT_LEVEL` no acepta `ultracode` . La palabra clave `ultracode` en el prompt solo dispara un workflow si la escribís vos en el prompt interactivo (no en `-p` ni en tareas programadas).

Cómo usar prompts.md

1 Fase 0 primero, una sola vez.

Crea el repo, `CLAUDE.md`, `.claude/settings.json` con permisos y hooks (typecheck + tests al terminar cada turno), subagentes (reviewer, tester, security), skills del proyecto, y copia las secciones clave de este PDF a `docs/spec/` como fuente de verdad para Claude.

2 Una sub-fase = una sesión limpia.

`/clear`, `/model` y `/effort` según la cabecera, pegar el prompt entero. Cada prompt es autocontenido: contexto, objetivo, entregables, criterios de aceptación, comandos de verificación y qué NO hacer.

3 Explore → Plan → Code → Commit.

Los prompts de sub-fases grandes piden entrar en plan mode (Shift+Tab) y aprobar el plan (Ctrl+G para editarlo) antes de implementar. Si podés describir el diff en una frase, saltate el plan.

4 Verificación siempre.

Cada prompt termina con "run typecheck + tests + lint; show the evidence". Para sub-fases con UI, captura de pantalla con Playwright; el hook Stop bloquea el cierre del turno si fallan los tests (Claude Code lo ignora tras 8 bloqueos consecutivos).

5 Revisión adversarial antes de dar por cerrada la fase.

"Use a subagent to review the diff against docs/spec/...; report gaps, not style"; o `/code-review` para bugs. Chasing every finding leads to over-engineering: solo lo que afecta corrección o requisitos.

- 6

Contexto limpio.
Si corrigiste dos veces lo mismo, `/clear` y un prompt mejor. `/compact <instrucciones>` en sesiones largas; `/btw` para preguntas que no deben entrar al contexto; `/rewind` para volver a un checkpoint.
- 7

Paralelizar con worktrees.
Fases independientes (F11, F12, F13) en sesiones separadas con `--worktree` ; patrón Writer/Reviewer con dos sesiones para el código crítico.

Configuración de Claude Code que la fase 0 deja lista

Pieza	Contenido
CLAUDE.md	Corto y concreto: comandos (<code>pnpm dev</code> , <code>pnpm test</code> , <code>pnpm typecheck</code> , <code>pnpm e2e</code>), convenciones (ESM, TypeScript strict, Biome, nombres de paquetes, dónde vive cada cosa), reglas del dominio que Claude no puede inferir (campos FSRS 1:1 con ts-fsrs, ids UUIDv7, soft deletes, "core no importa Electron"), gotchas (better-sqlite3 solo en main, CSP, IPC vía contrato), etiqueta de commits, y <code>@docs/spec/*.md</code> importados. Se poda cuando Claude ignora reglas: si el archivo es largo, las reglas se pierden.
.claude/settings.json	<code>permissions.allow</code> para <code>pnpm test/typecheck/lint/build</code> , <code>git commit</code> , <code>git status/diff</code> ; <code>deny</code> para <code>rm -rf</code> , <code>git push --force</code> ,escritura en <code>migrations/</code> sin revisión; hooks: <code>PostToolUse</code> (<code>matcher Edit Write</code>) → <code>biome check --write</code> del archivo; <code>Stop</code> → <code>pnpm typecheck && pnpm test --changed</code> (<code>exit 2</code> bloquea); <code>PreToolUse</code> (<code>Bash</code>) → script que deniega comandos destructivos.
.claude/agents/	<code>reviewer</code> (model: opus, tools: <code>Read/Grep/Glob/Bash</code> ; revisa diff contra spec), <code>tester</code> (model: sonnet; escribe y corre tests), <code>security-reviewer</code> (model: opus; IPC, CSP, claves, inyección), <code>explorer</code> (model: haiku; búsquedas baratas). Frontmatter: <code>name</code> , <code>description</code> , <code>tools</code> , <code>model</code> , <code>effort</code> , <code>permissionMode</code> , <code>maxTurns</code> .
.claude/skills/	<code>fsrs-domain</code> (reglas del scheduler), <code>activity-type</code> (cómo añadir un tipo: archivo + prompt + fixtures + story), <code>ipc-channel</code> (cómo añadir un canal), <code>release</code> (checklist de release con <code>disable-model-invocation: true</code>).
MCP	<code>claude mcp add</code> GitHub (issues/PRs) y, opcionalmente, Playwright para capturas de la app; Context7 o similar para documentación actualizada de librerías.
docs/spec/	Extractos de este PDF en Markdown: decisiones, esquema de datos, fórmulas FSRS, tabla de importancia, taxonomía de actividades y familias de payload, pipeline de generación con prompts P1-P11, matriz de proveedores, diseño de UX. Es lo que los prompts referencian con <code>@docs/spec/...</code> .

14 Costos, riesgos y decisiones abiertas

Cuánto cuesta desarrollar y operar Retenia para vos, qué puede salir mal y qué queda por verificar o decidir antes de fijar precios a terceros.

Costos fijos de desarrollo y distribución

Concepto	USD	Cuándo	Notas
Certificado de firma de código (Certum Cloud Individual)	≈ 139 / año	Antes del primer beta público	Alternativa SSL.com IV/OV 65–75/año + eSigner 20/mes para firmar desde CI. Azure Artifact Signing no disponible para individuos en Argentina.
Apple Developer Program	99 / año	Fase 14 (macOS)	Notarización y distribución fuera del App Store.
Dominio retenia.app / .io	≈ 15–40 / año	Cuando fijas el nombre	Verificar WHOIS y marca (INPI).
Azure AI Speech	0 (F0) → uso	Fase 11	F0: 5 h/mes de STT/PA y 0,5M caracteres de TTS; S0 para producción.
Claude Code (suscripción o API)	según plan	Todo el desarrollo	Las sub-fases con Fable 5.1 y UltraCode son las que más consumen; están reservadas a decisiones fundacionales y sweeps.
Cuentas de IA con créditos prepagos	20–50 / mes	Desde la fase 7	Anthropic + Google AI Studio (pago) + Azure; OpenRouter opcional para probar modelos chinos sin abrir cuentas.
GitHub (repo privado, Actions)	0–4 / mes	Fase 0	Runners Windows consumen minutos más rápido; releases como draft.
Sentry / PostHog	0 (free tiers)	Fase 1 / 13	Solo con usuarios reales.

Costo operativo mensual de IA (resumen)

Escenario	USD / mes	Composición
Uso liviano (1 ruta/mes, repaso diario, poca voz)	≈ 6–12	Sonnet 5 batch + Gemini Flash + Azure dentro del free tier + local
Uso intensivo recomendado (2 rutas, 30 lecciones, 500 correcciones, 300 chats, 7–15 h de voz, 100 imágenes)	≈ 20–34	Texto 9–14 + voz 6–14 + media 5–6 + video ligero 0,20
Premium (Opus 5 en rutas, imágenes Pro, Mistral OCR, real-time voice)	≈ 45–70	Sin video generativo
Con 2 minutos de video generativo al mes	+ 25–45	Solo opt-in con costo por clip visible

Riesgos principales

Riesgo	Probabilidad · impacto	Mitigación
Volatilidad de precios y modelos (Gemini ×2 en 2027; DeepSeek ×10 en agosto; deprecaciones de Imagen 4, Sora, gpt-image-1.5)	Alta · medio	Tabla de precios versionada y editable en la app; capa de proveedores intercambiables; presupuesto con alertas; re-verificar trimestralmente.
Calidad pedagógica insuficiente de la generación (shallow, recall-only, alucinaciones)	Media · alto	QA gates (fidelidad por claims, cobertura, variedad Bloom, juez pedagógico), preview editable, "Reportar error" con cita, eval propio de 50 ítems en español.
Alcance demasiado grande para una persona	Alta · alto	Plan por fases con hitos usables; MVP en 12–16 semanas; V2/Later diseñados pero no construidos; UltraCode para sweeps.
Copyright de fuentes (Udemy, YouTube, libros)	Media · medio	Procesamiento local, sin redistribución, sin yt-dlp en el producto base, términos claros; el usuario aporta sus archivos.
Privacidad: las fuentes salen a APIs	Media · medio	Proveedores por defecto que no entrenan con datos (Anthropic, Google pago), opción "solo proveedor X", redacción de PII opcional, modo local.
Firma, SmartScreen y antivirus	Alta · medio	Certificado desde el primer beta; mismo cert; sidecars descargados bajo demanda; sin Python en v1.
Deuda de sincronización al comercializar	Media · alto	Esquema sync-ready y repos por puerto desde F3; spike de PowerSync temprano.
Evaluación de pronunciación imperfecta fuera de en-US (sin prosodia, sin IPA) y sin en-IE	Alta · medio	Accent Lab por capas; mostrar "inteligibilidad" no "acento"; ELSA/SpeechSuper como upgrade para inglés; GOP local en v2.

Riesgo	Probabilidad · impacto	Mitigación
Fatiga de gamificación o percepción de "electronic whip"	Media · bajo	Modo sobrio; nunca castigar; notificaciones limitadas; métricas de sustancia visibles.
Dependencias inestables (Vidstack, sqlite-vec pre-v1, EmbedPDF 3, TS 7 API, AI SDK majors)	Alta · bajo	Wrappers finos, versiones exactas, Renovate mensual, codemods.

Decisiones abiertas (para tomar durante el desarrollo)

1. **Nombre y marca:** Retenia es de trabajo; verificar WHOIS y buscar en INPI antes de registrar el dominio y diseñar el logo.
2. **Optimizador FSRs:** `@open-spaced-repetition/binding` (napi, verificar prebuild win32-x64) vs `fsrs-browser` (WASM, sin binarios nativos).
3. **Reproductor de video:** media-chrome (seguro) vs Vidstack 1.x (mejor API React, actividad incierta): decidir en la sub-fase 6.4 tras revisar commits.
4. **Whisper local:** sherpa-onnx (un addon para STT + VAD + TTS) vs whisper-cli bundleado (sin compilar): decidir en 6.4 según facilidad de empaquetado con CUDA.
5. **Embeddings por defecto:** EmbeddingGemma (mejor español, 300M) vs multilingual-e5-small (más rápido en CPU): medir en 6.3 con tu corpus.
6. **Locale de referencia para acentos:** en-GB vs en-US para el scoring segmental del "Accent Lab" irlandés (en-US da NBest y prosodia; en-GB vocales más cercanas).
7. **Tier premium:** si Opus 5 se ofrece como opción por ruta ("alta calidad", $\approx 2\times$ costo) o solo para explicar errores.
8. **Sync futuro:** PowerSync (backend propio Postgres) vs Evolu (E2E, esquema propio): decidir antes de la capa comercial; el esquema de F3 es compatible con ambos.
9. **Video ligero:** si Remotion (gratis para individuos) se incluye en v1 o queda para V2.
10. **Telemetría:** PostHog opt-in desde el beta o solo Sentry hasta tener usuarios.

Qué re-verificar antes de fijar precios o depender de ello

- Página oficial de precios de DeepSeek (servía otro contenido): confirmar USD 0,22/0,66 valle y 0,44/1,32 pico (V4 Flash), max output 384K.
- Multiplicadores de long context de OpenAI ($> 272K$ tokens: $2\times$ input / $1,5\times$ output) y precio de Flex.
- gpt-image-2 por imagen (OpenAI solo publica USD 30/M tokens de salida); Ideogram (página 404); Veo 3.1 Lite y Seedance 2.0 (terceros).
- Relación créditos $\leftrightarrow$ caracteres de ElevenLabs por modelo; nombres de voces en-IE; precios de Gemini TTS en Cloud.
- Azure: formato de nombres de fonema fuera de en-US; si el add-on de prosodia se cobra en F0; lista de regiones con Pronunciation Assessment; comportamiento de content assessment en el SDK actual.
- DeepL Free (500k caracteres/mes) y Pro; Marker v2 (umbral de licencia comercial USD 2M vs 5M); MiniMax y PlayHT (precios no públicos).
- Free tier de Gemini (RPM/RPD por modelo ya no se publica en la doc: mirar el dashboard de AI Studio).
- Estado real de `node:sqlite` en Electron 44; EP DirectML/CUDA en onnxruntime-node 1.29; versión y estado de faster-whisper, promptfoo, papaparse, @mozilla/readability.
- Comportamiento de Certum SimplySign en CI desatendido; estructura interna del export "RemNote complete".
- Percepciones impositivas argentinas sobre compras de créditos de IA en moneda extranjera y aceptación de tarjetas locales en cada consola (Stripe 3DS/AVS).

- Throughput real de Qwen3.5 9B / Gemma 4 12B / Parakeet / whisper-turbo en la RTX 4070 Super (todas las cifras son extrapolaciones).
- Cifras de meta-análisis citadas de memoria (Rowland 2014, Adesope 2017; Mizumoto & Eguchi 2023) antes de usarlas en marketing.
- RemNote: "Flashcards 2.0" y "Look Back" no aparecen con esos nombres en la ayuda ni en el changelog; SuperMemo: detalles de Sleep Chart, Tasklist y Advanced English (páginas con 403).

15 Glosario y fuentes

Términos que aparecen en el documento y en prompts.md, y las fuentes primarias consultadas (todas verificadas por acceso directo el 1-2 de septiembre de 2026; las que devolvieron error están indicadas en cada sección).

Glosario

Activity engine	Paquete que renderiza, califica y registra cualquier tipo de actividad a partir de un JSON con <code>type</code> y <code>payload.family</code> .
Batch API	Modo asíncrono de Anthropic/OpenAI/Gemini con 50 % de descuento; resultados típicamente en menos de una hora, máximo 24 h.
Blind solve	Validación en la que un segundo modelo resuelve el ítem sin ver la clave; si discrepa, el ítem se marca para revisión.
Blueprint	Tabla de especificaciones de un examen: tema × nivel Bloom × dificultad con pesos.
BYOK	Bring Your Own Key: el usuario carga sus propias claves de API.
Contextual retrieval	Técnica de Anthropic: anteponer a cada chunk un breve contexto generado por LLM para mejorar la recuperación.
Desired retention (DR)	Probabilidad de recuerdo objetivo con la que FSRS programa el próximo repaso (0,90 por defecto).
DSR	Difficulty, Stability, Retrievability: las tres variables del modelo de memoria de FSRS.
Elo-lite	Estimador de habilidad por módulo inspirado en Elo (Pelánek 2016) usado en el diagnóstico.
Final drill	Repetir al final de la sesión todo lo calificado por debajo de Good hasta aprobarlo (SuperMemo).
FSRS-6	Free Spaced Repetition Scheduler, versión 6: algoritmo abierto con 21 parámetros optimizables.
Fuzz	Aleatorización pequeña del intervalo para que las tarjetas creadas juntas no venzan juntas.
Importancia	Nivel por ítem (Urgente, Alta, Normal, Mantenimiento, Pausado) que fija DR, tope de intervalo, orden y política de postpone.
Item bank	Banco de ítems con metadatos (módulo, Bloom, dificultad, uso) del que se muestrean diagnósticos, refuerzos y exámenes.
Knowledge graph	Grafo de conceptos con aristas de prerrequisito y relación construido en la síntesis de la ruta.
Leech	Tarjeta olvidada repetidamente (8 lapsos en Anki, 4 en RemNote) que conviene reescribir o suspender.
Load balancer	Dentro del rango de fuzz, elegir el día con menos repasos.
Mercy / Postpone	Redistribuir o aplazar repasos atrasados protegiendo lo importante (SuperMemo).
Misconception	Error conceptual frecuente, extraído por la IA y usado para distractores y remediación.
Need to Learn	Cola de ítems candidatos aún no programados (RemNote).
Outbox	Tabla que registra cambios locales pendientes de sincronizar (vacía en v1).
Parallel forms	Dos versiones equivalentes de un examen (A/B) que no comparten ítems.
PathSpec	Especificación congelada de una ruta (orden e ids), versionada.
Pronunciation Assessment (PA)	Servicio de Azure que puntúa precisión, fluidez, completitud y prosodia por fonema/palabra.
Readiness	Preparación estimada para un examen: $\sum R$ el día del examen ponderada por blueprint.
Retrievability (R)	Probabilidad de recordar un ítem ahora.
RRF	Reciprocal Rank Fusion: combinar rankings de BM25 y vectorial.
Stability (S)	Intervalo tras el cual R cae a 0,9.
Structured outputs	Salida JSON garantizada por esquema (Anthropic, OpenAI, Gemini).
True retention	Porcentaje real de aciertos en el primer repaso del día de tarjetas maduras.
UltraCode	Ajuste de Claude Code: esfuerzo xhigh + orquestación automática de workflows con muchos subagentes.
utilityProcess	Proceso hijo de Electron con Node para trabajo pesado fuera del main.
xAPI	Formato de eventos de aprendizaje (verb, object, result) que usan H5P y SCORM/cmi5.

Fuentes principales

Repeticón espaciada y ciencia del aprendizaje

- <https://github.com/open-spaced-repetition/awesome-fsrs/wiki/The-Algorithm> · <https://expertium.github.io/Algorithm.html> · <https://github.com/open-spaced-repetition/ts-fsrs> · <https://github.com/open-spaced-repetition/fsrs-rs> · <https://github.com/open-spaced-repetition/srs-benchmark> · <https://expertium.github.io/Benchmark.html> · <https://github.com/open-spaced-repetition/fsrs-vs-sm17> · <https://github.com/open-spaced-repetition/fsrs4anki/wiki/The-optimal-retention> · <https://github.com/open-spaced-repetition/fsrs4anki-helper> · <https://expertium.github.io/Retention.html>
- <https://docs.ankiweb.net/deck-options.html> · <https://docs.ankiweb.net/filtered-decks.html> · <https://docs.ankiweb.net/leeches.html> · <https://docs.ankiweb.net/stats.html> · <https://faq.ankiweb.net/what-spaced-repetition-algorithm.html> · <https://github.com/ankitects/anki/releases>
- https://supermemo.guru/wiki/Algorithm_SM-17 · https://supermemo.guru/wiki/Algorithm_SM-20 · https://supermemo.guru/wiki/Two_components_of_memory · https://supermemo.guru/wiki/Final_drill · <https://super-memory.com/help/priority.htm> · <https://super-memory.com/help/postpone.htm> · <https://supermemo.store/products/supermemo-20-upgrade> · <https://www.supermemo.com/en/blog/twenty-rules-of-formulating-knowledge>
- <https://help.remnote.com/en/articles/9102040-understanding-the-exam-scheduler> · <https://help.remnote.com/en/articles/9124137-the-fsrs-spaced-repetition-algorithm> · <https://help.remnote.com/en/articles/7950982-setting-priorities-and-disabling-flashcards> · <https://help.remnote.com/en/articles/15724936-guided-learn-mode> · <https://help.remnote.com/en/articles/10103884-ai-tutor-chat> · <https://feedback.remnote.com/rss/channel.xml> · <https://www.remnote.com/pricing>
- <https://github.com/fasiha/ebisu> · <https://aclanthology.org/P16-1174.pdf> (HLR) · <https://mochi.cards/docs/reviewing/fsrs/> · <https://www.brainscape.com/academy/confidence-based-repetition-definition/>
- <https://www.aft.org/ae/fall2013/dunlosky> · <https://bjorklab.psych.ucla.edu/research/> · <https://gwern.net/spaced-repetition> · <https://www.learningscientists.org/downloadable-materials> · <https://www.kinnu.xyz/blog/research/what-we-found-in-our-10000-person-learning-experiments/>

Actividades interactivas

- <https://www.articulatesupport.com/article/Storyline-360-Adding-Form-Based-Questions> · <https://www.articulatesupport.com/article/Storyline-360-Adding-Freeform-Questions> · <https://help.rise.com/en/articles/2059261-choose-lesson-and-block-types>
- <https://helpx.adobe.com/captivate/classic/set-question-slides.html> · <https://helpx.adobe.com/captivate/classic/whats-new.html> · <https://helpx.adobe.com/captivate/help/create-interactions.html> · <https://blog.lilybiri.com/tips-learning-interactions>
- https://docs.moodle.org/en/Question_types · https://docs.moodle.org/en/Third-party_question_types · <https://docs.moodle.org/en/Activities> · https://docs.moodle.org/en/Game_module · https://docs.moodle.org/en/CodeRunner_question_type · <https://docs.stack-assessment.org/en/> · https://moodle.org/plugins/mod_vpl · https://moodle.org/plugins/block_xp · https://moodle.org/plugins/mod_minilesson
- <https://h5p.org/content-types-and-applications> · <https://h5p.org/interactive-video> · <https://h5p.org/branching-scenario> · <https://h5p.org/licensing> · <https://github.com/tunapanda/h5p-standalone> · <https://github.com/Lumieducation/H5P-Nodejs-library>
- <https://blog.duolingo.com/learning-how-to-help-you-learn-introducing-birdbrain> · <https://blog.duolingo.com/large-language-model-duolingo-lessons/> · <https://blog.duolingo.com/new-duolingo-home-screen-design> · <https://blog.duolingo.com/duolingo-max> · <https://blog.duolingo.com/duolingo-o-score/> · <https://blog.duolingo.com/chess-course> · <https://www.lennysnewsletter.com/p/how-duolingo-reignited-user-growth>
- <https://github.com/Khan/perseus> · <https://wordwall.net/features> · <https://www.educaplay.com/> · <https://learningapps.org/createApp.php> · <https://elsaspeak.com/en/> · <https://www.speak.com/> · <https://praktika.ai/> · <https://www.boldvoice.com/>

Rutas con IA y diseño instruccional

- <https://support.google.com/notebooklm/answer/16213268> · <https://support.google.com/notebooklm/answer/16215270> · <https://support.google.com/notebooklm/answer/16958963> · <https://blog.google/innovation-and-ai/products/gemini-notebook/notebooklm-gemini-notebook/> · <https://research.google/blog/learn-your-way-reimagining-textbooks-with-generative-ai/> · <https://arxiv.org/abs/2412.16429> (LearnLM) · <https://openai.com/index/chatgpt-study-mode/> · <https://blog.khanacademy.org/new-ai-tools-bring-interactive-diagrams-and-targeted-practice-thanks-to-khan-academy-partnership-with-google-org/>
- <https://www.anthropic.com/engineering/contextual-retrieval> · <https://arxiv.org/abs/2307.15337> (Skeleton-of-Thought) · <https://arxiv.org/abs/2303.17651> (Self-Refine) · <https://arxiv.org/abs/2309.11495> (CoVe) · <https://arxiv.org/abs/2305.14251> (FActScore) · <https://arxiv.org/abs/2411.15594> (LLM-as-a-judge)
- <https://www.fi.muni.cz/~xpelanek/publications/CAE-elo.pdf> · https://www.aleks.com/about_aleks/knowledge_space_theory · <https://aclanthology.org/2020.tacl-1.17/> (DET) · <https://arxiv.org/abs/2405.02985> (Henkel, grading) · <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0290691> (Cheung, MCQ) · <https://arxiv.org/abs/2307.16338> (distractores) · <https://arxiv.org/html/2506.22303v1> (DLELP) · <https://arxiv.org/abs/2105.15106> (knowledge tracing survey)
- <https://www.niu.edu/citl/resources/guides/instructional-guide/gagnes-nine-events-of-instruction.shtml> · <https://primmportal.com/> · <https://www.nbme.org/item-writing-guide> · <https://www.coe.int/en/web/common-european-framework-reference-languages/table-1-cefr-3.3-common-reference-levels-global-scale>

Proveedores de IA (precios verificados)

- <https://platform.claude.com/docs/en/about-claude/pricing> · <https://platform.claude.com/docs/en/models/overview> · <https://platform.claude.com/docs/en/build-with-claude/prompt-caching> · <https://platform.claude.com/docs/en/build-with-claude/batch-processing> · <https://platform.claude.com/docs/en/build-with-claude/streaming>

- e.com/docs/en/build-with-claude/pdf-support · <https://platform.claude.com/docs/en/build-with-claude/structured-outputs> · <https://platform.claude.com/docs/en/api/rate-limits> · <https://www.anthropic.com/supported-countries>
- <https://developers.openai.com/api/docs/pricing> · <https://developers.openai.com/api/docs/models> · <https://developers.openai.com/api/docs/guides/structured-outputs> · <https://developers.openai.com/api/docs/models/gpt-image-2> · <https://developers.openai.com/api/docs/guides/speech-to-text> · <https://developers.openai.com/api/docs/guides/text-to-speech>
 - <https://ai.google.dev/gemini-api/docs/pricing> · <https://ai.google.dev/gemini-api/docs/models> · <https://ai.google.dev/gemini-api/docs/tokens> · <https://ai.google.dev/gemini-api/docs/document-processing> · <https://ai.google.dev/gemini-api/docs/image-generation> · <https://ai.google.dev/gemini-api/docs/audio> · <https://ai.google.dev/gemini-api/docs/terms> · <https://ai.google.dev/gemini-api/docs/available-regions>
 - <https://api-docs.deepseek.com/news/news260813> · <https://platform.kimi.ai/docs/pricing/chat> · <https://www.alibabacloud.com/help/en/model-studio/model-pricing> · <https://docs.z.ai/guides/overview/pricing> · <https://x.ai/api> · <https://mistral.ai/pricing/api/> · <https://openrouter.ai/docs/faq> · <https://docs.fireworks.ai/serverless/pricing> · <https://www.together.ai/pricing>
 - <https://arena.ai/leaderboard/text> · <https://artificialanalysis.ai/leaderboards/models> · <https://ai-sdk.dev/providers/ai-sdk-providers> · <https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data> · <https://vercel.com/changelog/ai-sdk-7> · <https://docs.ollama.com/api/openai-compatibility> · <https://lmstudio.ai/docs/app/api/endpoints/openai> · <https://ollama.com/library>

Voz

- <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-pronunciation-assessment> · <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-support?tabs=pronunciation-assessment> · <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/pronunciation-assessment-tool> · <https://prices.azure.com/api/retail/prices> (Speech, eastus) · <https://azure.microsoft.com/en-us/pricing/details/cognitive-services/speech-services/> · <https://github.com/microsoft/cognitive-services-speech-sdk-js>
- <https://www.speechace.com/api-plans/> · <https://www.speechsuper.com/pricing.html> · <https://elsaspeak.com/en/elsa-api/> · <https://docs.languagelive.com/confidence.ai/> · <https://accent-strength.boldvoice.com/> · <https://arxiv.org/html/2606.15325> (LLMs de audio y pronunciación) · <https://arxiv.org/html/2506.02080v2> (GOP)
- <https://deepgram.com/pricing> · <https://www.assemblyai.com/pricing> · <https://elevenlabs.io/pricing/api> · <https://elevenlabs.io/docs/overview/models> · <https://cloud.google.com/text-to-speech/pricing> · <https://aws.amazon.com/polly/pricing/> · <https://console.groq.com/docs/speech-to-text>
- <https://github.com/k2-fsa/sherpa-onnx> · <https://github.com/ggml-org/whisper.cpp> · <https://github.com/SYSTRAN/faster-whisper> · <https://github.com/m-bain/whisperX> · <https://huggingface.co/nvidia/parakeet-tdt-0.6b-v3> · <https://huggingface.co/hexgrad/Kokoro-82M> · <https://github.com/researcherai/chatbox> · <https://github.com/ricky0123/vad> · <https://github.com/ianprime0509/pitchy> · <https://github.com/pipecat-ai/pipecat> · <https://livekit.com/pricing>

Imagen, video, embeddings, parsing

- <https://blog.google/innovation-and-ai/technology/ai/nano-banana-2/> · <https://ai.google.dev/gemini-api/docs/imagen> · https://docs.bfl.ai/quick_start/pricing · <https://www.recraft.ai/pricing?tab=api> · <https://docs.x.ai/developers/models/grok-imagine-image> · <https://vibedex.ai/blog/best-ai-text-rendering-2026> · <https://pricepertoken.com/image>
- <https://mermaid.js.org/> · <https://github.com/excalidraw/excalidraw> · <https://tldraw.dev/legal/tldraw-license> · <https://www.remotion.dev/docs/license> · <https://cloud.google.com/vertex-ai/generative-ai/pricing> (Veo) · <https://fal.ai/models> (Kling, Hailuo, Seedream)
- <https://docs.voyageai.com/docs/pricing> · <https://developers.openai.com/api/docs/guides/embeddings> · <https://cohere.com/pricing> · <https://github.com/asg017/sqlite-vec> · <https://lancedb.github.io/lancedb/> · <https://huggingface.co/blog/embeddinggemma> · <https://github.com/huggingface/transformers.js>
- <https://mistral.ai/pricing> (OCR) · <https://www.llamaindex.ai/llamaparse> · <https://docs.datalab.to/> · <https://github.com/docling-project/docling> · <https://github.com/PaddlePaddle/PaddleOCR> · <https://github.com/naptha/tesseract.js> · <https://github.com/kepano/defuddle> · <https://jina.ai/reader/> · <https://www.firecrawl.dev/pricing> · <https://www.youtube.com/static?template=terms> · <https://pyodide.org/en/stable/>

Stack de escritorio

- <https://releases.electronjs.org/> · <https://www.electronjs.org/docs/latest/tutorial/security> · <https://www.electronjs.org/docs/latest/api/protocol> · <https://www.electronjs.org/docs/latest/api/utility-process> · <https://www.electronjs.org/docs/latest/api/safe-storage> · <https://electron-vite.org/blog/> · <https://github.com/electron-userland/electron-builder/releases> · <https://www.electron.build/code-signing-win.html> · <https://learn.microsoft.com/en-us/azure/trusted-signing/quickstart> · <https://www.sslmentor.com/certum/certumcodecloudindividual> · <https://www.ssl.com/guide/esigner-pricing-for-code-signing/>
- <https://www.react.dev/blog/2025/10/01/react-19-2> · <https://ui.shadcn.com/docs/changelog> · <https://motion.dev/docs/react-upgrade-guide> · <https://tiptap.dev/docs/editor/getting-started/overview> · <https://www.blocknotejs.org/pricing> · <https://www.embedpdf.com/> · <https://github.com/johnfactotum/fooliate.js> · <https://github.com/vidstack/player> · <https://wavesurfer.xyz/docs/plugins/record/>
- <https://github.com/WiseLibs/better-sqlite3/releases> · <https://orm.drizzle.team/docs/sqlite/latest-releases> · <https://docs.powersync.com/client-sdk-references/node> · <https://www.evolu.dev/> · <https://github.com/vlcn-io/cr-sqlite> · <https://blog.pyodide.org/posts/314-release/> · <https://webcontainers.io/enterprise> · <https://github.com/laverdet/isolated-vm>
- <https://eikowagenknecht.com/posts/understanding-the-anki-apkg-format/> · <https://help.remnote.com/en/articles/7898019-exporting-notes> · <https://www.stephenmwangi.com/obsidian-spaced-repetition/flashcards/cloze-cards/> · <https://www.infoq.com/news/2026/08/typescript-7-released/> · <https://docs.sentry.io/platforms/javascript/guides/electron/> · <https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/> · <https://www.electronjs.org/docs/latest/tutorial/accessibility>

Claude Code

- <https://code.claude.com/docs/en/model-config> · <https://code.claude.com/docs/en/workflows> · <https://code.claude.com/docs/en/sub-agents> · <https://code.claude.com/docs/en/hooks> · <https://code.claude.com/docs/en/settings> · <https://code.claude.com/docs/en/best-practices> · <https://code.claude.com/docs/en/skills> · <https://code.claude.com/docs/en/permission-modes>